

AI for A0

We Are Gagging

September 12, 2025

Contents

1	Giới thiệu về Python cho Trí tuệ Nhân tạo	1
1.1	Cơ bản về Python	1
1.1.1	Vòng lặp và Cấu trúc điều khiển	1
1.1.2	Hàm trong Python	2
1.2	NumPy và Pandas để xử lý dữ liệu	2
1.2.1	NumPy	2
1.2.2	Pandas	3
1.3	Matplotlib và Seaborn để trực quan hóa dữ liệu	4
1.3.1	Matplotlib	4
1.3.2	Seaborn	4
1.4	Scikit-learn	5
1.4.1	Huấn luyện một mô hình đơn giản	5
1.5	Cơ bản về PyTorch	6
1.5.1	Giới thiệu về Tensor	6
1.6	Cơ bản về tái tạo kết quả	7
1.6.1	Thiết lập seed	7
1.7	Huấn luyện mô hình trên CPU và GPU	7
1.8	Sử dụng Weights and Biases để theo dõi thực nghiệm	8
1.9	Kết luận	8
2	Học có giám sát – Supervised Learning	11
2.1	Giới thiệu	11
2.2	Linear Regression (Hồi quy tuyến tính)	11
2.3	Logistic Regression (Hồi quy logistic)	12
2.4	K-Nearest Neighbors (K-NN)	12
2.5	Decision Trees (Cây quyết định)	13
2.6	Random Forests (Rừng ngẫu nhiên)	13
2.7	Gradient Boosting	13

2.8	Support Vector Machines (SVM)	14
3	Học không giám sát – Unsupervised Learning	15
3.1	Phân cụm K-Means	16
3.1.1	Giới thiệu	16
3.1.2	Thuật toán Hoạt động Như Thế Nào	16
3.1.3	Ứng Dụng Thực Tế	17
3.1.4	Cải tiến và Biến thể	17
3.1.5	Ví dụ Python với <code>scikit-learn</code>	17
3.2	Phân Tích Thành Phần Chính (PCA)	18
3.2.1	Giới thiệu về Phân Tích Thành Phần Chính (PCA)	18
3.2.2	Giải thích Thuật toán	19
3.2.3	Ví dụ Minh Họa	20
3.2.4	Ứng Dụng Thực Tế	21
3.3	t-SNE, MAP, và những cách giảm chiều dữ liệu khác	21
3.3.1	Giới thiệu	21
3.3.2	t-SNE: t-distributed Stochastic Neighbor Embedding	22
3.3.3	MAP: Manifold Approximation and Projection	23
3.3.4	Các phương pháp giảm chiều khác	24
3.3.5	Ứng dụng thực tiễn	25
3.4	Phân Cụm DBSCAN	25
3.4.1	Giới thiệu	25
3.4.2	Giải thích thuật toán	25
3.4.3	Ví dụ minh hoạ	26
3.4.4	Ứng dụng thực tiễn	27
3.5	Phân Cụm Phân Cấp (Hierarchical Clustering)	28
3.5.1	Giới thiệu	28
3.5.2	Cách Thuật Toán Hoạt Động	28
3.5.3	Ví dụ Minh Họa	29
3.5.4	Ứng Dụng Thực Tiễn	30
4	Đánh Giá Trong Trí Tuệ Nhân Tạo - Evaluation	31
4.1	Các Chỉ Số Đánh Giá Mô Hình	31
4.1.1	Độ Chính Xác (Accuracy)	31
4.1.2	Độ Chính Xác Của Dự Đoán Dương (Precision)	32
4.1.3	Khả Năng Thu Hồi (Recall)	32

4.1.4	F1-Score	32
4.1.5	So Sánh Giữa Các Chỉ Số	33
4.2	Hiện Tượng Underfitting và Overfitting	33
4.2.1	Underfitting	33
4.2.2	Overfitting	33
4.3	Tinh Chỉnh Siêu Tham Số (Hyperparameter Tuning)	34
4.3.1	Tại Sao Cần Tinh Chỉnh Siêu Tham Số?	34
4.3.2	Tìm Kiếm Lưới (Grid Search)	35
4.3.3	Tìm Kiếm Ngẫu Nhiên (Random Search)	35
4.4	Xác Thực Chéo (Cross-Validation)	36
4.4.1	K-Fold Cross-Validation	36
4.5	Ma Trận Nhầm Lẫn và Đường Cong ROC	36
4.5.1	Ma Trận Nhầm Lẫn	36
4.5.2	Đường Cong ROC và AUC	37
5	Neural Networks	39
5.1	Kiến trúc cơ bản về Perceptron	39
5.2	Thuật toán Gradient Descent	41
5.3	Lan truyền ngược (Backpropagation)	42
5.4	Hàm kích hoạt (Activation Functions)	44
5.5	Hàm mất mát (Cost Functions)	45
6	Deep learning	47
6.1	Perceptron Đa Lớp (MLP)	47
6.1.1	MLP là gì?	47
6.1.2	Các thành phần chính	48
6.1.3	Cài đặt MLP bằng PyTorch	49
6.1.4	Huấn luyện MLP trên MNIST	49
6.1.5	Tại sao MLP hoạt động?	50
6.1.6	Mẹo thực hành cho cuộc thi	51
6.1.7	Bài tập thực hành	51
6.2	Stochastic Gradient Descent (SGD) và Mini-Batch Gradient Descent	52
6.2.1	Tổng quan về Gradient Descent	52
6.2.2	Các biến thể chính	53
6.2.3	Lựa chọn batch size	54

6.3	Phương pháp Momentum (Adam và AdamW)	55
6.3.1	Tại sao cần phương pháp Momentum?	55
6.3.2	Các phương pháp Momentum phổ biến	56
6.3.3	Khi nào sử dụng Adam/AdamW?	57
6.3.4	Thực hành với PyTorch	58
6.3.5	Mẹo thi đấu IOAI	58
6.4	Tốc độ học thích ứng (Adaptive Learning Rates)	59
6.4.1	Tại sao cần tốc độ học thích ứng?	59
6.4.2	Các phương pháp chính	60
6.4.3	Thực hành với PyTorch	62
6.4.4	Mẹo thi đấu IOAI	62
6.5	Sự Hội Tụ và Tốc Độ Học	63
6.5.1	Tốc độ học là gì?	63
6.5.2	Các vấn đề về hội tụ	64
6.5.3	Cách chọn LR phù hợp	65
6.5.4	Theo dõi sự hội tụ	66
6.5.5	Mẹo thi đấu IOAI	66
6.6	Điều chuẩn Trọng số (Weight Regularization)	68
6.6.1	Tại sao cần điều chuẩn trọng số?	68
6.6.2	Các phương pháp chính	68
6.6.3	Cài đặt trong PyTorch	69
6.6.4	Khi nào nên sử dụng?	70
6.6.5	Mẹo thi đấu IOAI	70
6.7	Dừng Sớm (Early Stopping)	71
6.7.1	Early Stopping là gì?	71
6.7.2	Tại sao cần Early Stopping?	72
6.7.3	Cài đặt trong PyTorch	72
6.7.4	Cách sử dụng hiệu quả	75
6.7.5	Mẹo thi đấu IOAI	75
6.8	Dropout và Gaussian Noise	76
6.8.1	Dropout là gì?	76

6.8.2	Gaussian Noise	77
6.8.3	Khi nào sử dụng?	79
6.8.4	Mẹo thi đấu IOAI	79
6.9	Khởi tạo Trọng số (Weight Initialization)	81
6.9.1	Tại sao cần khởi tạo đúng cách?	81
6.9.2	Các phương pháp phổ biến	81
6.9.3	Thực hành với PyTorch	82
6.9.4	Mẹo thi đấu IOAI	83
6.10	Chuẩn hóa theo Batch (Batch Normalization)	84
6.10.1	BatchNorm là gì?	85
6.10.2	Tại sao cần BatchNorm?	85
6.10.3	Cài đặt trong PyTorch	85
6.10.4	Mẹo thi đấu IOAI	86
6.11	Autoencoders và Sparse Encoders	87
6.11.1	Autoencoder là gì?	87
6.11.2	Sparse Encoders	88
6.11.3	Ứng dụng thực tiễn	90
6.11.4	Mẹo thi đấu IOAI	91

7 NLP 93

7.1	Word Embedding: Word2Vec và GloVe	93
7.1.1	1. Ý tưởng trực quan	93
7.1.2	2. Mô hình Word2Vec	93
7.1.3	3. Mô hình GloVe	94
7.1.4	4. Code mẫu	95
7.1.5	5. Bài tập thực hành	96
7.2	Cơ bản về Transformer và Attention Mechanism	97
7.2.1	1. Ý tưởng trực quan	97
7.2.2	2. Scaled Dot-Product Attention	98
7.2.3	3. Multi-Head Attention	98
7.2.4	4. Code mẫu với PyTorch	98
7.2.5	5. Bài tập thực hành	100
7.3	Thực hành Phân loại Văn bản	100
7.3.1	1. Quy trình chung	100

7.3.2	2. Phương pháp cơ bản	101
7.3.3	3. Code mẫu	101
7.3.4	4. Bài tập thực hành	103
7.4	Giới thiệu các Mô hình NLP Đã được Pre-train: BERT và GPT	104
7.4.1	1. Tại sao cần Pre-trained Models?	104
7.4.2	2. BERT (Bidirectional Encoder Representations from Transformers)	104
7.4.3	3. GPT (Generative Pre-trained Transformer)	105
7.4.4	4. Code mẫu	106
7.4.5	5. Bài tập thực hành	107
7.5	Thực hành Question Answering với Mô hình Pre-trained	107
7.5.1	1. Ý tưởng trực quan	107
7.5.2	2. Công thức toán học nền tảng	108
7.5.3	3. Fine-tuning BERT cho Extractive QA	108
7.5.4	4. Code mẫu với Hugging Face & PyTorch	108
7.5.5	5. Bài tập thực hành	110
7.6	Fine-Tuning Mô hình: Phương pháp và Giới hạn (LoRA, Adapters, v.v.)	111
7.6.1	1. Giới thiệu	111
7.6.2	2. LoRA: Low-Rank Adaptation	111
7.6.3	3. Adapters	112
7.6.4	4. Code mẫu với Hugging Face	112
7.6.5	5. Bài tập thực hành	113
7.7	Giới thiệu về Large Language Models (LLMs)	114
7.7.1	Ý tưởng chính đằng sau LLMs	114
7.7.2	Transformer và Attention	114
7.8	Xây dựng chatbot đơn giản sử dụng NLP	115
7.8.1	Mô hình hoạt động cơ bản của chatbot	115
7.8.2	Tiền xử lý văn bản (Text Preprocessing)	115
7.8.3	Nhận diện ý định (Intent Detection)	116
7.8.4	Sinh câu trả lời (Response Generation)	116
7.8.5	Tích hợp thành chatbot hoàn chỉnh	117
7.8.6	Mở rộng: Dùng mô hình GPT	117
7.8.7	Tổng kết	118

7.9	LLM Agents: Cơ bản và Ứng dụng	118
7.9.1	Ý tưởng chính	118
7.9.2	Ứng dụng ví dụ	118
7.9.3	Ví dụ mã Python: Agent đơn giản với LangChain . . .	119
7.10	Kết luận	119
8	Computer Vision	121
8.1	Fundamentals of Convolutional Layers	121
8.2	Pooling Techniques (Max, Average)	124
8.3	Basic Image Classification	128
8.4	Object Detection Basics (YOLO, SSD)	132
8.5	Image Segmentation Basics (U-Net)	136
8.6	Transfer Learning for Image Classification	140
8.7	Image Augmentation Techniques	143
8.8	Feature Extraction Using Pre-Trained Models	146
8.9	Introduction to GANs (Generating Images)	149
8.10	Introduction to Self-Supervised Learning for Vision	153
8.11	Vision Transformers (ViT) Basics	156
8.12	CLIP and Multimodal Learning	159
8.13	Generative Models	162

Chapter 1

Giới thiệu về Python cho Trí tuệ Nhân tạo

Chương này giới thiệu các kiến thức lập trình cơ bản cần thiết cho các dự án Trí tuệ Nhân tạo (AI), sử dụng Python làm ngôn ngữ chính. Chúng ta sẽ tìm hiểu các khái niệm cơ bản của Python, các thư viện thiết yếu và các bài thực hành giúp bạn phát triển mô hình và dự án AI.

Vì bạn đã giỏi C++, chương này tập trung so sánh Python và C++, kèm theo các ví dụ bằng mã lệnh Python để giúp bạn xây dựng nền tảng vững chắc cho hành trình AI của mình.

1.1 Cơ bản về Python

Python là ngôn ngữ lập trình cấp cao, linh hoạt, được sử dụng rộng rãi trong lĩnh vực AI nhờ sự đơn giản và hệ sinh thái thư viện phong phú. Trong phần này, chúng ta sẽ ôn lại các khái niệm thiết yếu của Python.

1.1.1 Vòng lặp và Cấu trúc điều khiển

Python hỗ trợ các cấu trúc điều khiển cơ bản như vòng lặp và câu lệnh điều kiện, điều bạn đã quen thuộc trong C++. Dưới đây là ví dụ về vòng lặp đơn giản:

```
# A simple for loop in Python
for i in range(5):
```

```
print(i)
```

Đoạn mã trên sẽ in ra các số từ 0 đến 4. Lưu ý hàm `range(N)` trong Python tạo ra dãy số lên đến $N-1$, khác với C++ , bạn dùng điều kiện $i < N$.

Bài tập thực hành:

Viết chương trình Python in ra tất cả các số chẵn từ 10 đến 50 sử dụng vòng lặp `for`.

1.1.2 Hàm trong Python

Hàm trong Python được định nghĩa bằng từ khóa `def`, như ví dụ sau:

```
# Function that adds two numbers
def add_numbers(a, b):
    return a + b

# Function call
result = add_numbers(3, 4)
print(result)
```

Hàm `add_numbers` nhận hai tham số và trả về tổng của chúng. Trong Python, ta sử dụng từ khóa `return` để trả về giá trị.

Bài tập thực hành: Viết hàm Python nhận vào một chuỗi và trả về chuỗi đó được đảo ngược.

1.2 NumPy và Pandas để xử lý dữ liệu

Dữ liệu là nền tảng của AI, và hai thư viện Python được sử dụng rộng rãi nhất để xử lý và thao tác dữ liệu là NumPy và Pandas.

1.2.1 NumPy

NumPy là thư viện hỗ trợ các mảng nhiều chiều và ma trận. Thư viện cũng cung cấp một tập hợp lớn các hàm toán học để thao tác trên mảng.

Ví dụ cách tạo một mảng NumPy đơn giản:

```
import numpy as np

# Create a 1D array
arr = np.array([1, 2, 3, 4])
print(arr)

# Create a 2D array (Matrix)
matrix = np.array([[1, 2], [3, 4]])
print(matrix)
```

Bài tập thực hành: Tạo một mảng NumPy kích thước 3x3 với các số nguyên ngẫu nhiên từ 1 đến 10. Sau đó tính tổng tất cả các phần tử.

1.2.2 Pandas

Pandas được xây dựng trên NumPy, được thiết kế để làm việc với dữ liệu có cấu trúc (dạng bảng). Nó cung cấp hai cấu trúc dữ liệu chính: **Series** (dữ liệu 1 chiều) và **DataFrame** (dữ liệu 2 chiều).

Ví dụ tạo một **DataFrame** với Pandas:

```
import pandas as pd

# Create a simple DataFrame
data = {'Name': ['Alice', 'Bob', 'Charlie'], 'Age': [24, 30, 22]}
df = pd.DataFrame(data)

# Display the DataFrame
print(df)
```

Bài tập thực hành:

Dùng Pandas để tải một file CSV chứa thông tin sinh viên (tên, điểm, tuổi).
Hiển thị các dòng có điểm lớn hơn 75.

1.3 Matplotlib và Seaborn để trực quan hóa dữ liệu

Trực quan hóa dữ liệu là một phần quan trọng trong AI để hiểu và phân tích dữ liệu. Trong phần này, chúng ta sẽ tìm hiểu cách vẽ biểu đồ bằng Matplotlib và Seaborn.

1.3.1 Matplotlib

Matplotlib là thư viện vẽ đồ thị trong Python. Bạn có thể tạo đồ thị đơn giản bằng `matplotlib.pyplot`:

```
import matplotlib.pyplot as plt

# Sample data
x = [1, 2, 3, 4]
y = [1, 4, 9, 16]

# Create a plot
plt.plot(x, y)

# Add titles and labels
plt.title("Example Plot")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")

# Show the plot
plt.show()
```

1.3.2 Seaborn

Seaborn là thư viện xây dựng trên Matplotlib, giúp việc vẽ biểu đồ thống kê trở nên dễ dàng và đẹp mắt hơn.

Ví dụ biểu đồ phân tán với Seaborn:

```
import seaborn as sns
```

```
# Load the built-in 'iris' dataset
iris = sns.load_dataset("iris")

# Create a scatter plot
sns.scatterplot(x='sepal_length', y='sepal_width', data=iris)
plt.show()
```

Bài tập thực hành:

Tạo biểu đồ cột so sánh độ tuổi trung bình của sinh viên các khoa khác nhau bằng Pandas và Seaborn.

1.4 Scikit-learn

Scikit-learn là thư viện nổi tiếng cho học máy. Nó cung cấp các công cụ để xây dựng và đánh giá mô hình, từ phân loại đơn giản đến hồi quy phức tạp.

1.4.1 Huấn luyện một mô hình đơn giản

Ví dụ huấn luyện mô hình cây quyết định:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# Load dataset
data = load_iris()
X = data.data
y = data.target

# Split the data
X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                    test_size=0.2)

# Train a Decision Tree model
model = DecisionTreeClassifier()
```

```
model.fit(X_train, y_train)

# Make predictions
y_pred = model.predict(X_test)

# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

Bài tập thực hành:

Huấn luyện mô hình hồi quy logistic với một tập dữ liệu bất kỳ và đánh giá độ chính xác.

1.5 Cơ bản về PyTorch

PyTorch là một framework học sâu hỗ trợ xây dựng và huấn luyện mạng nơ-ron. Nó sử dụng biểu đồ tính toán động, rất linh hoạt cho cả nghiên cứu và triển khai.

1.5.1 Giới thiệu về Tensor

Tensor là cấu trúc dữ liệu cốt lõi trong PyTorch, tương tự như mảng trong NumPy, nhưng hỗ trợ tính toán tự động và GPU.

Ví dụ tạo và thao tác với Tensor:

```
import torch

# Create a tensor
tensor = torch.tensor([[1, 2], [3, 4]])

# Perform operations on the tensor
tensor_add = tensor + 2
print(tensor_add)
```

Bài tập thực hành:

Tạo tensor 3x3 và thực hiện phép nhân ma trận với một tensor khác có kích thước phù hợp.

1.6 Cơ bản về tái tạo kết quả

Đảm bảo khả năng tái tạo kết quả là điều rất quan trọng trong nghiên cứu AI. Điều này bao gồm việc thiết lập seed ngẫu nhiên, chỉ định thiết bị (CPU hoặc GPU) và đảm bảo mô hình suy luận nhất quán.

1.6.1 Thiết lập seed

Thiết lập seed giúp bạn có được kết quả giống nhau qua các lần chạy:

```
import torch
import numpy as np
import random

# Set random seeds
torch.manual_seed(42)
np.random.seed(42)
random.seed(42)
```

Bài tập thực hành:

Viết chương trình Python tạo số ngẫu nhiên và cho thấy cách seed tạo ra cùng một kết quả ở các lần chạy khác nhau.

1.7 Huấn luyện mô hình trên CPU và GPU

PyTorch cho phép bạn huấn luyện mô hình trên cả CPU và GPU, tùy theo phần cứng. Kiểm tra GPU và chuyển mô hình:

```
device = torch.device("cuda" if torch.cuda.is_available() else
                       "cpu")

# Create a model
model = torch.nn.Linear(10, 5)
```

```
# Move model to GPU
model.to(device)
```

Bài tập thực hành:

Tạo một mạng nơ-ron đơn giản với PyTorch và huấn luyện trên một tập dữ liệu, thử nghiệm trên cả CPU và GPU (nếu có), và so sánh thời gian huấn luyện.

1.8 Sử dụng Weights and Biases để theo dõi thực nghiệm

Weights and Biases (W&B) là công cụ phổ biến để theo dõi quá trình huấn luyện mô hình AI. Bạn có thể ghi lại siêu tham số, độ chính xác, loss và nhiều chỉ số khác.

Ví dụ khởi tạo W&B:

```
import wandb

# Initialize W&B
wandb.init(project="my-ai-project")

# Log a hyperparameter
wandb.config.learning_rate = 0.001
```

Bài tập thực hành:

Thiết lập W&B để theo dõi quá trình huấn luyện của một mạng nơ-ron đơn giản trên một tập dữ liệu tùy chọn.

1.9 Kết luận

Chương này cung cấp kiến thức nền tảng về Python cho AI. Chúng ta đã đi qua các thư viện thiết yếu như NumPy, Pandas, Matplotlib, và PyTorch, cũng như các thực hành tốt về tái tạo kết quả và huấn luyện mô hình. Sau chương

này, bạn đã sẵn sàng để bắt đầu xây dựng các dự án AI của riêng mình và đi sâu hơn vào các chủ đề như học máy, thị giác máy tính và xử lý ngôn ngữ tự nhiên.

Chapter 2

Học có giám sát – Supervised Learning

2.1 Giới thiệu

Học có giám sát (Supervised Learning) là một nhánh quan trọng trong học máy (Machine Learning). Ý tưởng cơ bản là: từ một tập dữ liệu đã biết đầu vào (features) và đầu ra (labels), ta học một mô hình để dự đoán đầu ra mới khi có dữ liệu đầu vào mới.

Hai loại bài toán chính:

- **Regression (Hồi quy):** Dự đoán đầu ra là giá trị số liên tục.
- **Classification (Phân loại):** Dự đoán đầu ra là nhãn rời rạc (ví dụ: "cat" hay "dog").

2.2 Linear Regression (Hồi quy tuyến tính)

Dùng để dự đoán giá trị liên tục. Mô hình:

$$\hat{y} = w^T x + b$$

Mục tiêu là tìm w, b để sai số nhỏ nhất. Sử dụng hàm mất mát bình phương:

$$L = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

```
from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(X_train, y_train)
```

2.3 Logistic Regression (Hồi quy logistic)

Thường dùng cho bài toán phân loại nhị phân. Mô hình tính xác suất bằng hàm sigmoid:

$$P(y = 1|x) = \frac{1}{1 + e^{-w^T x - b}}$$

Hàm mất mát: log loss (cross entropy)

$$L = -\frac{1}{n} \sum [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

```
from sklearn.linear_model import LogisticRegression
model = LogisticRegression()
model.fit(X_train, y_train)
```

2.4 K-Nearest Neighbors (K-NN)

Không có quá trình huấn luyện. Khi dự đoán, ta tìm K điểm gần nhất và lấy nhãn phổ biến nhất để dự đoán.

Ưu điểm: Dễ hiểu, dễ cài đặt.

Nhược điểm: Tốn bộ nhớ, chậm khi dữ liệu lớn.

```
from sklearn.neighbors import KNeighborsClassifier
model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)
```

2.5 Decision Trees (Cây quyết định)

Mô hình này chia nhỏ không gian đầu vào dựa trên các câu hỏi dạng "Nếu... thì...". Mỗi nút trong cây là một điều kiện (ví dụ: $x_j < t$), và các nhánh con chứa dữ liệu được chia.

Ý tưởng chính: Tìm điều kiện chia giúp nhóm dữ liệu trở nên "thuần" nhất về nhãn.

Hàm đo độ thuần:

- Gini Impurity: $G = 1 - \sum p_i^2$
- Entropy: $H = - \sum p_i \log p_i$

Ưu điểm: Dễ diễn giải, phù hợp dữ liệu rời rạc lẫn liên tục.

Nhược điểm: Dễ overfitting nếu không cắt tỉa (pruning).

```
from sklearn.tree import DecisionTreeClassifier
model = DecisionTreeClassifier(max_depth=4)
model.fit(X_train, y_train)
```

2.6 Random Forests (Rừng ngẫu nhiên)

Là tổ hợp của nhiều cây quyết định. Mỗi cây học trên tập con dữ liệu khác nhau và chọn ngẫu nhiên đặc trưng.

Lợi ích: Giảm overfitting, tăng độ chính xác.

```
from sklearn.ensemble import RandomForestClassifier
model = RandomForestClassifier(n_estimators=100)
model.fit(X_train, y_train)
```

2.7 Gradient Boosting

Tăng cường dần bằng cách huấn luyện các mô hình (thường là cây nông) theo thứ tự, mỗi mô hình cố gắng sửa lỗi của mô hình trước.

Ý tưởng: Học sai số còn lại (residuals).

```
from sklearn.ensemble import GradientBoostingClassifier
model = GradientBoostingClassifier()
model.fit(X_train, y_train)
```

2.8 Support Vector Machines (SVM)

Ý tưởng: Tìm siêu phẳng (hyperplane) phân chia hai lớp với khoảng cách (margin) lớn nhất.

Phương trình siêu phẳng:

$$w^T x + b = 0$$

Tối ưu hóa khoảng cách giữa siêu phẳng và các điểm gần nhất:

$$\min_{w,b} \frac{1}{2} \|w\|^2$$

Với ràng buộc:

$$y_i(w^T x_i + b) \geq 1$$

Soft margin: Cho phép vi phạm nhẹ bằng biến ξ_i :

$$\min \frac{1}{2} \|w\|^2 + C \sum \xi_i$$

Kernel trick: Dùng để phân loại phi tuyến:

- Linear kernel: $K(x_i, x_j) = x_i^T x_j$
- RBF kernel: $K(x_i, x_j) = e^{-\gamma \|x_i - x_j\|^2}$

Ưu điểm: Mạnh với dữ liệu nhiều chiều, có khả năng phân loại tốt khi kết hợp kernel.

```
from sklearn.svm import SVC
model = SVC(kernel='rbf', C=1.0)
model.fit(X_train, y_train)
```


Chapter 3

Học không giám sát – Unsupervised Learning

Giới thiệu

Trong bức tranh rộng lớn của học máy, học không giám sát là một phương pháp mạnh mẽ nhưng thường bị đánh giá thấp. Khác với học có giám sát, nơi các thuật toán học từ dữ liệu đã được gán nhãn, học không giám sát đi sâu vào những điều chưa biết — khám phá ra các cấu trúc, mẫu hình và mối quan hệ ẩn trong dữ liệu mà không cần đến bất kỳ nhãn hay kết quả định sẵn nào.

Về bản chất, học không giám sát nhằm mục đích hiểu dữ liệu bằng cách tổ chức chúng theo những cách có ý nghĩa. Điều này bao gồm việc gom nhóm các điểm dữ liệu tương tự lại với nhau, giảm số chiều để đơn giản hóa các bộ dữ liệu phức tạp, và khám phá các đặc trưng tiềm ẩn có thể nắm bắt được bản chất của không gian đầu vào. Những phương pháp này là không thể thiếu trong các tình huống mà việc gán nhãn cho dữ liệu là không khả thi, tốn kém hoặc thậm chí là bất khả thi.

Chương này sẽ cung cấp một cái nhìn tổng quan toàn diện về học không giám sát, bắt đầu với các khái niệm nền tảng và tiến đến các kỹ thuật phổ biến nhất. Chúng ta sẽ khám phá các thuật toán phân cụm như k-means và phân cụm phân cấp, đi sâu vào các phương pháp giảm chiều như Phân tích Thành phần Chính (PCA), và xem xét các tiếp cận hiện đại bao gồm autoencoder và các mô hình sinh (generative models). Ngoài ra, chúng ta cũng sẽ thảo luận

về các ứng dụng thực tiễn, những thách thức, và các chiến lược đánh giá liên quan đến các bài toán học không giám sát.

Kết thúc chương này, bạn sẽ có một nền tảng vững chắc về cách các thuật toán học không giám sát có thể được tận dụng để khám phá cấu trúc ẩn trong dữ liệu — từ đó cho phép xây dựng các hệ thống thông minh hơn, linh hoạt hơn, có khả năng học hỏi mà không cần chỉ dẫn rõ ràng.

3.1 Phân cụm K-Means

3.1.1 Giới thiệu

Hãy tưởng tượng bạn là một người thủ thư đang cố gắng sắp xếp hàng ngàn cuốn sách mà không biết thể loại của chúng. Sẽ ra sao nếu bạn có thể tự động nhóm các cuốn sách tương tự nhau dựa trên mô tả của chúng? Đó chính là lúc **Thuật toán Phân cụm K-Means** phát huy tác dụng — một thuật toán học không giám sát mạnh mẽ giúp tìm ra các nhóm tự nhiên trong dữ liệu mà không cần ví dụ đã được gán nhãn.

Cốt lõi của K-Means là trả lời một câu hỏi đơn giản:

Chúng ta có thể nhóm các điểm dữ liệu này thành K cụm riêng biệt dựa trên mức độ tương đồng của chúng không?

Khác với học có giám sát, nơi chúng ta huấn luyện mô hình với dữ liệu đã có nhãn, **K-Means không yêu cầu bất kỳ nhãn nào**. Thay vào đó, nó khám phá cấu trúc của dữ liệu và nhóm các điểm gần nhau trong không gian đặc trưng lại với nhau.

3.1.2 Thuật toán Hoạt động Như Thế Nào

K-Means hoạt động thông qua một số bước lặp như sau:

1. **Chọn số cụm, K .**
2. **Khởi tạo ngẫu nhiên K trung tâm cụm (centroid).**
3. **Gán mỗi điểm dữ liệu cho trung tâm cụm gần nhất.**
Khoảng cách thường được đo bằng khoảng cách Euclid.

4. Cập nhật các trung tâm cụm.

Với mỗi cụm, tính trung bình của tất cả các điểm được gán vào cụm đó. Trung bình này trở thành trung tâm cụm mới.

5. Lặp lại bước 3 và 4 cho đến khi hội tụ (tức là các trung tâm cụm không thay đổi đáng kể nữa).

Quá trình này thường hội tụ nhanh chóng và tạo ra các cụm được phân tách rõ ràng.

3.1.3 Ứng Dụng Thực Tế

K-Means được sử dụng rộng rãi trong nhiều lĩnh vực:

- **Phân khúc khách hàng:** Nhóm khách hàng theo hành vi mua sắm.
- **Phân cụm tài liệu:** Sắp xếp bài viết hoặc tài liệu theo chủ đề.
- **Nén ảnh:** Giảm số lượng màu trong ảnh (lượng tử hóa màu).
- **Phân tích định vị địa lý:** Xác định cụm hoạt động trong dữ liệu không gian địa lý.

3.1.4 Cải tiến và Biến thể

Một số cải tiến phổ biến bao gồm:

- **K-Means++:** Chiến lược khởi tạo thông minh hơn để cải thiện hiệu suất.
- **Mini-Batch K-Means:** Phân cụm nhanh hơn cho các tập dữ liệu lớn.
- **Silhouette Score và Elbow Method:** Các kỹ thuật để xác định số cụm tối ưu.
- **Kernel K-Means:** Cho phép ranh giới cụm phi tuyến tính.

3.1.5 Ví dụ Python với scikit-learn

Dưới đây là một ví dụ thực tế sử dụng dữ liệu tổng hợp:

```
from sklearn.cluster import KMeans
```

```
from sklearn.datasets import make_blobs
import matplotlib.pyplot as plt

# Generate sample data
X, y = make_blobs(n_samples=300, centers=4, cluster_std=0.6,
                  random_state=42)

# Apply K-Means
kmeans = KMeans(n_clusters=4, random_state=42)
kmeans.fit(X)
y_kmeans = kmeans.predict(X)

# Plot the results
plt.scatter(X[:, 0], X[:, 1], c=y_kmeans, s=50, cmap='viridis'
            )
plt.scatter(kmeans.cluster_centers_[ :, 0], kmeans.
            cluster_centers_[ :, 1], s=200,
            c='red', marker='X', label='Centroids')
plt.title("K-Means Clustering Result")
plt.legend()
plt.show()
```

3.2 Phân Tích Thành Phần Chính (PCA)

3.2.1 Giới thiệu về Phân Tích Thành Phần Chính (PCA)

Phân tích thành phần chính (PCA) là một thuật toán học không giám sát mạnh mẽ, được sử dụng rộng rãi cho *giảm chiều dữ liệu*, *trực quan hóa dữ liệu*, và *lọc nhiễu*. Thuật toán này chuyển đổi dữ liệu có nhiều chiều thành dạng có ít chiều hơn nhưng vẫn giữ lại càng nhiều thông tin (phương sai) càng tốt.

Về bản chất, PCA xác định các *hướng* (gọi là các thành phần chính) mà tại đó dữ liệu thay đổi nhiều nhất. Bằng cách chiếu dữ liệu lên những hướng này, PCA giúp giảm độ phức tạp, loại bỏ sự dư thừa và thường cải thiện hiệu suất cũng như khả năng diễn giải của các mô hình sau đó.

Tại sao PCA hữu ích?

- Dữ liệu có nhiều chiều thường chứa các đặc trưng dư thừa hoặc nhiễu.
- Giảm chiều có thể cải thiện hiệu quả tính toán và hiệu suất mô hình.
- Trực quan hóa dữ liệu nhiều chiều đòi hỏi phải giảm chiều.
- PCA giúp phát hiện cấu trúc ẩn trong dữ liệu.

3.2.2 Giải thích Thuật toán

PCA bao gồm các bước chính sau:

1. Chuẩn hóa dữ liệu

Chuẩn hóa các đặc trưng để có trung bình bằng 0 và phương sai bằng 1.

Điều này đảm bảo mỗi đặc trưng đóng góp như nhau.

2. Tính ma trận hiệp phương sai

Tính ma trận hiệp phương sai để hiểu cách các đặc trưng biến đổi cùng nhau:

$$\mathbf{C} = \frac{1}{n-1} \mathbf{X}^T \mathbf{X}$$

trong đó $\mathbf{X}$ là ma trận dữ liệu đã được chuẩn hóa.

3. Tính các trị riêng và vector riêng

Phân tích trị riêng (eigen decomposition) trên ma trận hiệp phương sai.

Các vector riêng là hướng (các thành phần chính), và các trị riêng đo lường lượng phương sai mà mỗi hướng mang lại.

4. Sắp xếp và chọn các thành phần chính hàng đầu

Sắp xếp các vector riêng theo thứ tự giảm dần của trị riêng tương ứng và chọn k thành phần chính hàng đầu.

5. Chiếu dữ liệu

Biến đổi dữ liệu gốc sang không gian đặc trưng mới:

$$\mathbf{X}_{\text{reduced}} = \mathbf{X} \cdot \mathbf{W}_k$$

trong đó $\mathbf{W}_k$ chứa k vector riêng hàng đầu.

3.2.3 Ví dụ Minh Họa

Hãy xem xét một tập dữ liệu 2 chiều đơn giản như chiều cao và cân nặng. Hai đặc trưng này thường có mối tương quan — người cao hơn thường nặng hơn. PCA có thể xác định hướng mà dữ liệu thay đổi nhiều nhất (một tổ hợp tuyến tính của chiều cao và cân nặng), và chiếu dữ liệu lên trục mới này.

Dưới đây là ví dụ thực tiễn với Python:

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
from sklearn.datasets import load_iris

# Load dataset
iris = load_iris()
X = iris.data[:, :2] # Only first two features

# Standardize
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Apply PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

# Plot
df_pca = pd.DataFrame(X_pca, columns=['PC1', 'PC2'])
df_pca['target'] = iris.target

sns.scatterplot(data=df_pca, x='PC1', y='PC2', hue='target',
                palette='Set1')
plt.title('PCA on Iris Dataset')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.grid(True)
plt.show()
```

Kết quả là biểu đồ 2D với các trục đại diện cho các hướng biến thiên lớn nhất trong dữ liệu gốc.

3.2.4 Ứng Dụng Thực Tế

PCA được ứng dụng rộng rãi trong nhiều lĩnh vực:

- **Tiền xử lý dữ liệu:** Giảm số chiều trước khi áp dụng các mô hình học máy.
- **Lọc nhiễu:** Loại bỏ các thành phần có phương sai thấp — thường là nhiễu.
- **Tài chính:** Phân tích xu hướng và mối tương quan trong giá cổ phiếu hoặc chỉ số tài chính.
- **Genomics:** Trực quan hóa dữ liệu biểu hiện gene nhiều chiều.
- **Thị giác máy tính:** Nhận dạng khuôn mặt với các kỹ thuật như Eigenfaces.

3.3 t-SNE, MAP, và những cách giảm chiều dữ liệu khác

3.3.1 Giới thiệu

Trong các tập dữ liệu có nhiều chiều, việc nhận diện các mẫu (patterns) có thể trở nên khó khăn. Hãy tưởng tượng bạn đang cố hình dung hành vi khách hàng dựa trên một tập dữ liệu với hàng chục đặc trưng—từ nhân khẩu học đến lịch sử duyệt web. Trực giác con người không còn hiệu quả khi vượt quá ba chiều, khiến việc lý giải cấu trúc dữ liệu trở nên khó khăn.

Các kỹ thuật giảm chiều dữ liệu nhằm giải quyết vấn đề này bằng cách chuyển dữ liệu nhiều chiều sang dạng ít chiều hơn—thường là hai hoặc ba chiều—trong khi vẫn bảo toàn các cấu trúc quan trọng như cụm (clusters) hoặc lân cận (neighborhoods). Điều này đặc biệt hữu ích trong **học không giám sát**, nơi chúng ta thường không có nhãn để hướng dẫn việc phân tích.

Trong phần này, chúng ta sẽ khám phá:

- **t-SNE (t-distributed Stochastic Neighbor Embedding)**
- **MAP (Manifold Approximation and Projection)**
- Tổng quan ngắn về các phương pháp khác như **PCA**, **Isomap**, và **UMAP**

Các phương pháp này là công cụ mạnh mẽ để hiểu phân bố dữ liệu, trực quan hóa các cụm, và thậm chí cải thiện hiệu suất của các mô hình phía sau.

3.3.2 t-SNE: t-distributed Stochastic Neighbor Embedding

Tổng quan

t-SNE là một kỹ thuật giảm chiều **phi tuyến** đặc biệt phù hợp cho việc **trực quan hóa các tập dữ liệu nhiều chiều**. Được phát triển bởi Laurens van der Maaten và Geoffrey Hinton, t-SNE hoạt động bằng cách chuyển đổi sự tương đồng giữa các điểm dữ liệu thành xác suất kết hợp và tối thiểu hóa độ lệch giữa hai phân phối này trong không gian nhiều chiều và ít chiều.

Cách hoạt động

1. Tính toán mức độ tương đồng giữa các cặp điểm trong không gian nhiều chiều bằng phân phối Gauss.
2. Định nghĩa tương đồng trong không gian ít chiều bằng phân phối t của Student.
3. Tối thiểu hóa độ lệch Kullback–Leibler giữa hai phân phối bằng phương pháp gradient descent.

Kết quả là một bản đồ 2D hoặc 3D trong đó các điểm giống nhau sẽ gần nhau, còn các điểm khác biệt sẽ cách xa nhau.

Ví dụ

```
from sklearn.manifold import TSNE
from sklearn.datasets import load_digits
```



```
import matplotlib.pyplot as plt

# Load sample data
digits = load_digits()
X = digits.data
y = digits.target

# Apply t-SNE
tsne = TSNE(n_components=2, random_state=42)
X_reduced = tsne.fit_transform(X)

# Visualize
plt.scatter(X_reduced[:, 0], X_reduced[:, 1], c=y, cmap='tab10', s=15)
plt.title("t-SNE on Digits Dataset")
plt.colorbar()
plt.show()
```

3.3.3 MAP: Manifold Approximation and Projection

Tổng quan

MAP (thường được sử dụng đồng nghĩa với **UMAP**, một cách triển khai phổ biến) là một thuật toán giảm chiều hiện đại dựa trên các nguyên lý học manifold. Nó nổi bật với khả năng **nhANH hơn và mở rộng tốt hơn** t-SNE, đồng thời bảo toàn **cả cấu trúc cục bộ lẫn toàn cục**.

Cách hoạt động

1. Xây dựng đồ thị có trọng số của các điểm gần nhất trong không gian nhiều chiều.
2. Tối ưu một đồ thị ít chiều sao cho tương tự về cấu trúc với đồ thị ban đầu bằng tối ưu hóa ngẫu nhiên.
3. Sử dụng cấu trúc tô-pô mờ (fuzzy topological structure) để giữ được mối quan hệ cục bộ và toàn cục.

Ví dụ

```
import umap
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits

# Load sample data
digits = load_digits()
X = digits.data
y = digits.target

# Apply UMAP
reducer = umap.UMAP(n_components=2, random_state=42)
X_umap = reducer.fit_transform(X)

# Visualize
plt.scatter(X_umap[:, 0], X_umap[:, 1], c=y, cmap='tab10', s
            =15)
plt.title("UMAP on Digits Dataset")
plt.colorbar()
plt.show()
```

3.3.4 Các phương pháp giảm chiều khác

PCA (Principal Component Analysis): Một kỹ thuật tuyến tính chiếu dữ liệu theo các hướng có phương sai lớn nhất. Nhanh và dễ diễn giải, nhưng kém trong việc nắm bắt các mẫu phi tuyến phức tạp.

Isomap: Mở rộng PCA bằng cách sử dụng khoảng cách địa lý (geodesic distance) trên đồ thị lân cận. Tốt cho việc nắm bắt manifold ẩn, nhưng tốn kém về tính toán.

Autoencoders: Mạng nơ-ron được huấn luyện để tái tạo dữ liệu đầu vào. Các lớp ẩn (latent layers) có thể được sử dụng để giảm chiều, đặc biệt hiệu quả với dữ liệu phi tuyến phức tạp.

3.3.5 Ứng dụng thực tiễn

- **Trực quan hóa dữ liệu:** Phục vụ phân tích dữ liệu khám phá (EDA), t-SNE và UMAP có thể làm lộ ra các cụm hoặc mẫu tiềm ẩn.
- **Phát hiện bất thường:** Các điểm ngoại lai trở nên rõ ràng hơn trong không gian ít chiều.
- **Tiền xử lý:** Giảm chiều trước khi phân cụm hoặc phân loại có thể cải thiện tốc độ và độ chính xác.
- **Genomics và y tế:** Trực quan hóa biểu hiện gen hoặc chỉ số sức khỏe có nhiều chiều.
- **Hệ thống gợi ý:** Gán nhãn cho item hoặc người dùng trong không gian ngầm (latent space) để tìm kiếm tương đồng.

3.4 Phân Cụm DBSCAN

3.4.1 Giới thiệu

DBSCAN (**Density-Based Spatial Clustering of Applications with Noise**) là một thuật toán học không giám sát mạnh mẽ, có khả năng nhóm các điểm dữ liệu gần nhau lại với nhau, và đánh dấu các điểm ở vùng thưa thớt là ngoại lai (outliers). Không giống như K-Means hay phân cụm phân cấp, DBSCAN không yêu cầu xác định trước số cụm và có thể phát hiện các cụm có hình dạng bất kỳ — điều này đặc biệt hữu ích với các tập dữ liệu thực tế, nơi mà cấu trúc dữ liệu không được biết trước.

Ý tưởng cốt lõi rất trực quan: các điểm trong vùng mật độ cao sẽ tạo thành cụm, trong khi các điểm ở vùng mật độ thấp được xem là nhiễu (noise).

3.4.2 Giải thích thuật toán

DBSCAN yêu cầu hai tham số:

- **eps (epsilon):** khoảng cách tối đa giữa hai điểm để một điểm được xem là hàng xóm của điểm kia.

- **min_samples**: số điểm tối thiểu trong vùng lân cận để một điểm được xem là điểm lõi.

Dựa trên hai tham số này, DBSCAN phân loại mỗi điểm dữ liệu như sau:

1. **Điểm lõi (Core Point)**: Có ít nhất **min_samples** điểm (bao gồm chính nó) nằm trong khoảng cách **eps**.
2. **Điểm biên (Border Point)**: Nằm trong khoảng **eps** của một điểm lõi nhưng có ít hơn **min_samples** điểm lân cận.
3. **Điểm nhiễu (Noise Point)**: Không phải điểm lõi và cũng không thể tiếp cận trực tiếp từ điểm lõi.

Thuật toán hoạt động theo các bước sau:

1. Chọn một điểm chưa được thăm.
2. Nếu điểm đó có ít nhất **min_samples** điểm lân cận trong khoảng **eps**, đánh dấu nó là **điểm lõi** và bắt đầu một cụm mới.
3. Mở rộng cụm bằng cách đệ quy thăm tất cả các điểm lõi có thể tiếp cận được.
4. Các điểm nằm trong khoảng **eps** của một điểm lõi nhưng không phải điểm lõi sẽ được thêm vào cụm dưới dạng **điểm biên**.
5. Các điểm không thể tiếp cận từ bất kỳ điểm lõi nào sẽ được gán nhãn là **nhiều**.
6. Lặp lại cho đến khi tất cả các điểm được thăm.

3.4.3 Ví dụ minh họa

Hãy tưởng tượng một biểu đồ tán xạ 2 chiều của khách hàng trong trung tâm thương mại, dựa trên tần suất họ ghé thăm và số tiền họ chi tiêu. Một số khách hàng tạo thành các nhóm tự nhiên (ví dụ như người tiêu dùng thường xuyên và chi tiêu lớn so với người ghé thăm hiếm hoi), nhưng cũng có những người không phù hợp với bất kỳ mẫu cụ thể nào.

DBSCAN có thể tự động tìm ra các cụm này mà không cần xác định trước số lượng nhóm. Hơn nữa, nó có thể gán nhãn cho các điểm ngoại lệ — chẳng hạn như người chỉ ghé một lần nhưng chi tiêu rất cao — là nhiễu.

Ví dụ bằng mã Python

```
from sklearn.cluster import DBSCAN
from sklearn.datasets import make_moons
import matplotlib.pyplot as plt

# Create synthetic dataset
X, _ = make_moons(n_samples=300, noise=0.05, random_state=0)

# Apply DBSCAN
dbscan = DBSCAN(eps=0.2, min_samples=5)
labels = dbscan.fit_predict(X)

# Plotting the clusters
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis')
plt.title("DBSCAN Clustering Example")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()
```

Đoạn mã trên tạo ra tập dữ liệu gồm hai hình bán nguyệt lồng vào nhau (“moons”), trong đó DBSCAN xác định đúng các cụm, khác với K-Means vốn thất bại do cấu trúc không hình cầu của dữ liệu.

3.4.4 Ứng dụng thực tiễn

DBSCAN rất phù hợp với các bài toán có dữ liệu nhiễu và cụm không nhất thiết phải hình cầu:

- **Phân tích địa lý (Geospatial Analysis):** Phát hiện các điểm nóng trong dữ liệu tội phạm hoặc hoạt động địa lý.
- **Phát hiện bất thường:** Nhận diện hành vi gian lận hoặc số đo sai từ cảm biến.
- **Xử lý ảnh:** Phân vùng các khu vực trong ảnh nhiễu hoặc phức tạp.
- **Phân khúc thị trường:** Nhóm khách hàng theo hành vi mà không cần nhãn trước.

- **Thiên văn học:** Tìm kiếm cụm sao hoặc cấu trúc thiên hà từ dữ liệu không gian.

Khả năng gán nhãn cho dữ liệu nhiễu giúp DBSCAN đặc biệt hữu ích trong các lĩnh vực thực tế nơi dữ liệu không liên quan hoặc bất thường xuất hiện thường xuyên.

3.5 Phân Cụm Phân Cấp (Hierarchical Clustering)

3.5.1 Giới thiệu

Phân cụm phân cấp là một kỹ thuật học không giám sát được sử dụng để nhóm các đối tượng tương tự vào các cụm dựa trên độ tương đồng của chúng. Không giống như các phương pháp phân vùng như k -means, phân cụm phân cấp không yêu cầu xác định số lượng cụm trước. Thay vào đó, nó xây dựng một cấu trúc dạng cây gọi là **dendrogram**, đại diện cho các nhóm dữ liệu lồng vào nhau và mức độ gần gũi của chúng.

Phương pháp này đặc biệt hữu ích khi cấu trúc dữ liệu có tính phân cấp, hoặc khi bạn muốn khám phá các mức độ chi tiết khác nhau trong phân cụm. Phân cụm phân cấp có tính trực quan, dễ hiểu và hoạt động tốt với các tập dữ liệu nhỏ đến vừa, nơi mà mối quan hệ giữa các điểm dữ liệu quan trọng hơn là các ranh giới cụm chính xác.

3.5.2 Cách Thuật Toán Hoạt Động

Có hai loại phân cụm phân cấp chính:

- **Agglomerative (Từ dưới lên):** Bắt đầu với mỗi điểm dữ liệu như là một cụm riêng biệt và liên tục hợp nhất các cụm gần nhau cho đến khi tất cả các điểm dữ liệu tạo thành một cụm duy nhất.
- **Divisive (Từ trên xuống):** Bắt đầu với một cụm lớn chứa tất cả các điểm dữ liệu và chia nó thành các cụm nhỏ hơn một cách đệ quy.

Phân cụm agglomerative là phương pháp phổ biến hơn, vì vậy trong phần này chúng ta sẽ tập trung vào phương pháp đó.

Các Bước Của Phân Cụm Agglomerative

1. **Tính Ma Trận Khoảng Cách:** Tính khoảng cách (hoặc độ tương đồng) giữa tất cả các điểm dữ liệu bằng một chỉ số thích hợp (ví dụ: khoảng cách Euclid).
2. **Khởi tạo Các Cụm:** Xem mỗi điểm dữ liệu là một cụm riêng biệt.
3. **Hợp Nhất Các Cụm Gần Nhau Nhất:** Xác định hai cụm có khoảng cách nhỏ nhất và hợp nhất chúng lại.
4. **Cập Nhật Ma Trận Khoảng Cách:** Tính lại khoảng cách giữa cụm mới và tất cả các cụm còn lại bằng một *phương pháp liên kết* (ví dụ: phương pháp đơn, đầy đủ, trung bình, hoặc phương pháp Ward).
5. **Lặp lại:** Tiếp tục hợp nhất các cụm cho đến khi chỉ còn một cụm duy nhất hoặc đạt được số lượng cụm mong muốn.
6. **Xây dựng Dendrogram:** Hiển thị quá trình hợp nhất dưới dạng dendrogram để xác định số lượng cụm phù hợp bằng cách "cắt" cây tại một mức độ thích hợp.

3.5.3 Ví dụ Minh Họa

Xem xét năm điểm dữ liệu 2D và áp dụng phân cụm phân cấp sử dụng phương pháp liên kết Ward:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.cluster.hierarchy import linkage, dendrogram,
    fcluster

# Sample data
X = np.array([[1, 2], [2, 3], [5, 8], [6, 9], [8, 8]])

# Step 1: Compute linkage matrix
Z = linkage(X, method='ward') # 'ward' minimizes the variance
    within clusters

# Step 2: Plot dendrogram
plt.figure(figsize=(8, 4))
```

```
dendrogram(Z)
plt.title("Hierarchical Clustering Dendrogram")
plt.xlabel("Data Points")
plt.ylabel("Distance")
plt.show()

# Step 3: Form flat clusters (e.g., form 2 clusters)
clusters = fcluster(Z, t=2, criterion='maxclust')
print("Cluster assignments:", clusters)
```

Trục dọc của dendrogram đại diện cho khoảng cách mà tại đó các cụm được hợp nhất. Bằng cách thiết lập một ngưỡng dọc theo trục này, bạn có thể chọn số lượng cụm phù hợp với dữ liệu của mình.

3.5.4 Ứng Dụng Thực Tiễn

Phân cụm phân cấp có ứng dụng rộng rãi trong nhiều lĩnh vực:

- **Sinh học thông tin (Bioinformatics):** Nhóm các gen hoặc protein dựa trên mức độ biểu hiện hoặc sự tương đồng cấu trúc.
- **Phân khúc thị trường (Market Segmentation):** Xác định các nhóm khách hàng có hành vi hoặc sở thích tương tự.
- **Phân nhóm tài liệu (Document Clustering):** Tổ chức các bộ tài liệu lớn (ví dụ: các bài báo) thành các nhóm chủ đề.
- **Phân tích mạng xã hội (Social Network Analysis):** Phát hiện cộng đồng trong các mạng lưới người hoặc thực thể.
- **Phân đoạn hình ảnh (Image Segmentation):** Nhóm các điểm ảnh có đặc trưng tương tự để đơn giản hóa các tác vụ xử lý hình ảnh.

Một trong những điểm mạnh của phân cụm phân cấp là tính khả dụng của nó trong việc giải thích, làm cho nó trở nên lý tưởng cho phân tích dữ liệu khám phá khi cấu trúc dữ liệu chưa được hiểu rõ.

Chapter 4

Đánh Giá Trong Trí Tuệ Nhân Tạo - Evaluation

4.1 Các Chỉ Số Đánh Giá Mô Hình

Đánh giá hiệu suất của mô hình là rất quan trọng để xác định mức độ mô hình có thể tổng quát hóa lên dữ liệu chưa thấy trước đó. Dưới đây là một số chỉ số cơ bản:

4.1.1 Độ Chính Xác (Accuracy)

Độ chính xác là tỷ lệ giữa số mẫu dự đoán đúng trên tổng số mẫu.

$$\text{Độ chính xác} = \frac{TP + TN}{TP + TN + FP + FN}$$

Ví dụ Python:

```
from sklearn.metrics import accuracy_score

y_true = [1, 0, 1, 1, 0, 1, 0]
y_pred = [1, 0, 1, 0, 0, 1, 1]
print("Accuracy:", accuracy_score(y_true, y_pred))
```

4.1.2 Độ Chính Xác Của Dự Đoán Dương (Precision)

Precision đo lường trong số các dự đoán dương thì có bao nhiêu là thực sự dương.

$$\text{Precision} = \frac{TP}{TP + FP}$$

Ví dụ Python:

```
from sklearn.metrics import precision_score
print("Precision:", precision_score(y_true, y_pred))
```

4.1.3 Khả Năng Thu Hồi (Recall)

Recall là tỷ lệ các quan sát dương được dự đoán đúng trên tổng số quan sát dương.

$$\text{Recall} = \frac{TP}{TP + FN}$$

Ví dụ Python:

```
from sklearn.metrics import recall_score
print("Recall:", recall_score(y_true, y_pred))
```

4.1.4 F1-Score

F1 Score là trung bình điều hòa của Precision và Recall.

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Ví dụ Python:

```
from sklearn.metrics import f1_score
print("F1 Score:", f1_score(y_true, y_pred))
```

4.1.5 So Sánh Giữa Các Chỉ Số

- **Accuracy** tốt khi dữ liệu cân bằng, nhưng có thể gây hiểu lầm nếu dữ liệu bị lệch.
- **Precision** quan trọng khi chi phí của false positive cao (ví dụ: phát hiện spam).
- **Recall** quan trọng khi chi phí của false negative cao (ví dụ: chẩn đoán bệnh).
- **F1-Score** hữu ích khi cần cân bằng giữa precision và recall, đặc biệt trong các tập dữ liệu mất cân đối.

4.2 Hiện Tượng Underfitting và Overfitting

4.2.1 Underfitting

Underfitting xảy ra khi mô hình quá đơn giản để học được mẫu ẩn của dữ liệu.

Dấu hiệu:

- Lỗi huấn luyện cao
- Lỗi kiểm tra cao

4.2.2 Overfitting

Overfitting xảy ra khi mô hình học quá kỹ dữ liệu huấn luyện, bao gồm cả nhiễu.

Dấu hiệu:

- Lỗi huấn luyện thấp
- Lỗi kiểm tra cao

Ví dụ Python: Hồi quy Đa Thức

```
from sklearn.linear_model import LinearRegression
```

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import make_pipeline
import numpy as np
import matplotlib.pyplot as plt

X = np.random.rand(100, 1) * 2 - 1
y = X**3 + np.random.randn(100, 1) * 0.1

# Underfitting model
model1 = make_pipeline(PolynomialFeatures(1), LinearRegression())

# Overfitting model
model2 = make_pipeline(PolynomialFeatures(15),
                        LinearRegression())

model1.fit(X, y)
model2.fit(X, y)
```

4.3 Tinh Chỉnh Siêu Tham Số (Hyperparameter Tuning)

Siêu tham số là các thiết lập cấu hình để điều khiển quá trình học. Khác với tham số mô hình được học từ dữ liệu, siêu tham số được thiết lập trước khi huấn luyện.

4.3.1 Tại Sao Cần Tinh Chỉnh Siêu Tham Số?

- Cải thiện hiệu suất của mô hình trên dữ liệu chưa thấy.
- Tìm sự cân bằng giữa bias và variance.
- Tránh underfitting hoặc overfitting.
- Mỗi bộ dữ liệu có thể cần cấu hình khác nhau để đạt kết quả tối ưu.

4.3.2 Tìm Kiểm Lưới (Grid Search)

Grid search thử tất cả các kết hợp của tập giá trị siêu tham số được cung cấp.

Ví dụ Python:

```
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestClassifier

param_grid = {
    'n_estimators': [10, 50, 100],
    'max_depth': [None, 10, 20]
}

clf = GridSearchCV(RandomForestClassifier(), param_grid, cv=5)
clf.fit(X_train, y_train)
print("Best Parameters:", clf.best_params_)
```

4.3.3 Tìm Kiểm Ngẫu Nhiên (Random Search)

Random search chọn ngẫu nhiên các kết hợp siêu tham số để thử.

Ưu điểm so với Grid Search:

- Nhanh hơn khi không gian tìm kiếm lớn.
- Hiệu quả hơn trong việc tìm kết hợp tốt trong thời gian ngắn.

Ví dụ Python:

```
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import randint

param_dist = {
    'n_estimators': randint(10, 200),
    'max_depth': randint(1, 30)
}

clf = RandomizedSearchCV(RandomForestClassifier(), param_dist,
    n_iter=10, cv=5)
clf.fit(X_train, y_train)
print("Best Parameters:", clf.best_params_)
```

4.4 Xác Thực Chéo (Cross-Validation)

Xác thực chéo giúp đánh giá mức độ tổng quát hóa của mô hình trên dữ liệu độc lập.

4.4.1 K-Fold Cross-Validation

K-Fold chia dữ liệu thành K phần bằng nhau. Mô hình được huấn luyện K lần, mỗi lần sử dụng $K - 1$ phần để huấn luyện và 1 phần để kiểm tra.

Lợi ích:

- Giảm phương sai trong đánh giá mô hình.
- Ước lượng hiệu suất đáng tin cậy hơn.
- Mỗi điểm dữ liệu đều được dùng để huấn luyện và kiểm tra.

Ví dụ Python:

```
from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LogisticRegression

scores = cross_val_score(LogisticRegression(), X, y, cv=5)
print("Cross-Validation Scores:", scores)
print("Average Score:", scores.mean())
```

4.5 Ma Trận Nhầm Lẫn và Đường Cong ROC

4.5.1 Ma Trận Nhầm Lẫn

Là bảng 2x2 cho phân loại nhị phân, biểu diễn giá trị thực tế so với giá trị dự đoán.

Cấu trúc:

	Dự đoán Dương	Dự đoán Âm
Thực tế Dương	TP	FN
Thực tế Âm	FP	TN

Diễn giải:

- TP: Dương thật – Dự đoán đúng
- TN: Âm thật – Dự đoán đúng
- FP: Dương giả – Dự đoán sai (Lỗi loại I)
- FN: Âm giả – Dự đoán sai (Lỗi loại II)

Ví dụ Python:

```
from sklearn.metrics import confusion_matrix
print("Confusion Matrix:\n", confusion_matrix(y_true, y_pred))
```

4.5.2 Đường Cong ROC và AUC

Đường cong ROC vẽ tỷ lệ dương thực (TPR) so với tỷ lệ dương giả (FPR) ở các ngưỡng khác nhau.

Công thức:

$$TPR = \frac{TP}{TP + FN} \quad (\text{Recall})$$

$$FPR = \frac{FP}{FP + TN}$$

AUC (Diện tích dưới đường cong):

- Đo diện tích toàn phần dưới đường cong ROC.
- Giá trị từ 0 đến 1. AUC cao hơn cho thấy mô hình tốt hơn.

Ví dụ Python:

```
from sklearn.metrics import roc_curve, roc_auc_score
import matplotlib.pyplot as plt
```

```
fpr, tpr, thresholds = roc_curve(y_true, y_score)
```

```
plt.plot(fpr, tpr, label='ROC Curve')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve')
plt.legend()
plt.show()

print("AUC:", roc_auc_score(y_true, y_score))
```

Khi Nào Dùng ROC Curve:

- Hữu ích khi quan tâm đến sự đánh đổi giữa TPR và FPR.
- Đặc biệt hiệu quả với tập dữ liệu mất cân đối.

Chapter 5

Neural Networks

5.1 Kiến thức cơ bản về Perceptron

Lý thuyết

Perceptron là dạng đơn giản nhất của mạng nơ-ron. Nó là một bộ phân loại nhị phân tuyến tính, nhận nhiều đầu vào, áp dụng trọng số và độ lệch, sau đó truyền qua một hàm kích hoạt.

$$y = f\left(\sum w_i x_i + b\right)$$

Trong đó:

- x_i : Đặc trưng đầu vào
- w_i : Trọng số
- b : Độ lệch (bias)
- f : Hàm kích hoạt (thường là hàm bước)

Mã Python

```
import numpy as np

def step_function(x):
    return 1 if x >= 0 else 0
```

```
def perceptron(inputs, weights, bias):  
    total = np.dot(inputs, weights) + bias  
    return step_function(total)  
  
inputs = np.array([1, 0])  
weights = np.array([0.5, -0.6])  
bias = 0.1  
  
output = perceptron(inputs, weights, bias)  
print("Output:", output)
```

Tóm tắt

- Perceptron là một bộ phân loại tuyến tính.
- Phù hợp với dữ liệu phân tách tuyến tính.
- Là khối xây dựng cơ bản của mạng nơ-ron.

5.2 Thuật toán Gradient Descent

Lý thuyết

Gradient descent là một thuật toán tối ưu được sử dụng để giảm thiểu hàm chi phí. Nó điều chỉnh các tham số theo hướng ngược lại của gradient.

$$w = w - \eta \nabla J(w)$$

Trong đó:

- η : Tốc độ học (learning rate)
- $\nabla J(w)$: Đạo hàm của hàm chi phí

Mã Python

```
def gradient_descent(w, grad, learning_rate):  
    return w - learning_rate * grad  
  
w = 10  
learning_rate = 0.1  
for _ in range(100):  
    grad = 2 * w  
    w = gradient_descent(w, grad, learning_rate)  
print("Minimum at:", w)
```

Tóm tắt

- Thuật toán tối ưu quan trọng trong huấn luyện mạng nơ-ron.
- Tốc độ học ảnh hưởng đến tốc độ hội tụ và độ ổn định.
- Có ba biến thể chính: toàn bộ, ngẫu nhiên, và mini-batch.

5.3 Lan truyền ngược (Backpropagation)

Lý thuyết

Backpropagation tính toán hiệu quả gradient của hàm mất mát với từng trọng số bằng cách áp dụng quy tắc chuỗi.

Các bước:

1. Lan truyền xuôi
2. Tính toán hàm mất mát
3. Lan truyền ngược
4. Cập nhật trọng số

Mã Python

```
import torch
import torch.nn as nn

model = nn.Sequential(
    nn.Linear(2, 2),
    nn.ReLU(),
    nn.Linear(2, 1)
)

criterion = nn.MSELoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.01)

x = torch.tensor([[1.0, 2.0]])
y = torch.tensor([[0.0]])

output = model(x)
loss = criterion(output, y)
loss.backward()
optimizer.step()
```

Tóm tắt

- Cho phép huấn luyện mạng nơ-ron sâu.
- Sử dụng quy tắc chuỗi để tính gradient.
- Được triển khai trong các thư viện hiện đại (vd: PyTorch, TensorFlow).

5.4 Hàm kích hoạt (Activation Functions)

Lý thuyết

Hàm kích hoạt giúp mạng nơ-ron có tính phi tuyến.

Một số hàm kích hoạt phổ biến:

- ReLU: $f(x) = \max(0, x)$
- Sigmoid: $f(x) = \frac{1}{1+e^{-x}}$
- Tanh: $f(x) = \tanh(x)$

Mã Python

```
import torch.nn.functional as F
import torch

x = torch.tensor([-1.0, 0.0, 1.0])
print("ReLU:", F.relu(x))
print("Sigmoid:", torch.sigmoid(x))
print("Tanh:", torch.tanh(x))
```

Tóm tắt

- Cần thiết để tạo ranh giới quyết định phức tạp.
- ReLU thường dùng trong các lớp ẩn.
- Lựa chọn hàm kích hoạt tùy thuộc vào hành vi và yêu cầu của mô hình.

5.5 Hàm mất mát (Cost Functions)

Lý thuyết

Hàm mất mát dùng để đo lường sai số của mô hình.

Một số hàm mất mát phổ biến:

- MSE (Sai số bình phương trung bình): $\frac{1}{n} \sum (y_i - \hat{y}_i)^2$
- MAE (Sai số tuyệt đối trung bình): $\frac{1}{n} \sum |y_i - \hat{y}_i|$
- Cross Entropy: $-\sum y \log(\hat{y})$

Mã Python

```
import torch
import torch.nn as nn

y_true = torch.tensor([[1.0], [0.0]])
y_pred = torch.tensor([[0.8], [0.3]])

mse = nn.MSELoss()
print("MSE:", mse(y_pred, y_true).item())

y_true_cls = torch.tensor([1])
y_pred_cls = torch.tensor([0.1, 0.9])
ce = nn.CrossEntropyLoss()
print("Cross Entropy:", ce(y_pred_cls, y_true_cls).item())
```

Tóm tắt

- MSE và MAE thường dùng cho bài toán hồi quy.
- Cross Entropy dùng cho bài toán phân loại.
- Việc lựa chọn hàm mất mát ảnh hưởng đến hiệu quả học của mô hình.

Chapter 6

Deep learning

6.1 Perceptron Đa Lớp (MLP)

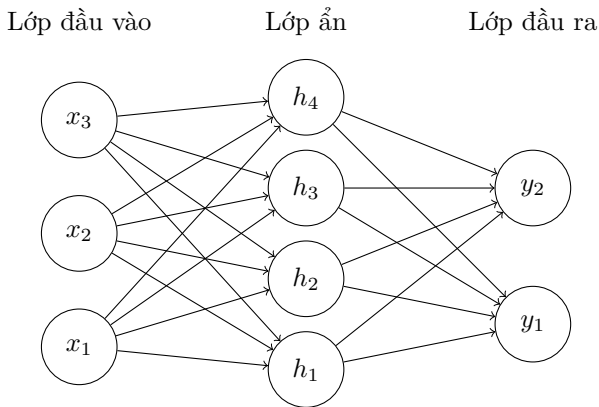


Figure 6.1: Kiến trúc MLP cơ bản với các lớp kết nối đầy đủ (fully connected).

6.1.1 MLP là gì?

Perceptron Đa Lớp (Multi-Layer Perceptron - MLP) là:

- Một loại mạng nơ-ron nhân tạo **cơ bản nhất**
- Gồm nhiều lớp nơ-ron kết nối **liên tiếp**
- Có khả năng học **mối quan hệ phi tuyến** giữa dữ liệu đầu vào và đầu

ra

6.1.2 Các thành phần chính

Tầng ẩn (Hidden Layers)

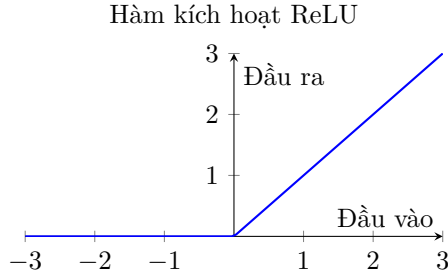


Figure 6.2: Hàm kích hoạt ReLU (Rectified Linear Unit) được dùng phổ biến trong các tầng ẩn.

- Thêm tính **phi tuyến** vào mạng
- Các hàm kích hoạt phổ biến:
 - ReLU: $f(x) = \max(0, x)$
 - Sigmoid: $f(x) = \frac{1}{1+e^{-x}}$
 - Tanh: $f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$

Lan truyền tiến (Forward Propagation)

Quá trình tính toán đầu ra:

$$h = \sigma(W_1x + b_1)$$

$$y = \sigma(W_2h + b_2)$$

Trong đó:

- W_1, W_2 : Trọng số
- b_1, b_2 : Độ chệch (bias)

- σ : Hàm kích hoạt

6.1.3 Cài đặt MLP bằng PyTorch

```
import torch
import torch.nn as nn

class MLP(nn.Module):
    def __init__(self, input_size=784, hidden_size=128,
                  num_classes=10):
        super().__init__()
        self.fc1 = nn.Linear(input_size, hidden_size)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(hidden_size, num_classes)

    def forward(self, x):
        x = x.view(x.size(0), -1)
        out = self.fc1(x)
        out = self.relu(out)
        out = self.fc2(out)
        return out

model = MLP(input_size=784, hidden_size=256, num_classes=10)

criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.01)
```

6.1.4 Huấn luyện MLP trên MNIST

```
from torchvision import datasets, transforms

transform = transforms.Compose([transforms.ToTensor()])

train_data = datasets.MNIST(root='data', train=True, transform
                             =transform, download=True)
train_loader = torch.utils.data.DataLoader(train_data,
                                             batch_size=64, shuffle=True)
```

```
for epoch in range(10):  
    for images, labels in train_loader:  
        outputs = model(images)  
        loss = criterion(outputs, labels)  
  
        optimizer.zero_grad()  
        loss.backward()  
        optimizer.step()  
  
print(f'Epoch {epoch+1}, Loss: {loss.item():.4f}')
```

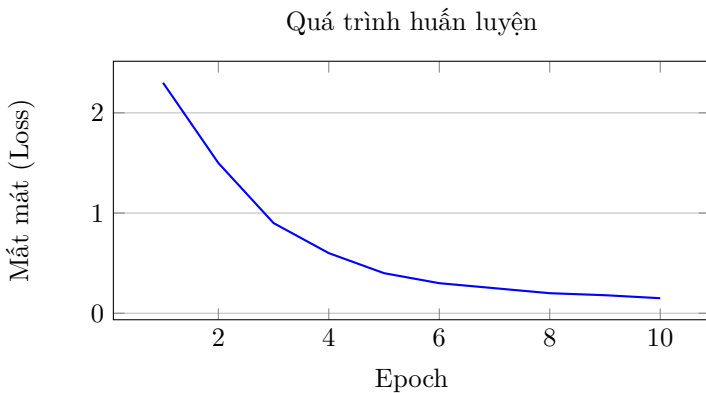


Figure 6.3: Biểu đồ mất mát giảm dần qua các epoch khi huấn luyện MLP.

6.1.5 Tại sao MLP hoạt động?

Định lý xấp xỉ phổ quát

MLP với một tầng ẩn đủ lớn có thể xấp xỉ bất kỳ hàm liên tục nào với độ chính xác tùy ý.

Backpropagation

Thuật toán lan truyền ngược giúp:

1. Tính toán gradient của hàm mất mát

2. Cập nhật trọng số bằng gradient descent

Công thức cập nhật trọng số:

$$W_{new} = W_{old} - \eta \cdot \frac{\partial \mathcal{L}}{\partial W}$$

với η là tốc độ học.

6.1.6 Mẹo thực hành cho cuộc thi

- Khởi tạo trọng số bằng `nn.init.kaiming_normal_`
- Thử nhiều hàm kích hoạt khác nhau (ReLU thường tốt nhất)
- Điều chỉnh số nơ-ron trong tầng ẩn (thường 64-512)
- Thêm Batch Normalization để ổn định huấn luyện
- Sử dụng Dropout để tránh overfitting

Ví dụ điều chỉnh kiến trúc:

```
class BetterMLP(nn.Module):  
    def __init__(self):  
        super().__init__()  
        self.net = nn.Sequential(  
            nn.Linear(784, 512),  
            nn.BatchNorm1d(512),  
            nn.ReLU(),  
            nn.Dropout(0.2),  
            nn.Linear(512, 256),  
            nn.ReLU(),  
            nn.Linear(256, 10)  
        )
```

6.1.7 Bài tập thực hành

1. Triển khai MLP nhận diện chữ số MNIST (đạt >98% accuracy)
2. Thử nghiệm với số lượng tầng ẩn khác nhau (1-5 tầng)
3. So sánh các hàm kích hoạt (ReLU vs Sigmoid vs Tanh)

4. Áp dụng MLP cho bài toán hồi quy (dự đoán giá nhà)

Lưu ý quan trọng: MLP phù hợp cho dữ liệu có cấu trúc (tabular data) nhưng không hoạt động tốt với ảnh (nên dùng CNN) hoặc chuỗi thời gian (nên dùng RNN/LSTM).

6.2 Stochastic Gradient Descent (SGD) và Mini-Batch Gradient Descent

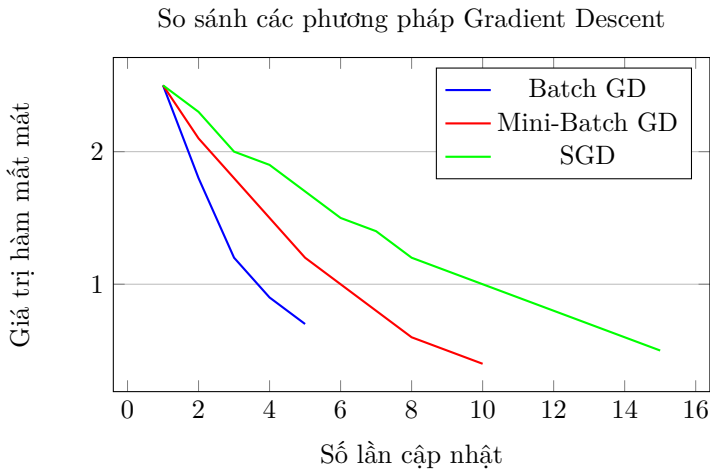


Figure 6.4: So sánh tốc độ hội tụ của các biến thể Gradient Descent khác nhau. Batch GD cập nhật mỗi epoch, Mini-Batch GD cập nhật sau mỗi batch, SGD cập nhật liên tục từng điểm dữ liệu.

6.2.1 Tổng quan về Gradient Descent

Gradient Descent (GD) là thuật toán tối ưu cơ bản nhất trong deep learning, hoạt động bằng cách:

- Tính gradient (độ dốc) của hàm mất mát
- Cập nhật tham số theo hướng ngược gradient

- Lặp lại cho đến khi hội tụ

Công thức cập nhật:

$$\theta_{t+1} = \theta_t - \eta \cdot \nabla_{\theta} J(\theta)$$

Trong đó:

- θ : Các tham số của mô hình
- η : Tốc độ học (learning rate)
- $\nabla_{\theta} J(\theta)$: Gradient của hàm mất mát

6.2.2 Các biến thể chính

Batch Gradient Descent

- Tính toán gradient trên TOÀN BỘ tập dữ liệu
- Ưu điểm: Hướng cập nhật chính xác
- Nhược điểm: Chậm với dữ liệu lớn

Stochastic Gradient Descent (SGD)

- Tính toán gradient trên TỪNG điểm dữ liệu
- Ưu điểm:
 - Tốc độ nhanh
 - Thoát cực tiểu địa phương tốt
- Nhược điểm:
 - Nhiễu lớn, dao động mạnh
 - Cần lịch trình learning rate

```
optimizer = torch.optim.SGD(model.parameters(), lr=0.01)
```

Mini-Batch Gradient Descent

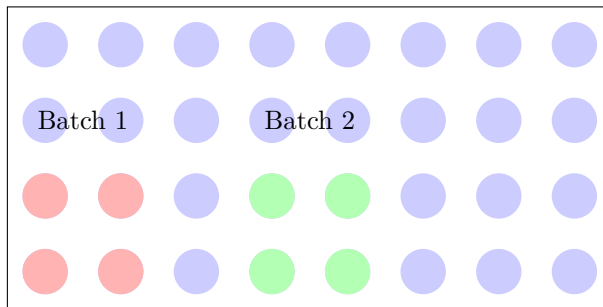
- Cân bằng giữa Batch GD và SGD

- Tính gradient trên từng batch nhỏ (thường 32-256 mẫu)
- Phổ biến nhất trong thực tế

```
from torch.utils.data import DataLoader

train_loader = DataLoader(dataset, batch_size=64, shuffle=True)

for epoch in range(epochs):
    for batch_idx, (data, target) in enumerate(train_loader):
        optimizer.zero_grad()
        output = model(data)
        loss = criterion(output, target)
        loss.backward()
        optimizer.step()
```



Mini-Batch = Nhóm các điểm dữ liệu được xử lý cùng lúc

Figure 6.5: Minh họa Mini-Batch GD: Dữ liệu được chia thành các batch nhỏ, mỗi lần cập nhật sử dụng 1 batch.

6.2.3 Lựa chọn batch size

- **Batch nhỏ (1-32):**
 - Tốc độ hội tụ nhanh hơn
 - Tiếng ồn giúp thoát cực tiểu địa phương
- **Batch lớn (256+):**

- Tận dụng tốt GPU
- Gradient ổn định hơn

6.3 Phương pháp Momentum (Adam và AdamW)

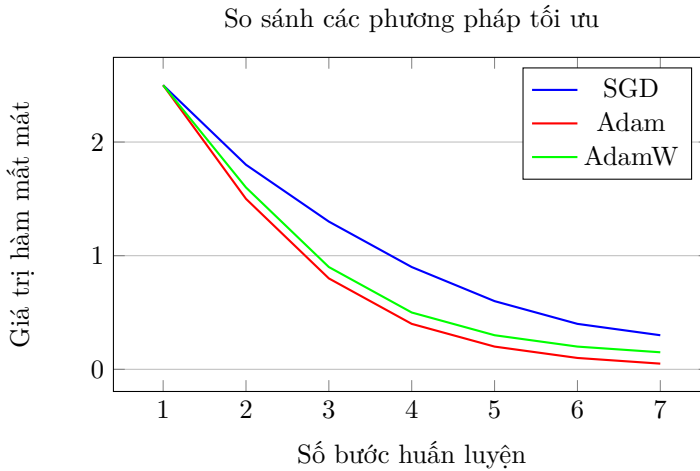


Figure 6.6: So sánh tốc độ hội tụ của các bộ tối ưu hóa khác nhau. Adam và AdamW thường hội tụ nhanh hơn SGD thông thường.

6.3.1 Tại sao cần phương pháp Momentum?

Các vấn đề với SGD thông thường:

- Dao động nhiều ở các vùng hẹp
- Tốc độ hội tụ chậm
- Nhạy cảm với learning rate

Momentum giúp:

- Giảm dao động bằng cách "tích lũy" gradient

- Tăng tốc hội tụ ở hướng ổn định
- Vượt qua cực tiểu địa phương tốt hơn

6.3.2 Các phương pháp Momentum phổ biến

Adam (Adaptive Moment Estimation)

- Kết hợp Momentum và RMSprop
- Tự động điều chỉnh learning rate cho từng tham số
- Công thức cập nhật:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (\text{Momentum})$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (\text{RMSprop})$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

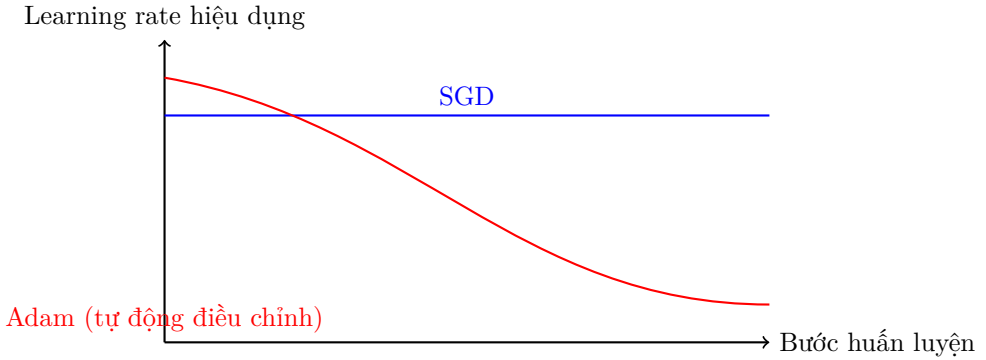
$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

```
optimizer = torch.optim.Adam(  
    model.parameters(),  
    lr=0.001,  
    betas=(0.9, 0.999),  
    eps=1e-8  
)
```

AdamW (Adam with Weight Decay)

- Cải tiến của Adam với weight decay đúng cách
- Giúp mô hình tổng quát hóa tốt hơn
- Đặc biệt hiệu quả với Transformer và CNN lớn

```
optimizer = torch.optim.AdamW(
    model.parameters(),
    lr=0.001,
    weight_decay=0.01
)
```



Adam giảm learning rate tự động cho các tham số phổ biến

Figure 6.7: Adam tự động điều chỉnh learning rate khác nhau cho từng tham số.

6.3.3 Khi nào sử dụng Adam/AdamW?

- **Adam** phù hợp khi:
 - Dữ liệu có nhiều nhiễu
 - Cần hội tụ nhanh
 - Không biết chọn learning rate nào
- **AdamW** tốt hơn khi:
 - Mô hình lớn, dễ overfitting
 - Cần regularization mạnh
 - Huấn luyện Transformer
- **Không nên dùng** khi:
 - Dữ liệu cực kỳ nhỏ

- Cần kết quả tái tạo chính xác

6.3.4 Thực hành với PyTorch

```
import torch
from torch.optim import Adam, AdamW

model = torch.nn.Sequential(
    torch.nn.Linear(784, 256),
    torch.nn.ReLU(),
    torch.nn.Linear(256, 10)
)

optimizer = AdamW(
    model.parameters(),
    lr=1e-3,
    betas=(0.9, 0.999),
    eps=1e-8,
    weight_decay=0.01
)

for epoch in range(10):
    for batch_idx, (data, target) in enumerate(train_loader):
        outputs = model(data)
        loss = criterion(outputs, target)

        optimizer.zero_grad()
        loss.backward()

        optimizer.step()

    print(f'Epoch {epoch+1}, Loss: {loss.item():.4f}')
```

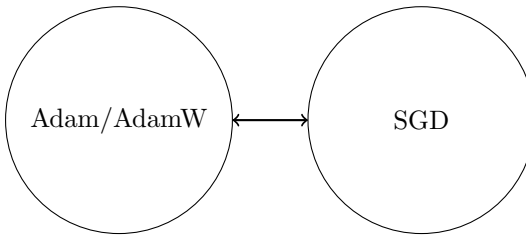
6.3.5 Mẹo thi đấu IOAI

- Luôn thử Adam/AdamW trước khi dùng SGD
- Learning rate mặc định 0.001 thường hoạt động tốt

- Giảm learning rate 10 lần nếu loss không giảm
- Dùng weight decay (0.01-0.1) để tránh overfitting
- Đối với bài toán nhỏ, có thể thử SGD + Momentum
- Ghi lại lịch sử loss để so sánh các optimizer

Câu hỏi kiểm tra:

1. Adam khác SGD như thế nào?
2. Tại sao AdamW tốt hơn Adam cho mô hình lớn?
3. Khi nào không nên sử dụng Adam?



- | | |
|-------------------------|---------------------|
| • Tự động điều chỉnh LR | • Kiểm soát tốt hơn |
| • Hội tụ nhanh | • Kết quả ổn định |
| • Ít tuning | • Cần tuning LR |

Figure 6.8: So sánh đặc điểm giữa Adam và SGD.

6.4 Tốc độ học thích ứng (Adaptive Learning Rates)

6.4.1 Tại sao cần tốc độ học thích ứng?

- Vấn đề:
 - Learning rate cố định có thể quá lớn (dao động) hoặc quá nhỏ (chậm hội tụ)

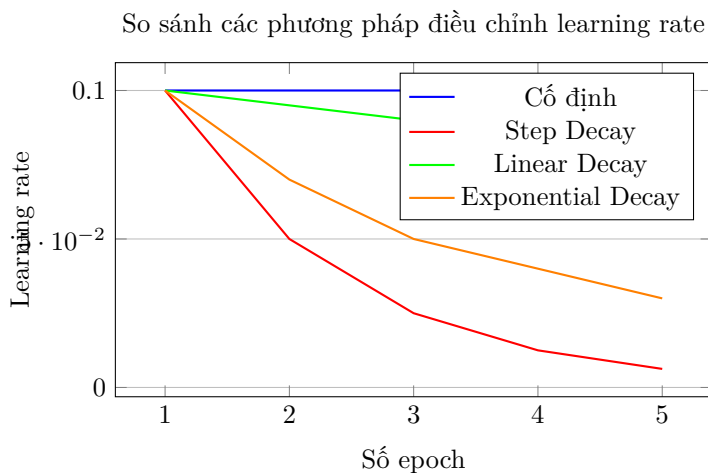


Figure 6.9: Các phương pháp điều chỉnh learning rate phổ biến. Tốc độ học thích ứng giúp mô hình hội tụ tốt hơn.

- Độ nhạy gradient khác nhau giữa các tham số
- **Giải pháp:**
 - Tự động điều chỉnh learning rate trong quá trình huấn luyện
 - Áp dụng learning rate khác nhau cho từng tham số

6.4.2 Các phương pháp chính

Learning Rate Schedulers

```
scheduler = torch.optim.lr_scheduler.StepLR(
    optimizer,
    step_size=30,
    gamma=0.1
)

scheduler = torch.optim.lr_scheduler.ExponentialLR(
    optimizer,
    gamma=0.9
)
```

```
)

scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
    optimizer,
    T_max=50
)
```

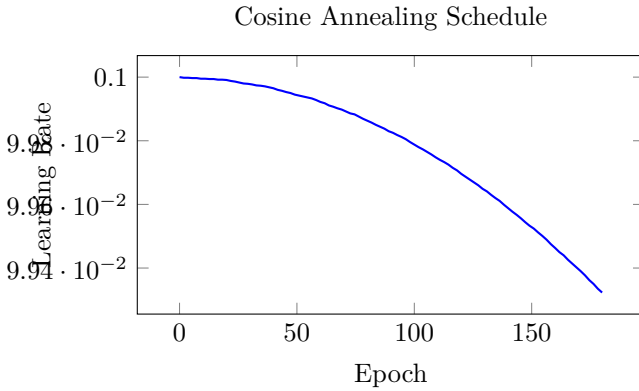


Figure 6.10: Cosine Annealing giúp learning rate giảm dần theo dạng sóng cosine, thường cho kết quả tốt hơn cách giảm đều.

Adaptive Optimizers

- **Adam:** Kết hợp momentum và điều chỉnh learning rate theo từng tham số
- **RMSprop:** Điều chỉnh dựa trên trung bình động của gradient bình phương
- **Adagrad:** Thích ứng dựa trên lịch sử gradient

```
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer,
    mode='min',
    factor=0.1,
```

```
    patience=5
)
```

6.4.3 Thực hành với PyTorch

```
import torch
from torch.optim import SGD
from torch.optim.lr_scheduler import StepLR, ReduceLROnPlateau

model = torch.nn.Linear(10, 2)
optimizer = SGD(model.parameters(), lr=0.1)

scheduler = StepLR(optimizer, step_size=30, gamma=0.1)

for epoch in range(100):
    train(...)

    val_loss = validate(...)

    scheduler.step()

    print(f'Epoch {epoch+1}, LR: {optimizer.param_groups[0]["lr"]:.6f}, Loss: {val_loss:.4f}')
```

6.4.4 Mẹo thi đấu IOAI

- Luôn bắt đầu với learning rate nhỏ (0.001-0.01) khi dùng Adam
- Thử CosineAnnealing cho bài toán vision
- Dùng ReduceLROnPlateau khi loss bị "kẹt"
- Giám sát learning rate trong quá trình huấn luyện
- Kết hợp warmup (tăng LR dần) cho mô hình lớn
- Ghi lại lịch sử LR và loss để phân tích

Câu hỏi kiểm tra:

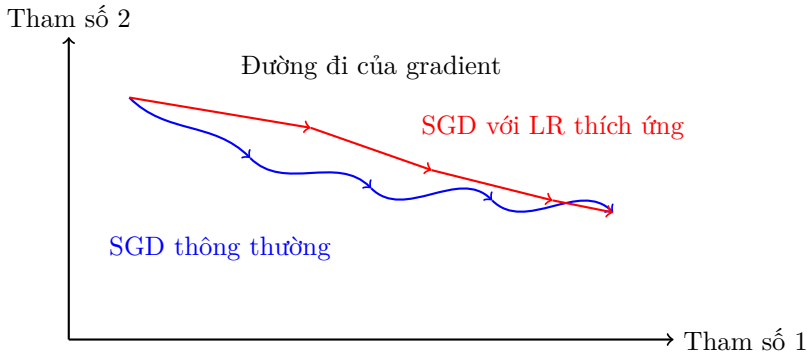


Figure 6.11: Minh họa đường đi tối ưu với learning rate thích ứng (đỏ) so với SGD thông thường (xanh).

1. Tại sao learning rate thích ứng quan trọng?
2. Khen nào nên dùng StepLR vs CosineAnnealing?
3. Làm thế nào để phát hiện learning rate không phù hợp?



Learning rate quá lớn → quá nhỏ

Figure 6.12: Minh họa phạm vi learning rate từ quá lớn (gây dao động) đến quá nhỏ (hội tụ chậm).

6.5 Sự Hội Tụ và Tốc Độ Học

6.5.1 Tốc độ học là gì?

Tốc độ học (Learning Rate - LR) là siêu tham số quan trọng nhất dalam huấn luyện mạng nơ-ron, xác định kích thước bước cập nhật trọng số:

$$\theta_{t+1} = \theta_t - \eta \cdot \nabla J(\theta_t) \quad (6.1)$$

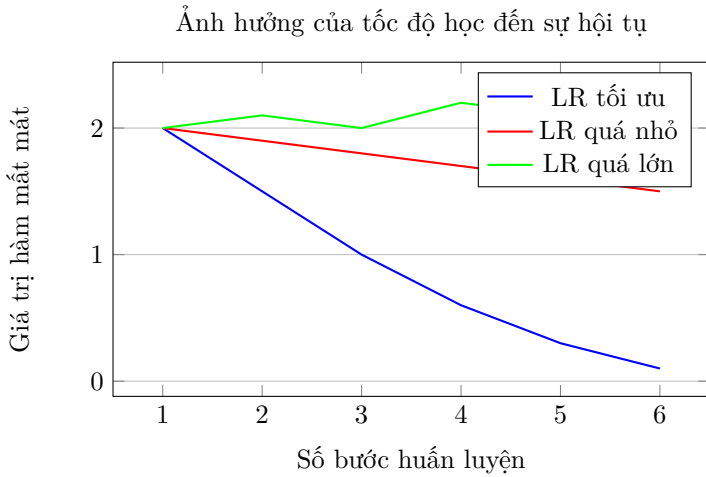


Figure 6.13: Biểu đồ cho thấy ảnh hưởng của tốc độ học (Learning Rate - LR) đến quá trình hội tụ của mô hình.

Trong đó:

- η : Tốc độ học
- $\nabla J(\theta_t)$: Gradient của hàm mất mát

6.5.2 Các vấn đề về hội tụ

LR quá lớn

- Mất mát dao động mạnh
- Không hội tụ được
- Có thể bị "nổ gradient"

LR quá nhỏ

- Huấn luyện chậm
- Mắc kẹt ở cực tiểu địa phương

- Không tận dụng được khả năng tính toán

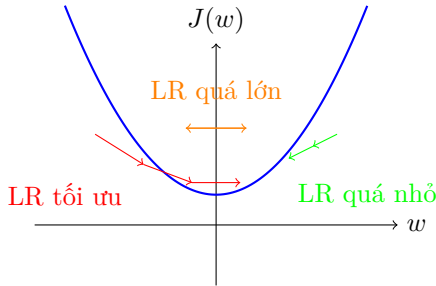


Figure 6.14: Minh họa ảnh hưởng của tốc độ học lên quá trình tối ưu hóa.

6.5.3 Cách chọn LR phù hợp

Phương pháp thử nghiệm

```
learning_rates = [1e-5, 1e-4, 1e-3, 1e-2]
for lr in learning_rates:
    optimizer = torch.optim.SGD(model.parameters(), lr=lr)
    train(model, optimizer)
    accuracy = evaluate(model)
    print(f"LR: {lr}, Accuracy: {accuracy:.2f}")
```

Learning Rate Finder

```
from torch_lr_finder import LRFinder

optimizer = torch.optim.Adam(model.parameters(), lr=1e-7)
criterion = nn.CrossEntropyLoss()
lr_finder = LRFinder(model, optimizer, criterion)

lr_finder.range_test(train_loader, end_lr=1, num_iter=100)
lr_finder.plot()
lr_finder.reset()
```

6.5.4 Theo dõi sự hội tụ

```
def train(model, optimizer, n_epochs=10):
    losses = []
    for epoch in range(n_epochs):
        model.train()
        for inputs, targets in train_loader:
            optimizer.zero_grad()
            outputs = model(inputs)
            loss = criterion(outputs, targets)
            loss.backward()
            optimizer.step()
            losses.append(loss.item())

        model.eval()
        with torch.no_grad():
            accuracy = evaluate(model, val_loader)

        print(f"Epoch {epoch+1}/{n_epochs} | Loss: {loss.item():.4f} | Acc: {accuracy:.2f}")

    plt.plot(losses)
    plt.xlabel('Iteration')
    plt.ylabel('Loss')
    plt.title('Training Convergence')
    plt.show()
```

6.5.5 Mẹo thi đấu IOAI

- Bắt đầu với LR khoảng 0.001-0.01 cho Adam, 0.1 cho SGD
- Sử dụng learning rate scheduling
- Theo dõi biểu đồ loss để phát hiện vấn đề sớm
- Thử nghiệm với nhiều LR khác nhau
- Sử dụng warmup cho mô hình lớn
- Ghi lại tất cả thử nghiệm để so sánh

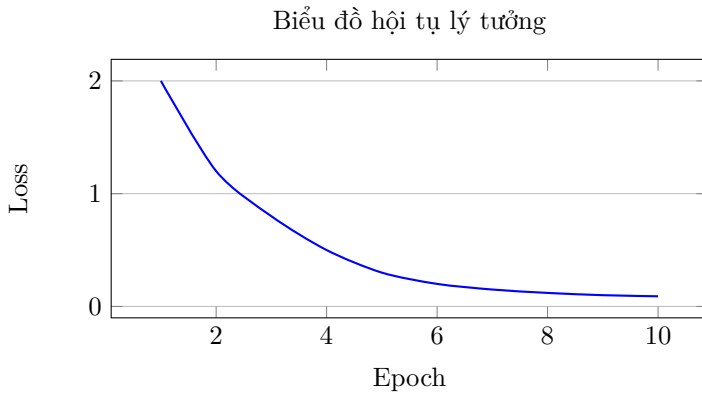


Figure 6.15: Biểu đồ hội tụ lý tưởng với loss giảm dần và ổn định.

Dấu hiệu nhận biết vấn đề:

- Loss không giảm: LR quá nhỏ
- Loss dao động mạnh: LR quá lớn
- Loss giảm rồi tăng: LR cần giảm dần

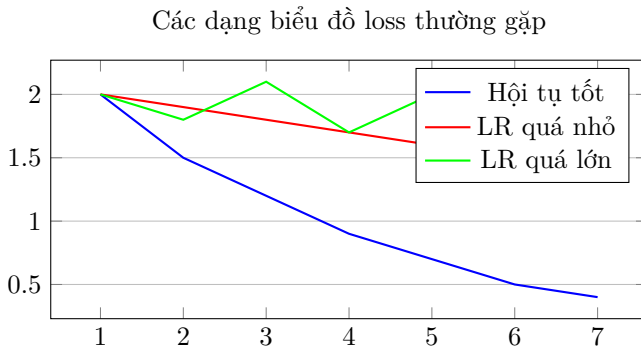


Figure 6.16: Nhận biết vấn đề qua biểu đồ loss.

6.6 Điều chuẩn Trọng số

(Weight Regularization)

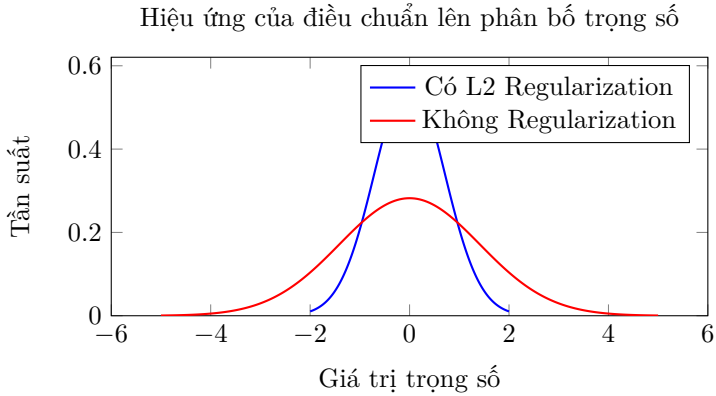


Figure 6.17: Regularization giúp phân bố trọng số tập trung quanh 0, ngăn chặn giá trị quá lớn.

6.6.1 Tại sao cần điều chuẩn trọng số?

- Vấn đề:
 - Overfitting: Mô hình học thuộc lòng dữ liệu huấn luyện
 - Trọng số có giá trị quá lớn
 - Hiệu suất kém trên dữ liệu mới
- Giải pháp:
 - Thêm vào hàm mất mát một thành phần phạt trọng số lớn
 - Đơn giản nhất: L1/L2 Regularization

6.6.2 Các phương pháp chính

L2 Regularization (Weight Decay)

$$J_{new}(\theta) = J(\theta) + \lambda \sum_{i=1}^n \theta_i^2 \quad (6.2)$$

- Ưu điểm: Giúp trọng số nhỏ đều
- Nhược điểm: Không đưa về 0 hoàn toàn

L1 Regularization (Lasso)

$$J_{new}(\theta) = J(\theta) + \lambda \sum_{i=1}^n |\theta_i| \quad (6.3)$$

- Ưu điểm: Tạo sự thưa thớt (đưa một số trọng số về 0)
- Nhược điểm: Không ổn định với features tương quan cao

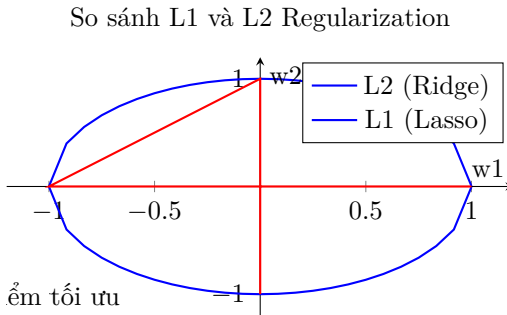


Figure 6.18: L1 (đỏ) có xu hướng giao với trục tọa độ, tạo tính thưa thớt. L2 (xanh) tạo phân bố đều.

6.6.3 Cài đặt trong PyTorch

Weight Decay với Optimizer

```
optimizer = torch.optim.SGD(
    model.parameters(),
    lr=0.01,
    weight_decay=0.01
)
```

Tự implement Regularization

```
def l2_regularization(model, lambda_l2=0.01):
    reg_loss = 0.0
    for param in model.parameters():
        reg_loss += torch.norm(param, p=2)
    return lambda_l2 * reg_loss

loss = criterion(outputs, labels) + l2_regularization(model)
```

6.6.4 Khi nào nên sử dụng?

Phương pháp	Tính chất	Ứng dụng
L1 Regularization	Tạo sự thưa thớt	Feature selection
L2 Regularization	Trọng số nhỏ đều	Ngăn overfitting
Elastic Net	Kết hợp L1 + L2	Dữ liệu nhiều features

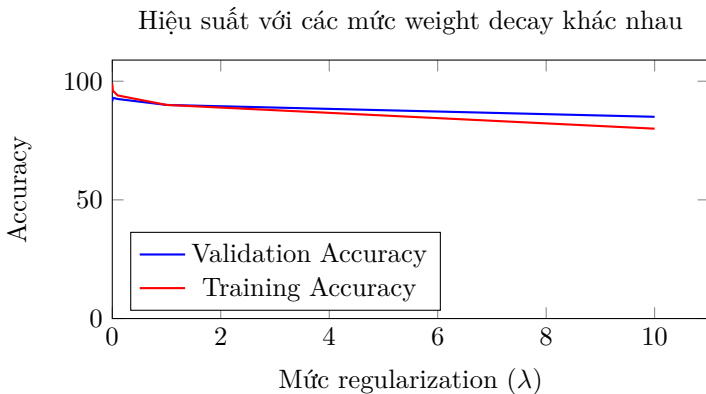


Figure 6.19: Tác động của mức regularization (λ) lên hiệu suất. Giá trị λ quá lớn làm giảm độ chính xác cả tập train và validation.

6.6.5 Mẹo thi đấu IOAI

- Bắt đầu với `weight_decay = 0.01` và điều chỉnh theo log scale
- Kết hợp với Dropout và BatchNorm sẽ hiệu quả hơn

- Kiểm tra phân bố trọng số sau huấn luyện
- Thử L1 nếu muốn làm thưa mô hình
- Ghi lại λ đã thử để so sánh hiệu quả

Dấu hiệu nhận biết:

- Training loss « Validation loss $\rightarrow$ Cần tăng λ
- Cả hai loss đều cao $\rightarrow$ Cần giảm λ
- Độ chính xác tăng rồi giảm khi tăng $\lambda \rightarrow$ Chọn λ tại điểm cao nhất trên validation

```
def plot_weights(model):  
    weights = []  
    for param in model.parameters():  
        if param.dim() > 1:  
            weights.extend(param.view(-1).tolist())  
  
    plt.hist(weights, bins=50)  
    plt.title("Phan bo trong so sau huan luyen")  
    plt.xlabel("Gia tri trong so")  
    plt.ylabel("Tan suat")  
    plt.show()  
  
plot_weights(model)
```

6.7 Dừng Sớm (Early Stopping)

6.7.1 Early Stopping là gì?

Early Stopping là kỹ thuật ngăn chặn overfitting bằng cách:

- Theo dõi hiệu suất trên tập validation
- Dừng huấn luyện khi hiệu suất không cải thiện
- Lưu lại trọng số tốt nhất

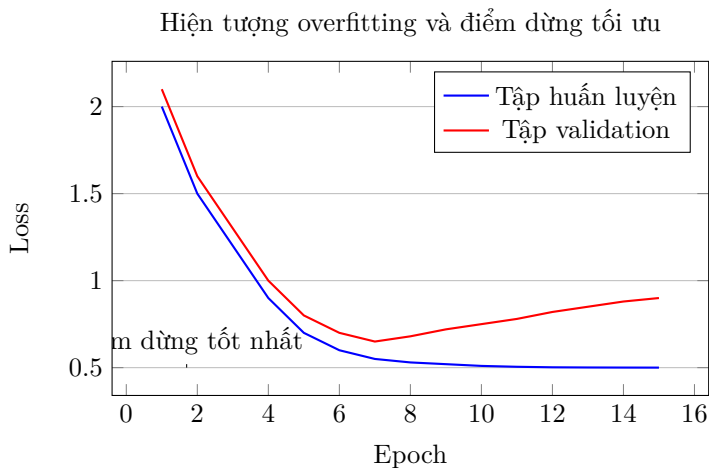


Figure 6.20: Khi loss tập validation bắt đầu tăng trong khi loss tập train tiếp tục giảm, đây là dấu hiệu overfitting và cần áp dụng early stopping.

6.7.2 Tại sao cần Early Stopping?

- **Tiết kiệm thời gian:** Không cần huấn luyện quá lâu
- **Ngăn overfitting:** Dừng trước khi mô hình học thuộc dữ liệu
- **Dễ triển khai:** Không cần thay đổi kiến trúc mô hình

6.7.3 Cài đặt trong PyTorch

Cách 1: Tự implement

```
import numpy as np
import torch

class EarlyStopping:
    def __init__(self, patience=5, delta=0):
        self.patience = patience
        self.delta = delta
        self.counter = 0
        self.best_score = None
```

```
self.early_stop = False

def __call__(self, val_loss, model):
    score = -val_loss

    if self.best_score is None:
        self.best_score = score
        self.save_checkpoint(val_loss, model)
    elif score < self.best_score + self.delta:
        self.counter += 1
        print(f'EarlyStopping counter: {self.counter}/{
            self.patience}')
        if self.counter >= self.patience:
            self.early_stop = True
    else:
        self.best_score = score
        self.save_checkpoint(val_loss, model)
        self.counter = 0

def save_checkpoint(self, val_loss, model):
    torch.save(model.state_dict(), 'checkpoint.pt')
    print(f'Validation loss decreased ({self.best_score:.6
        f} --> {-val_loss:.6f}). Saving model...')
```

Cách 2: Sử dụng Callback

```
from pytorch_lightning.callbacks import EarlyStopping

early_stop_callback = EarlyStopping(
    monitor='val_loss',
    patience=5,
    mode='min',
    verbose=True
)

trainer = Trainer(callbacks=[early_stop_callback])
```

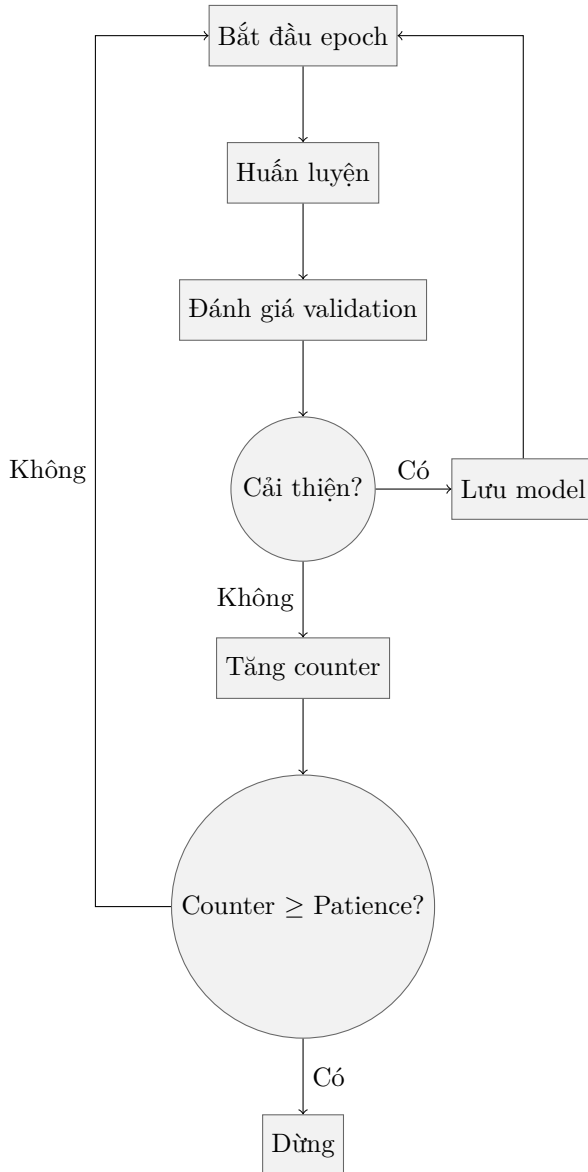


Figure 6.21: Lưu đồ thuật toán Early Stopping. Quá trình sẽ dừng khi không cải thiện sau một số epoch nhất định.

6.7.4 Cách sử dụng hiệu quả

Chọn tham số

- **Patience:** Thường từ 5-20 epoch
- **Delta:** Ngưỡng cải thiện tối thiểu (ví dụ: 0.001)
- **Theo dõi:** Validation loss hoặc độ chính xác

Training loop với Early Stopping

```
early_stopping = EarlyStopping(patience=7, delta=0.001)

for epoch in range(100):
    train_loss = train_one_epoch(model, train_loader)

    val_loss = evaluate(model, val_loader)

    early_stopping(val_loss, model)
    if early_stopping.early_stop:
        print("Early stopping triggered!")
        break

model.load_state_dict(torch.load('checkpoint.pt'))
```

6.7.5 Mẹo thi đấu IOAI

- Luôn sử dụng validation set riêng biệt
- Theo dõi cả loss và độ chính xác
- Kết hợp với Model Checkpointing
- Thử nghiệm với các patience khác nhau
- Vẽ biểu đồ loss để xác định điểm dừng phù hợp

Dấu hiệu nhận biết:

- Validation loss tăng liên tục trong khi training loss giảm

- Độ chính xác validation không cải thiện sau nhiều epoch
- Khoảng cách giữa training loss và validation loss lớn

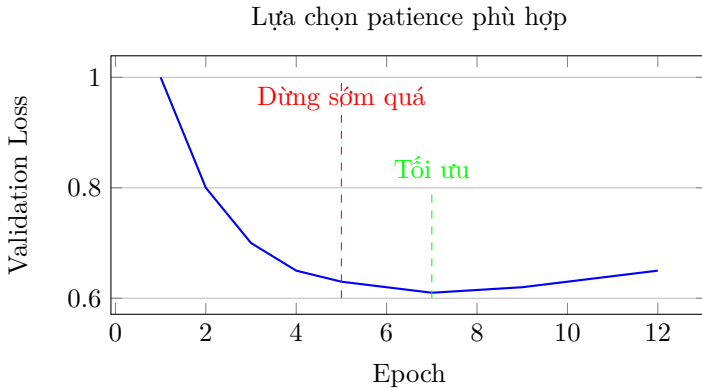


Figure 6.22: Ví dụ về lựa chọn patience. Nếu chọn patience quá nhỏ có thể dừng khi model chưa hội tụ tốt.

6.8 Dropout và Gaussian Noise

6.8.1 Dropout là gì?

Dropout là kỹ thuật regularization:

- **Cách hoạt động:** Tắt ngẫu nhiên các neuron trong quá trình huấn luyện
- **Tỉ lệ dropout (p):** Xác suất một neuron bị tắt (thường 0.2-0.5)
- **Ưu điểm:**
 - Ngăn chặn overfitting
 - Tạo "ensemble" nhiều mạng con
 - Giảm sự phụ thuộc giữa các neuron

Cài đặt Dropout trong PyTorch

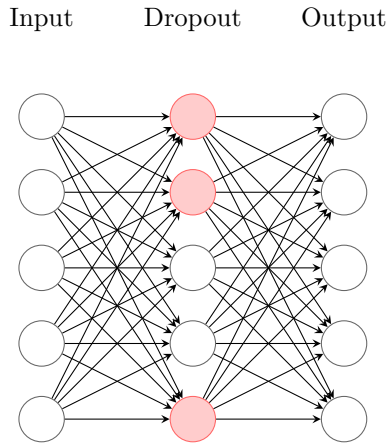


Figure 6.23: Minh họa Dropout trong mạng nơ-ron. Các neuron bị tắt ngẫu nhiên (màu đỏ) trong quá trình huấn luyện.

```
import torch.nn as nn

class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.fc1 = nn.Linear(784, 256)
        self.dropout = nn.Dropout(p=0.3)  # 30% dropout
        self.fc2 = nn.Linear(256, 10)

    def forward(self, x):
        x = nn.ReLU()(self.fc1(x))
        x = self.dropout(x)  # Only active during training
        x = self.fc2(x)
        return x
```

6.8.2 Gaussian Noise

Ý tưởng cơ bản

- Thêm nhiễu ngẫu nhiên vào:

- Đầu vào (input noise)
- Các lớp ẩn (hidden layer noise)
- Tác dụng tương tự Dropout:
 - Tăng tính tổng quát
 - Giúp mô hình robust hơn

Cài đặt Gaussian Noise

```
class GaussianNoise(nn.Module):
    def __init__(self, std=0.1):
        super().__init__()
        self.std = std

    def forward(self, x):
        if self.training:
            noise = torch.randn_like(x) * self.std
            return x + noise
        return x

class NetWithNoise(nn.Module):
    def __init__(self):
        super().__init__()
        self.noise1 = GaussianNoise(0.1)
        self.fc1 = nn.Linear(784, 256)
        self.noise2 = GaussianNoise(0.05)
        self.fc2 = nn.Linear(256, 10)

    def forward(self, x):
        x = self.noise1(x)
        x = nn.ReLU()(self.fc1(x))
        x = self.noise2(x)
        x = self.fc2(x)
        return x
```

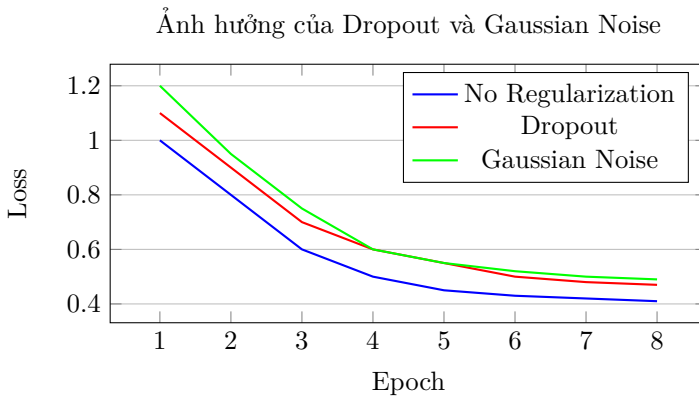



Figure 6.24: So sánh quá trình huấn luyện với các phương pháp regularization khác nhau.

6.8.3 Khi nào sử dụng?

Dropout phù hợp khi

- Mạng có nhiều lớp (deep networks)
- Kích thước tập huấn luyện nhỏ
- Cần giảm overfitting
- Dùng với Fully Connected Layers

Gaussian Noise tốt hơn khi

- Làm việc với dữ liệu nhiễu (ảnh, âm thanh)
- Xây dựng Autoencoders
- Dùng với các kiến trúc nhạy cảm với Dropout (RNN)

6.8.4 Mẹo thi đấu IOAI

- Bắt đầu với dropout rate 0.3-0.5 cho FC layers
- Không dùng dropout sau layer cuối cùng

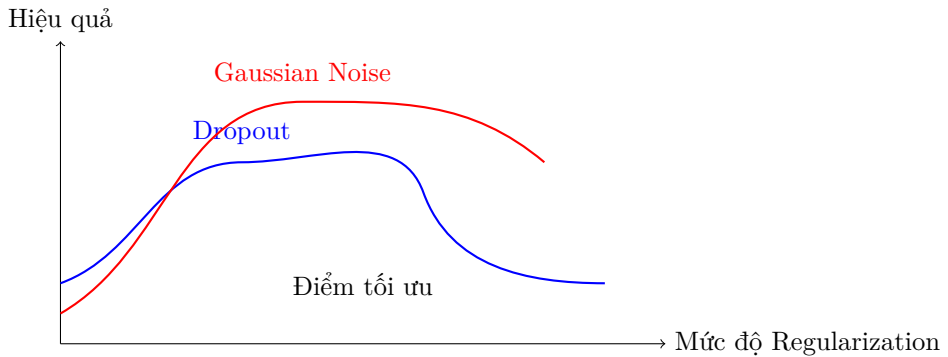


Figure 6.25: Mối quan hệ giữa mức độ regularization và hiệu quả. Quá nhiều regularization có thể làm giảm hiệu suất.

- Độ lệch chuẩn (std) của Gaussian Noise thường 0.05-0.2
- Kết hợp với BatchNorm để đạt hiệu quả tốt hơn
- Giảm dropout rate khi tăng kích thước mạng
- Với CNN: Dùng SpatialDropout thay vì Dropout thông thường

```
# Spatial Dropout for CNNs (drops entire channels)
class CNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 32, 3)
        self.dropout = nn.Dropout2d(p=0.2) # Spatial Dropout
        self.conv2 = nn.Conv2d(32, 64, 3)

    def forward(self, x):
        x = self.conv1(x)
        x = self.dropout(x)
        x = self.conv2(x)
        return x
```

Câu hỏi kiểm tra:

1. Tại sao không dùng Dropout trong quá trình test?
2. Khi nào nên dùng Gaussian Noise thay cho Dropout?

3. Làm thế nào để chọn dropout rate phù hợp?

6.9 Khởi tạo Trọng số (Weight Initialization)

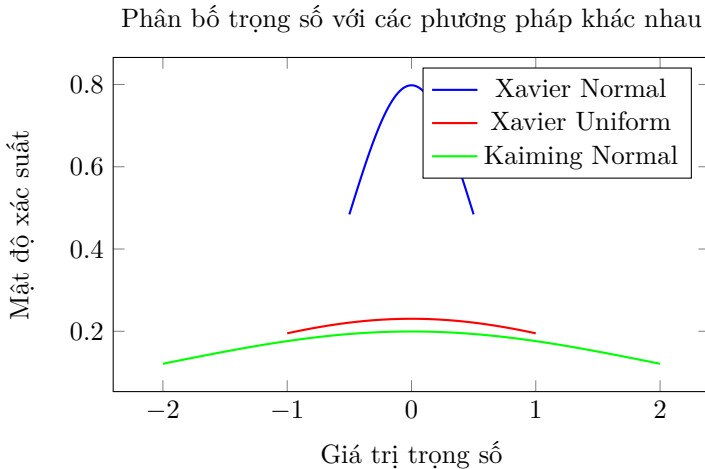


Figure 6.26: Các phương pháp khởi tạo trọng số khác nhau tạo ra phân bố giá trị khác nhau.

6.9.1 Tại sao cần khởi tạo đúng cách?

Khởi tạo trọng số ảnh hưởng lớn đến:

- Tốc độ hội tụ của mô hình
- Hiệu suất cuối cùng
- Vấn đề vanishing/exploding gradients

6.9.2 Các phương pháp phổ biến

Khởi tạo ngẫu nhiên đơn giản

```
import torch.nn as nn

layer = nn.Linear(100, 50)
nn.init.uniform_(layer.weight, -0.1, 0.1)
nn.init.zeros_(layer.bias)
```

Xavier/Glorot Initialization

- Phù hợp cho hàm kích hoạt Sigmoid/Tanh
- Công thức: $W \sim \mathcal{U}(-\sqrt{\frac{6}{n_{in}+n_{out}}}, \sqrt{\frac{6}{n_{in}+n_{out}}})$

```
# Xavier Uniform
nn.init.xavier_uniform_(layer.weight)

# Xavier Normal
nn.init.xavier_normal_(layer.weight)
```

Kaiming/He Initialization

- Phù hợp cho ReLU và biến thể
- Công thức: $W \sim \mathcal{N}(0, \sqrt{\frac{2}{n_{in}}})$

```
# Kaiming Normal cho ReLU
nn.init.kaiming_normal_(layer.weight, mode='fan_in',
    nonlinearity='relu')

# Kaiming Uniform cho LeakyReLU
nn.init.kaiming_uniform_(layer.weight, mode='fan_out',
    nonlinearity='leaky_relu')
```

6.9.3 Thực hành với PyTorch

```
class NeuralNet(nn.Module):
    def __init__(self, input_size, hidden_size, output_size):
        super(NeuralNet, self).__init__()
```

```
self.fc1 = nn.Linear(input_size, hidden_size)
self.relu = nn.ReLU()
self.fc2 = nn.Linear(hidden_size, output_size)

# Khởi tạo trọng số
self._initialize_weights()

def _initialize_weights(self):
    # Kaiming initialization cho layer có ReLU
    nn.init.kaiming_normal_(self.fc1.weight, mode='fan_in',
                             , nonlinearity='relu')
    nn.init.zeros_(self.fc1.bias)

    # Xavier initialization cho output layer
    nn.init.xavier_normal_(self.fc2.weight)
    nn.init.zeros_(self.fc2.bias)

def forward(self, x):
    out = self.fc1(x)
    out = self.relu(out)
    out = self.fc2(out)
    return out

# Model có 784 đầu vào, 256 nút ẩn, 10 đầu ra
model = NeuralNet(784, 256, 10)
```

6.9.4 Mẹo thi đấu IOAI

- Luôn khởi tạo trọng số thay vì để mặc định
- Với ReLU: Dùng Kaiming Normal/Uniform
- Với Sigmoid/Tanh: Dùng Xavier Normal/Uniform
- Khởi tạo bias bằng 0 hoặc giá trị nhỏ
- Kiểm tra phân bố trọng số sau khởi tạo

Câu hỏi kiểm tra:

1. Tại sao không nên khởi tạo tất cả trọng số bằng 0?

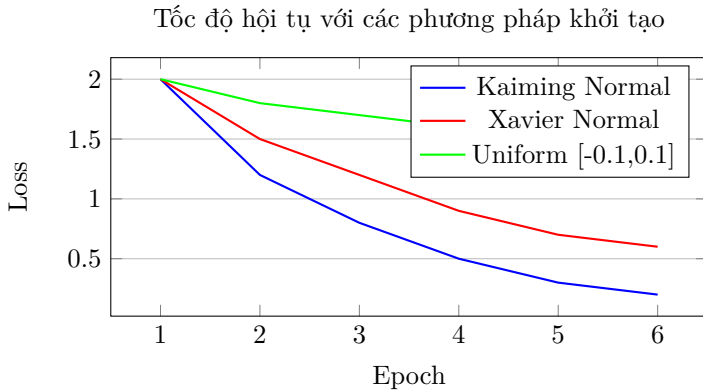


Figure 6.27: So sánh tốc độ hội tụ với các phương pháp khởi tạo khác nhau. Kaiming thường cho kết quả tốt hơn với ReLU.

2. Khi nào nên dùng Xavier thay vì Kaiming?
3. Làm sao để khởi tạo một lớp Conv2d trong CNN?

6.10 Chuẩn hóa theo Batch (Batch Normalization)

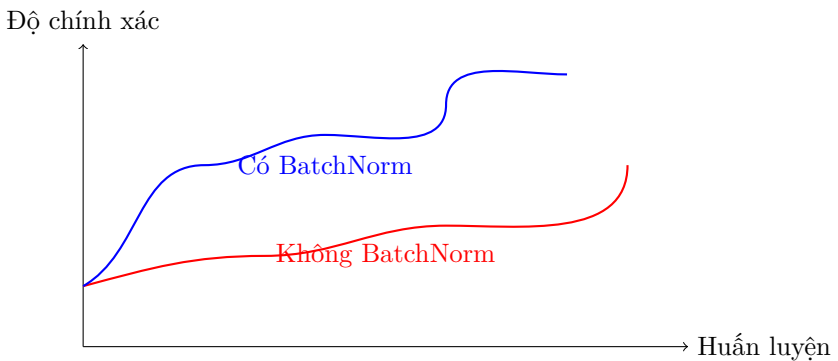


Figure 6.28: BatchNorm giúp mô hình hội tụ nhanh hơn và ổn định hơn.

6.10.1 BatchNorm là gì?

Batch Normalization là kỹ thuật:

- Chuẩn hóa đầu vào của mỗi tầng
- Tính trung bình và phương sai theo từng batch
- Áp dụng scale (γ) và shift (β) học được

Công thức:

$$y = \gamma \cdot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta \quad (6.4)$$

6.10.2 Tại sao cần BatchNorm?

- **Giảm Internal Covariate Shift:** Cân bằng phân phối đầu vào giữa các tầng
- **Cho phép learning rate lớn hơn:** Huấn luyện ổn định hơn
- **Giảm phụ thuộc vào khởi tạo trọng số**
- **Hoạt động như regularizer:** Giảm overfitting



Thường đặt sau tầng Linear, trước Activation

Figure 6.29: Vị trí thường dùng của BatchNorm trong mạng nơ-ron.

6.10.3 Cài đặt trong PyTorch

Cho mạng Fully Connected

```
import torch.nn as nn

class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.fc1 = nn.Linear(784, 256)
```

```
self.bn1 = nn.BatchNorm1d(256) # BatchNorm cho 1D
self.fc2 = nn.Linear(256, 10)

def forward(self, x):
    x = self.fc1(x)
    x = self.bn1(x)
    x = nn.ReLU()(x)
    x = self.fc2(x)
    return x
```

Cho mạng CNN

```
class CNN(nn.Module):
    def __init__(self):
        super(CNN, self).__init__()
        self.conv1 = nn.Conv2d(3, 16, 3)
        self.bn1 = nn.BatchNorm2d(16) # BatchNorm cho 2D
        self.conv2 = nn.Conv2d(16, 32, 3)
        self.bn2 = nn.BatchNorm2d(32)

    def forward(self, x):
        x = self.conv1(x)
        x = self.bn1(x)
        x = nn.ReLU()(x)
        x = self.conv2(x)
        x = self.bn2(x)
        x = nn.ReLU()(x)
        return x
```

6.10.4 Mẹo thi đấu IOAI

- Luôn dùng BatchNorm với các mạng sâu
- Đặt BatchNorm sau tầng Linear/Conv, trước Activation
- Có thể bỏ Dropout khi dùng BatchNorm
- Learning rate có thể tăng lên 2-5 lần

- Batch size không nên quá nhỏ (ít nhất 32)
- Không cần quan tâm đến bias trong Linear/Conv khi dùng BatchNorm

Câu hỏi kiểm tra:

1. Tại sao không nên dùng BatchNorm với batch size quá nhỏ?
2. Có nên dùng BatchNorm trong layer cuối cùng không?
3. BatchNorm khác gì so với LayerNorm?

6.11 Autoencoders và Sparse Encoders

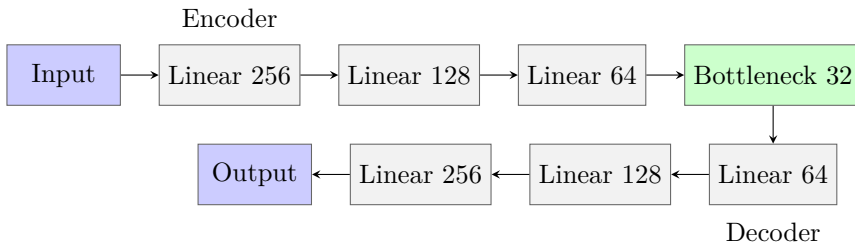


Figure 6.30: Kiến trúc Autoencoder cơ bản bao gồm Encoder, Bottleneck và Decoder.

6.11.1 Autoencoder là gì?

Autoencoder là mạng nơ-ron:

- **Mục tiêu:** Học biểu diễn nén (encoding) của dữ liệu
- **Cấu trúc:**
 - Encoder: Nén input thành latent representation
 - Bottleneck: Lớp ẩn có kích thước nhỏ
 - Decoder: Tái tạo lại input từ latent representation
- **Ứng dụng:** Giảm chiều dữ liệu, khử nhiễu, học biểu diễn đặc trưng

Cài đặt cơ bản trong PyTorch

```
import torch.nn as nn

class Autoencoder(nn.Module):
    def __init__(self, input_dim=784, latent_dim=32):
        super(Autoencoder, self).__init__()

        # Encoder
        self.encoder = nn.Sequential(
            nn.Linear(input_dim, 256),
            nn.ReLU(),
            nn.Linear(256, 128),
            nn.ReLU(),
            nn.Linear(128, latent_dim) # Bottleneck
        )

        # Decoder
        self.decoder = nn.Sequential(
            nn.Linear(latent_dim, 128),
            nn.ReLU(),
            nn.Linear(128, 256),
            nn.ReLU(),
            nn.Linear(256, input_dim),
            nn.Sigmoid() # For image data
        )

    def forward(self, x):
        encoded = self.encoder(x)
        decoded = self.decoder(encoded)
        return decoded
```

6.11.2 Sparse Encoders

Ý tưởng cơ bản

- Buộc autoencoder học biểu diễn thưa thớt

- Chỉ vài neuron được kích hoạt tại bottleneck
- Ưu điểm:
 - Giảm overfitting
 - Học được đặc trưng rõ ràng hơn

Cài đặt với Regularization

```
class SparseAutoencoder(Autoencoder):
    def __init__(self, input_dim=784, latent_dim=32,
                 sparsity_weight=0.01):
        super().__init__(input_dim, latent_dim)
        self.sparsity_weight = sparsity_weight

    def forward(self, x):
        encoded = self.encoder(x)

        # Add sparsity regularization (L1 penalty)
        sparsity_loss = torch.mean(torch.abs(encoded))

        decoded = self.decoder(encoded)
        return decoded, sparsity_loss

    def compute_loss(self, x, x_hat, sparsity_loss):
        recon_loss = nn.MSELoss()(x_hat, x)
        total_loss = recon_loss + self.sparsity_weight *
            sparsity_loss
        return total_loss
```

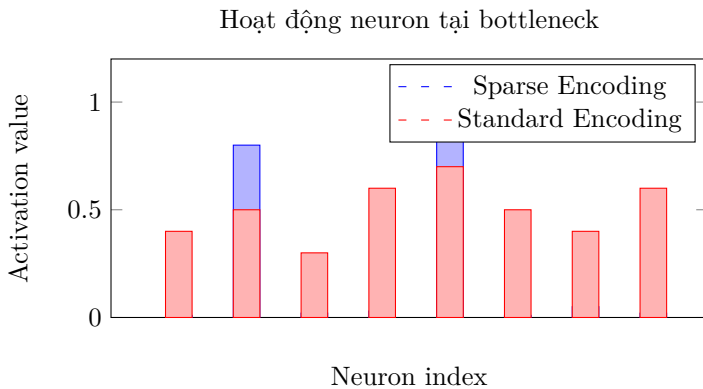


Figure 6.31: Sparse encoder chỉ kích hoạt một số ít neuron tại bottleneck (màu xanh), trong khi autoencoder thông thường kích hoạt đều (màu đỏ).

6.11.3 Ứng dụng thực tiễn

Khử nhiễu hình ảnh

```
class DenoisingAE(Autoencoder):
    def forward(self, x):
        # Add random noise to input
        noise = torch.randn_like(x) * 0.1
        noisy_x = x + noise
        return super().forward(noisy_x)
```

Giảm chiều dữ liệu

```
# Use encoder for dimensionality reduction
model = Autoencoder()
encoder = model.encoder # Use this to extract features

# Get 32-dim features from 784-dim input
features = encoder(input_data)
```

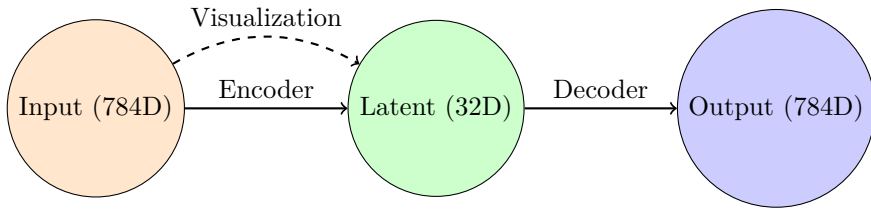


Figure 6.32: Quá trình giảm chiều dữ liệu từ đầu vào 784 chiều (ví dụ hình 28x28) xuống không gian ẩn 32 chiều.

6.11.4 Mẹo thi đấu IOAI

- Autoencoder thường hoạt động tốt khi:
 - Kích thước bottleneck 10-25% input size
 - Dùng kích hoạt ReLU cho encoder, Sigmoid cho decoder (hình ảnh)
- Sparse encoder cần:
 - Điều chỉnh hệ số sparsity (thử 0.001-0.1)
 - Theo dõi hoạt độ neuron tại bottleneck
- So sánh kết quả bằng:
 - Chất lượng tái tạo (MSE/PSNR)
 - Hiệu quả phân cụm sau mã hóa

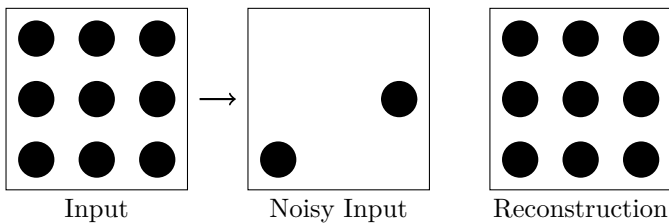


Figure 6.33: Ví dụ ứng dụng khử nhiễu của Denoising Autoencoder. Mô hình học cách loại bỏ nhiễu và tái tạo hình ảnh gốc.

Chapter 7

NLP

7.1 Word Embedding: Word2Vec và GloVe

7.1.1 1. Ý tưởng trực quan

Word Embedding là phương pháp ánh xạ mỗi từ thành một vector số thực trong không gian nhiều chiều, sao cho các từ có ngữ nghĩa tương đồng sẽ có vector gần nhau.

- **Ví dụ trực quan:** Tưởng tượng mỗi từ là một điểm trong không gian 2D/3D, “*king*” và “*queen*” sẽ nằm gần nhau, còn “*king*” và “*apple*” thì khá xa.
- **Tại sao cần?** Giúp máy tính hiểu được quan hệ ngữ nghĩa, phục vụ cho các bài toán như phân loại văn bản, dịch máy, trả lời câu hỏi, v.v.

7.1.2 2. Mô hình Word2Vec

2.1 Skip-gram và CBOW

- **CBOW (Continuous Bag-of-Words):** Dự đoán từ trung tâm dựa trên ngữ cảnh xung quanh.
- **Skip-gram:** Ngược lại, dùng từ trung tâm để dự đoán các từ ngữ cảnh.

2.2 Công thức toán học (Skip-gram)

Cho chuỗi từ $w_1, w_2, \dots, w_T$. Mục tiêu tối đa hóa xác suất:

$$\prod_{t=1}^T \prod_{-c \leq j \leq c, j \neq 0} P(w_{t+j} \mid w_t)$$

với

$$P(w_O \mid w_I) = \frac{\exp(v'_{w_O}{}^\top v_{w_I})}{\sum_{w=1}^{|V|} \exp(v'_w{}^\top v_{w_I})},$$

trong đó v_w là vector input của từ w , v'_w là vector output.

Giải pháp tối ưu: Dùng Negative Sampling để thay thế Softmax toàn phần, giảm độ phức tạp tính toán.

7.1.3 3. Mô hình GloVe

3.1 Ý tưởng

GloVe (Global Vectors) dựa trên ma trận tần suất đồng xuất hiện X , với X_{ij} đếm số lần từ i và j xuất hiện cùng nhau trong một cửa sổ.

3.2 Công thức toán học

Mục tiêu tối thiểu hóa:

$$J = \sum_{i,j=1}^{|V|} f(X_{ij})(w_i^\top \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij})^2,$$

với hàm trọng số

$$f(x) = \begin{cases} (x/x_{\max})^\alpha & \text{nếu } x < x_{\max}, \\ 1 & \text{ngược lại.} \end{cases}$$

Thường chọn $\alpha = 3/4$, $x_{\max} = 100$.

7.1.4 4. Code mẫu

4.1 Word2Vec với Gensim

```
from gensim.models import Word2Vec

# Assume the corpus is already tokenized: list of list of
tokens
sentences = [["toi", "yeu", "machine", "learning"], ["word2vec",
    "rat", "tuyet", "voi"]]

model = Word2Vec(sentences,
                  vector_size=100, # embedding size
                  window=5,       # context window size
                  min_count=1,     # minimum word frequency
                  sg=1,            # 1 = skip-gram, 0 = CBOW
                  negative=5,      # negative sampling
                  epochs=10)

# Get the vector for a word
vec = model.wv["machine"]

# Find most similar words
print(model.wv.most_similar("machine"))
```

4.2 GloVe đơn giản (Python thuần)

```
import numpy as np
from collections import Counter
from itertools import combinations

# 1. Build co-occurrence matrix
def build_cooccurrence(corpus, window=5):
    vocab = set(word for sent in corpus for word in sent)
    word2id = {w:i for i,w in enumerate(vocab)}
    X = np.zeros((len(vocab), len(vocab)))
    for sent in corpus:
        for i, w in enumerate(sent):
```

```
        for j in range(max(0, i-window), min(len(sent), i+
            window+1)):
            if i != j:
                X[word2id[w], word2id[sent[j]]] += 1
    return X, word2id

# 2. Train GloVe
def train_glove(X, dims=50, alpha=0.75, xmax=100, lr=0.05,
    epochs=100):
    V = X.shape[0]
    W = np.random.rand(V, dims)
    W_tilde = np.random.rand(V, dims)
    b = np.zeros(V)
    b_tilde = np.zeros(V)
    f = np.vectorize(lambda x: (x/xmax)**alpha if x < xmax
        else 1)
    X_weight = f(X)
    for _ in range(epochs):
        for i,j in zip(*X.nonzero()):
            xij = X[i,j]
            weight = X_weight[i,j]
            diff = (W[i].dot(W_tilde[j]) + b[i] + b_tilde[j] -
                np.log(xij))
            grad = weight * diff
            W[i] -= lr * grad * W_tilde[j]
            W_tilde[j] -= lr * grad * W[i]
            b[i] -= lr * grad
            b_tilde[j] -= lr * grad
    return W + W_tilde # final word vector

# Example
corpus = [["toi", "yeu", "nlp"], ["nlp", "rat", "hay"]]
X, w2id = build_cooccurrence(corpus)
embeddings = train_glove(X)
```

7.1.5 5. Bài tập thực hành

1. Huấn luyện Word2Vec:

- Sử dụng một tập dữ liệu tiếng Việt (VD: kho báo VietNews) để huấn luyện mô hình Word2Vec với tham số khác nhau (window size, vector size, negative sampling).
- So sánh kết quả: tìm top-5 từ tương đồng với “*học*”, “*máy*”, “*ngôn*”.

2. Phân tích ma trận đồng xuất hiện:

- Xây ma trận đồng xuất hiện cho một tập văn bản nhỏ (tự định nghĩa).
- Quan sát phân bố giá trị trong ma trận, vẽ heatmap (dùng matplotlib).

3. So sánh Word2Vec và GloVe:

- Huấn luyện cả hai mô hình trên cùng tập dữ liệu.
- Đánh giá chất lượng embedding bằng intrinsic evaluation (word analogy, word similarity).

7.2 Cơ bản về Transformer và Attention Mechanism

7.2.1 1. Ý tưởng trực quan

Transformer loại bỏ hoàn toàn RNN/CNN, thay vào đó sử dụng cơ chế attention để cho phép mỗi vị trí trong chuỗi đầu vào tập trung (focus) lên những vị trí quan trọng khác khi xây dựng biểu diễn ngữ cảnh của nó.

- **Trực quan:** Mỗi token tự hỏi “Tôi nên chú ý bao nhiêu đến token kia?” và gán trọng số tương ứng, giống như bạn lắng nghe nhiều hơn những từ quan trọng trong một câu :contentReference[oaicite:0]index=0.
- **Tại sao mạnh mẽ?** Cơ chế attention cho phép nắm bắt mối quan hệ dài hạn mà không gặp phải hạn chế về thứ tự tính toán tuần tự như RNN :contentReference[oaicite:1]index=1.

7.2.2 2. Scaled Dot-Product Attention

Cho ma trận $\mathbf{Q} \in \mathbb{R}^{n \times d_k}$ (queries), $\mathbf{K} \in \mathbb{R}^{m \times d_k}$ (keys), $\mathbf{V} \in \mathbb{R}^{m \times d_v}$ (values), ta tính:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}.$$

- Thuộc số $\sqrt{d_k}$ để tránh giá trị dot-product quá lớn gây bão hoà Softmax.
- Kết quả là ma trận trọng số $\alpha \in \mathbb{R}^{n \times m}$, sau đó nhân với $\mathbf{V}$ để cho ra biểu diễn ngữ cảnh.

7.2.3 3. Multi-Head Attention

Thay vì chỉ dùng một cặp (Q, K, V) , Transformer dùng h heads song song:

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \mathbf{W}^O,$$

với

$$\text{head}_i = \text{Attention}(\mathbf{Q}\mathbf{W}_i^Q, \mathbf{K}\mathbf{W}_i^K, \mathbf{V}\mathbf{W}_i^V),$$

và $\mathbf{W}_i^Q, \mathbf{W}_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $\mathbf{W}_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$.

7.2.4 4. Code mẫu với PyTorch

```
import torch
import torch.nn.functional as F

def scaled_dot_product_attention(Q, K, V, mask=None):
    d_k = Q.size(-1)
    scores = torch.matmul(Q, K.transpose(-2, -1)) / torch.sqrt(
        (torch.tensor(d_k, dtype=torch.float)))
    if mask is not None:
        scores = scores.masked_fill(mask == 0, float('-inf'))
    weights = F.softmax(scores, dim=-1)
    return torch.matmul(weights, V), weights

# Ví dụ
```

```
batch_size, seq_len, d_model = 2, 5, 16
Q = torch.rand(batch_size, seq_len, d_model)
K = torch.rand(batch_size, seq_len, d_model)
V = torch.rand(batch_size, seq_len, d_model)
output, attn_weights = scaled_dot_product_attention(Q, K, V)
print(output.shape)  # (2,5,16)

import torch.nn as nn

class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, num_heads):
        super().__init__()
        assert d_model % num_heads == 0
        self.d_k = d_model // num_heads
        self.num_heads = num_heads
        self.Wq = nn.Linear(d_model, d_model)
        self.Wk = nn.Linear(d_model, d_model)
        self.Wv = nn.Linear(d_model, d_model)
        self.Wo = nn.Linear(d_model, d_model)

    def forward(self, x, mask=None):
        B, N, _ = x.size()
        Q = self.Wq(x).view(B, N, self.num_heads, self.d_k).
            transpose(1,2)
        K = self.Wk(x).view(B, N, self.num_heads, self.d_k).
            transpose(1,2)
        V = self.Wv(x).view(B, N, self.num_heads, self.d_k).
            transpose(1,2)
        # attention
        scores = torch.matmul(Q, K.transpose(-2,-1)) / torch.
            sqrt(torch.tensor(self.d_k, dtype=torch.float))
        if mask is not None:
            scores = scores.masked_fill(mask == 0, float('-inf'
                ))
        weights = F.softmax(scores, dim=-1)
        out = torch.matmul(weights, V)
        # ết qhp các head
        out = out.transpose(1,2).contiguous().view(B, N, -1)
        return self.Wo(out)
```

7.2.5 5. Bài tập thực hành

1. Visual hóa Attention:

- Dùng ví dụ câu tiếng Anh, tính attention weights và vẽ heatmap (matplotlib) để quan sát mô hình “chú ý” đến từ nào.

2. So sánh Masked vs. Unmasked Attention:

- Thêm mask causal vào decoder, so sánh kết quả attention weights giữa có mask và không mask.

3. Xây dựng Transformer nhỏ:

- Kết hợp Encoder layer (Multi-Head + Feed-Forward) và Decoder layer, huấn luyện giải bài toán dịch đơn giản (ví dụ: tiếng Anh $\rightarrow$ tiếng Pháp nhỏ).

7.3 Thực hành Phân loại Văn bản

7.3.1 1. Quy trình chung

1. **Thu thập dữ liệu:** Các tập văn bản có gán nhãn (spam/ham, thể loại tin tức, cảm xúc...).
2. **Tiền xử lý:** Loại bỏ ký tự đặc biệt, chuyển thường, tokenization, stop-words, stemming/lemmatization.

3. Trích đặc trưng:

- Bag-of-Words (BoW): Đếm tần suất từ.
- TF-IDF:

$$\text{tfidf}(t, d) = \text{tf}(t, d) \times \log \frac{N}{\text{df}(t)}.$$

- Word Embedding: Word2Vec/GloVe/Transformer.

4. **Xây dựng mô hình:** Logistic Regression, SVM, Naive Bayes, CNN/RNN/Transformer.
5. **Đánh giá:** Accuracy, Precision, Recall, F1-score, ROC-AUC.

7.3.2 2. Phương pháp cơ bản

2.1 Bag-of-Words + Logistic Regression

Cho ma trận đặc trưng $X \in \mathbb{R}^{N \times V}$ (N mẫu, V từ vựng), nhãn $y \in \{0, 1\}^N$.
Hàm mất mát logistic:

$$L(\mathbf{w}, b) = -\frac{1}{N} \sum_{i=1}^N [y_i \log \sigma(\mathbf{w}^\top x_i + b) + (1 - y_i) \log(1 - \sigma(\mathbf{w}^\top x_i + b))],$$

với $\sigma(z) = 1/(1 + e^{-z})$.

2.2 TF-IDF + SVM

SVM tối ưu:

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^N \max(0, 1 - y_i(\mathbf{w}^\top x_i + b)).$$

2.3 Embedding + CNN/RNN

- **CNN cho text**: Dùng các filter kích thước khác nhau quét chuỗi embedding, áp dụng max-pooling.
- **RNN (LSTM/GRU)**: Duyệt tuần tự, lấy trạng thái cuối để phân loại.

7.3.3 3. Code mẫu

3.1 Scikit-learn: BoW + Logistic Regression

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
```

```
# Example data
```

```
docs = ["I love machine learning", "Spam: Buy now!", "Today's  
news..."]  
labels = [0, 1, 0] # 0 = ham, 1 = spam  
  
X_train, X_test, y_train, y_test = train_test_split(docs,  
    labels, test_size=0.3)  
  
model = Pipeline([  
    ('bow', CountVectorizer()),  
    ('clf', LogisticRegression(max_iter=1000))  
])  
model.fit(X_train, y_train)  
y_pred = model.predict(X_test)  
print(classification_report(y_test, y_pred))
```

3.2 PyTorch: Embedding + CNN

```
import torch  
import torch.nn as nn  
import torch.nn.functional as F  
  
class TextCNN(nn.Module):  
    def __init__(self, vocab_size, embed_dim, num_classes,  
        kernel_sizes=[3,4,5], num_filters=100):  
        super().__init__()  
        self.embedding = nn.Embedding(vocab_size, embed_dim)  
        self.convs = nn.ModuleList([  
            nn.Conv2d(1, num_filters, (k, embed_dim)) for k in  
                kernel_sizes  
        ])  
        self.fc = nn.Linear(num_filters * len(kernel_sizes),  
            num_classes)  
  
    def forward(self, x):  
        # x: [B, L]  
        x = self.embedding(x).unsqueeze(1) # [B, 1, L, E]  
        convded = [F.relu(conv(x)).squeeze(3) for conv in self.  
            convs] # each: [B, num_filters, L-k+1]
```



```
pooled = [F.max_pool1d(c, c.size(2)).squeeze(2) for c
            in convded] # [B, num_filters]
cat = torch.cat(pooled, dim=1) # [B, num_filters *
                             len(kernel_sizes)]
return self.fc(cat)

# Assume there is a DataLoader returning (inputs, labels)
# model = TextCNN(vocab_size=5000, embed_dim=100, num_classes
#                 =2)
# loss_fn = nn.CrossEntropyLoss(); optimizer = torch.optim.
#           Adam(model.parameters())
```

7.3.4 4. Bài tập thực hành

1. Spam Detection:

- Dùng tập dữ liệu SpamAssassin để huấn luyện Naive Bayes và Logistic Regression.
- So sánh Precision, Recall, F1-score.

2. Phân loại cảm xúc:

- Sử dụng tập IMDb movie reviews. Huấn luyện TextCNN và BiLSTM, so sánh accuracy.
- Vẽ learning curve (train vs. val loss).

3. Thử nghiệm TF-IDF + SVM vs. Embedding + RNN:

- Trên cùng tập dữ liệu, so sánh tốc độ huấn luyện, hiệu năng, và kích thước mô hình.

7.4 Giới thiệu các Mô hình NLP Đã được Pre-train: BERT và GPT

7.4.1 1. Tại sao cần Pre-trained Models?

Pre-trained models (mô hình đã được huấn luyện trước) cho phép tái sử dụng các biểu diễn ngôn ngữ học được học từ khối lượng dữ liệu khổng lồ, giúp tiết kiệm thời gian huấn luyện và cải thiện hiệu năng trên nhiều tác vụ khác nhau. Chúng thường bao gồm hai giai đoạn chính: pre-training trên dữ liệu không gắn nhãn với các nhiệm vụ tự giám sát, sau đó fine-tuning trên dữ liệu gắn nhãn cho từng tác vụ cụ thể.

7.4.2 2. BERT (Bidirectional Encoder Representations from Transformers)

2.1 Ý tưởng chính

BERT là mô hình chỉ gồm phần encoder của Transformer, được huấn luyện theo hai nhiệm vụ tự giám sát: Masked Language Model (MLM) và Next Sentence Prediction (NSP). Qua đó, BERT học được biểu diễn ngữ cảnh hai chiều (bidirectional), nghĩa là mỗi token trong câu “nhìn” được cả ngữ cảnh bên trái lẫn bên phải khi tính embedding :contentReference[oaicite:0]index=0.

2.2 Công thức cơ bản

- **Masked Language Model (MLM):** Với xác suất p , một số token đầu vào được thay bằng [MASK]. Mục tiêu là tối đa hóa

$$\sum_{i \in M} \log P(w_i \mid \mathbf{X}_{\setminus M}),$$

trong đó M là tập chỉ số token bị mask, $\mathbf{X}_{\setminus M}$ là chuỗi đầu vào đã mask.

- **Next Sentence Prediction (NSP):** Cho cặp câu (A, B) , mô hình dự

đoán nhãn “IsNext” hay “NotNext”:

$$\max \sum \log P(\text{IsNext} \mid A, B).$$

2.3 Ứng dụng và Fine-tuning

Sau pre-training, ta chỉ cần thêm một lớp phân loại (linear + softmax) lên embedding của token [CLS] để fine-tune cho các tác vụ như phân loại văn bản, hỏi-đáp, NER... :contentReference[oaicite:1]index=1.

7.4.3 3. GPT (Generative Pre-trained Transformer)

3.1 Ý tưởng chính

GPT là mô hình chỉ gồm phần decoder của Transformer, được huấn luyện theo nhiệm vụ Language Modeling (LM) tuần tự: tại mỗi bước, dự đoán token tiếp theo dựa trên toàn bộ ngữ cảnh trước đó. Mô hình sử dụng cơ chế causal (masked) self-attention để đảm bảo tính tuần tự.

3.2 Công thức cơ bản

Cho chuỗi đầu vào $w_1, w_2, \dots, w_T$, GPT tối đa hóa xác suất:

$$\prod_{t=1}^T P(w_t \mid w_{<t}) = \prod_{t=1}^T \frac{\exp(v_{w_t}^\top h_{t-1})}{\sum_{w' \in V} \exp(v_{w'}^\top h_{t-1})},$$

trong đó h_{t-1} là trạng thái ẩn được tính qua các lớp self-attention và feed-forward, và causal mask ngăn không cho token “nhìn” về phía tương lai.

3.3 Ứng dụng

GPT có khả năng sinh văn bản, trả lời câu hỏi, tóm tắt, dịch máy, v.v. Khi fine-tuning, ta thêm một đầu ra phù hợp (linear + softmax hoặc head tùy tác vụ) và tiếp tục huấn luyện trên dữ liệu gán nhãn nhỏ hơn.

7.4.4 4. Code mẫu

4.1 Fine-tuning BERT với Hugging Face Transformers

```
from transformers import BertForSequenceClassification,
    BertTokenizerFast, Trainer, TrainingArguments
import torch

tokenizer = BertTokenizerFast.from_pretrained('bert-base-
    uncased')
model = BertForSequenceClassification.from_pretrained('bert-
    base-uncased', num_labels=2)

# Example data
texts = ["I love NLP", "This is spam"]
labels = [0, 1]
encodings = tokenizer(texts, truncation=True, padding=True)
dataset = torch.utils.data.Dataset.from_dict({
    'input_ids': encodings['input_ids'],
    'attention_mask': encodings['attention_mask'],
    'labels': labels
})

args = TrainingArguments(output_dir='./bert-finetuned',
    num_train_epochs=3,
    per_device_train_batch_size=8)
trainer = Trainer(model=model, args=args, train_dataset=
    dataset)
trainer.train()
```

4.2 Sinh văn bản với GPT-2

```
from transformers import GPT2LMHeadModel, GPT2Tokenizer

tokenizer = GPT2Tokenizer.from_pretrained('gpt2')
model = GPT2LMHeadModel.from_pretrained('gpt2')

prompt = "Once upon a time"
```

```
inputs = tokenizer(prompt, return_tensors='pt')
outputs = model.generate(**inputs, max_length=50, do_sample=
    True, top_k=50)
print(tokenizer.decode(outputs[0], skip_special_tokens=True))
```

7.4.5 5. Bài tập thực hành

1. Fine-tune BERT cho cảm xúc:

- Sử dụng tập IMDb reviews, fine-tune BERT để phân loại cảm xúc.
- So sánh accuracy với Logistic Regression trên TF-IDF.

2. Sinh văn bản với GPT:

- Thử sinh truyện ngắn (prompt dài 100 token) với GPT-2 và GPT-3 (nếu có API).
- Đánh giá chất lượng bằng BLEU hoặc human evaluation.

3. So sánh BERT vs. GPT cho Q&A:

- Fine-tune BERT (QA head) và GPT (prefix-LM) trên SQuAD nhỏ.
- Đánh giá F1 và Exact Match.

7.5 Thực hành Question Answering với Mô hình Pre-trained

7.5.1 1. Ý tưởng trực quan

Question Answering (QA) là bài toán: cho trước một đoạn văn bản (context) và một câu hỏi (question), hệ thống cần trích xuất (extractive) hoặc sinh (generative) câu trả lời.

- **Extractive QA:** Mô hình tìm vị trí bắt đầu và kết thúc của câu trả lời trong đoạn context.
- **Generative QA:** Mô hình sinh câu trả lời tự do dựa trên context và question.

Các mô hình Pre-trained như BERT đã học biểu diễn ngữ cảnh sâu từ khối lượng lớn Wikipedia và sách, nhờ đó chỉ cần fine-tune nhẹ trên tập QA như SQuAD để đạt hiệu năng cao :contentReference[oaicite:0]index=0.

7.5.2 2. Công thức toán học nền tảng

Với Extractive QA, ta thêm hai lớp linear trên embedding cuối của token để dự đoán xác suất start và end:

$$P_{\text{start}}(i \mid \mathbf{X}) = \frac{\exp(\mathbf{w}_s^\top \mathbf{h}_i)}{\sum_{j=1}^n \exp(\mathbf{w}_s^\top \mathbf{h}_j)}, \quad P_{\text{end}}(i \mid \mathbf{X}) = \frac{\exp(\mathbf{w}_e^\top \mathbf{h}_i)}{\sum_{j=1}^n \exp(\mathbf{w}_e^\top \mathbf{h}_j)},$$

trong đó $\mathbf{h}_i$ là vector embedding của token thứ i , $\mathbf{w}_s, \mathbf{w}_e$ là tham số học được. Hàm mất mát là tổng cross-entropy cho cả hai phân phối:

$$\mathcal{L} = -[\log P_{\text{start}}(y_s) + \log P_{\text{end}}(y_e)],$$

với y_s, y_e là vị trí ground-truth của start và end :contentReference[oaicite:1]index=1.

7.5.3 3. Fine-tuning BERT cho Extractive QA

1. **Chuẩn bị dữ liệu:** SQuAD 1.1 (question, context, answer_start, answer_text).
2. **Tokenization:** Dùng tokenizer của BERT, nối [CLS] + question + [SEP] + context + [SEP].
3. **Tạo labels:** Với mỗi token, xác định index start/end của câu trả lời sau khi tokenize.
4. **Huấn luyện:** Dùng mô hình BertForQuestionAnswering và tối ưu cross-entropy như trên.

7.5.4 4. Code mẫu với Hugging Face & PyTorch

```
from transformers import BertTokenizerFast,
    BertForQuestionAnswering, Trainer, TrainingArguments
from datasets import load_dataset, load_metric
```

```
# 1. Load SQuAD
datasets = load_dataset("squad")
metric = load_metric("squad")

# 2. Tokenizer and normalization
tokenizer = BertTokenizerFast.from_pretrained("bert-base-uncased")

def preprocess(examples):
    questions = examples["question"]
    contexts = examples["context"]
    answers = examples["answers"]
    inputs = tokenizer(questions, contexts, truncation=True,
                      padding="max_length", max_length=384)
    start_positions = []
    end_positions = []
    for i, ans in enumerate(answers):
        start = ans["answer_start"][0]
        end = start + len(ans["text"][0])
        encoded = inputs.char_to_token(i, start), inputs.
            char_to_token(i, end-1)
        start_positions.append(encoded[0])
        end_positions.append(encoded[1])
    inputs["start_positions"] = start_positions
    inputs["end_positions"] = end_positions
    return inputs

tokenized = datasets.map(preprocess, batched=True,
                        remove_columns=datasets["train"].column_names)

# 3. Model & Training
model = BertForQuestionAnswering.from_pretrained("bert-base-uncased")
args = TrainingArguments(
    output_dir="bert-qa",
    evaluation_strategy="epoch",
    per_device_train_batch_size=8,
```

```
        num_train_epochs=2,
    )

    trainer = Trainer(
        model=model,
        args=args,
        train_dataset=tokenized["train"],
        eval_dataset= tokenized["validation"],
    )

    trainer.train()
```

7.5.5 5. Bài tập thực hành

1. Đánh giá trên SQuAD:

- Sau khi fine-tune, dùng `Trainer.predict` để sinh câu trả lời trên tập validation, tính EM và F1 với `load_metric("squad") :contentReference[oaicite:2]index=2`.

2. Custom Dataset QA:

- Tạo tập QA nhỏ (tự viết context và question), fine-tune mô hình, đánh giá kết quả.

3. So sánh với GPT-based QA:

- Sử dụng GPT-3/GPT-4 API để trả lời cùng câu hỏi trên context, so sánh chất lượng (human eval hoặc BLEU).

4. Visual hóa Attention:

- Trích attention weights từ layer đầu tiên của BERT khi sinh token answer, vẽ heatmap (matplotlib) để quan sát mô hình “chú ý” vào đâu khi trả lời.

7.6 Fine-Tuning Mô hình: Phương pháp và Giới hạn (LoRA, Adapters, v.v.)

7.6.1 1. Giới thiệu

Fine-tuning là quá trình điều chỉnh các mô hình ngôn ngữ lớn (LLMs) đã được huấn luyện trước để phù hợp với các tác vụ cụ thể. Tuy nhiên, việc fine-tune toàn bộ mô hình có thể tốn kém về mặt tính toán và lưu trữ. Do đó, các phương pháp fine-tuning hiệu quả về tham số (PEFT) như LoRA và Adapters đã được phát triển để giảm số lượng tham số cần điều chỉnh, tiết kiệm tài nguyên và thời gian huấn luyện.

7.6.2 2. LoRA: Low-Rank Adaptation

2.1 Ý tưởng chính

LoRA (Low-Rank Adaptation) là một phương pháp PEFT trong đó các trọng số của mô hình gốc được giữ nguyên (đóng băng), và chỉ thêm vào các ma trận có hạng thấp để điều chỉnh mô hình cho tác vụ mới. Điều này giúp giảm đáng kể số lượng tham số cần huấn luyện và tiết kiệm bộ nhớ.

2.2 Công thức toán học

Giả sử $W \in \mathbb{R}^{d \times k}$ là ma trận trọng số trong mô hình gốc. LoRA thêm vào một nhiễu hạng thấp $\Delta W = AB$, trong đó:

$$A \in \mathbb{R}^{d \times r}, \quad B \in \mathbb{R}^{r \times k}, \quad r \ll \min(d, k)$$

Trọng số được sử dụng trong mô hình là:

$$W' = W + \Delta W = W + AB$$

Chỉ A và B được huấn luyện, trong khi W được giữ nguyên.

7.6.3 3. Adapters

3.1 Ý tưởng chính

Adapters là các mô-đun nhỏ được chèn vào giữa các lớp của mô hình Transformer. Thay vì điều chỉnh toàn bộ mô hình, chỉ các tham số trong các mô-đun adapter được huấn luyện. Điều này giúp giảm số lượng tham số cần điều chỉnh và cho phép chia sẻ mô hình gốc giữa nhiều tác vụ :contentReference[oaicite:9]index=9.

3.2 Cấu trúc Adapter

Một mô-đun adapter thường bao gồm:

- Một lớp giảm chiều (down-projection): $h \in \mathbb{R}^d \rightarrow h' \in \mathbb{R}^r$
- Một hàm kích hoạt phi tuyến tính (ví dụ: ReLU)
- Một lớp tăng chiều (up-projection): $h' \in \mathbb{R}^r \rightarrow \hat{h} \in \mathbb{R}^d$

Đầu ra của adapter được cộng với đầu vào ban đầu (residual connection):

$$\text{Output} = h + \hat{h}$$

7.6.4 4. Code mẫu với Hugging Face

4.1 Fine-Tuning với LoRA

```
from transformers import AutoModelForSequenceClassification,
    AutoTokenizer
from peft import get_peft_model, LoraConfig, TaskType

model = AutoModelForSequenceClassification.from_pretrained("
    bert-base-uncased")
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")

lora_config = LoraConfig(
    task_type=TaskType.SEQ_CLS,
    r=8,
    lora_alpha=32,
```

```
lora_dropout=0.1,  
bias="none"  
)  
  
model = get_peft_model(model, lora_config)  
# Continue with training as usual
```

4.2 Sử dụng Adapters

```
from transformers import AutoModelWithHeads, AutoTokenizer  
  
model = AutoModelWithHeads.from_pretrained("bert-base-uncased"  
)  
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")  
  
model.add_adapter("classification_adapter")  
model.train_adapter("classification_adapter")  
# Continue with training as usual
```

7.6.5 5. Bài tập thực hành

1. **So sánh hiệu suất:** Fine-tune một mô hình BERT trên tập dữ liệu phân loại văn bản nhỏ bằng cả LoRA và Adapters. So sánh độ chính xác và thời gian huấn luyện.
2. **Kết hợp nhiều adapters:** Huấn luyện các adapters riêng biệt cho các tác vụ khác nhau (ví dụ: phân loại cảm xúc và phân loại chủ đề) và thử kết hợp chúng để cải thiện hiệu suất.
3. **Thử nghiệm với các giá trị r :** Trong LoRA, thay đổi giá trị r (hạng của ma trận) và quan sát ảnh hưởng đến hiệu suất và tài nguyên sử dụng.

7.7 Giới thiệu về Large Language Models (LLMs)

Large Language Models (LLMs) là các mô hình học sâu (deep learning models) có kích thước lớn (nhiều tham số) và được huấn luyện trên lượng dữ liệu văn bản khổng lồ. Một ví dụ nổi bật là GPT-4 của OpenAI.

7.7.1 Ý tưởng chính đằng sau LLMs

Mục tiêu là mô hình có thể học được cách dự đoán từ tiếp theo trong một câu dựa trên các từ trước đó:

$$P(w_t | w_1, w_2, \dots, w_{t-1}) \quad (7.1)$$

Tức là xác suất từ tiếp theo w_t dựa trên ngữ cảnh trước đó. Dựa vào cơ chế attention và transformer, mô hình có thể xử lý cả đoạn văn dài.

7.7.2 Transformer và Attention

Transformer là kiến trúc cơ bản của các LLM hiện đại. Cơ chế attention giúp mô hình "tập trung" vào những phần quan trọng của câu. Self-attention tính toán độ liên quan giữa các từ:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \quad (7.2)$$

Trong đó:

- Q : Query
- K : Key
- V : Value
- d_k : số chiều của vector key

7.8 Xây dựng chatbot đơn giản sử dụng NLP

Chatbot là một chương trình máy tính có thể giao tiếp với con người qua văn bản hoặc giọng nói. Nhờ sự phát triển của Natural Language Processing (NLP), việc xây dựng chatbot trở nên dễ dàng và hiệu quả hơn bao giờ hết.

Trong phần này, chúng ta sẽ tìm hiểu cách xây dựng một chatbot đơn giản sử dụng các kỹ thuật NLP cơ bản, phù hợp cho người học mới bắt đầu.

7.8.1 Mô hình hoạt động cơ bản của chatbot

Một chatbot cơ bản thường hoạt động theo các bước sau:

1. Nhận đầu vào từ người dùng (câu hỏi, yêu cầu)
2. Xử lý ngôn ngữ tự nhiên (NLP) để hiểu ý định (intent)
3. Tìm câu trả lời phù hợp hoặc thực hiện hành động tương ứng
4. Trả lời người dùng

7.8.2 Tiền xử lý văn bản (Text Preprocessing)

Để chatbot có thể hiểu đầu vào, bước đầu tiên là làm sạch và chuẩn hóa văn bản:

- **Lowercasing** — Chuyển tất cả thành chữ thường
- **Tokenization** — Chia câu thành các từ hoặc đơn vị nhỏ hơn
- **Stopword Removal** — Loại bỏ các từ không mang nhiều ý nghĩa (như "là", "và", ...)

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords

sentence = "I want to go to Hanoi. I heard that there was a
            nuke there yesterday."

# To lower
```

```
sentence = sentence.lower()

# Tokenization
words = word_tokenize(sentence)

# Remove stopwords
stop_words = set(stopwords.words('vietnamese'))
filtered = [w for w in words if w not in stop_words]
print(filtered)
```

7.8.3 Nhận diện ý định (Intent Detection)

Mục tiêu là xác định người dùng muốn làm gì. Ví dụ:

- "Tôi muốn đặt vé" $\Rightarrow$ `book_ticket`
- "Thời tiết hôm nay ra sao?" $\Rightarrow$ `check_weather`

Có thể dùng mô hình phân loại đơn giản như Logistic Regression hoặc mô hình nâng cao như BERT.

Mô hình phân loại cơ bản (Softmax):

$$P(y = k|\vec{x}) = \frac{e^{\vec{w}_k^T \vec{x}}}{\sum_{j=1}^K e^{\vec{w}_j^T \vec{x}}} \quad (7.3)$$

- $\vec{x}$: vector biểu diễn câu
- $\vec{w}_k$: trọng số ứng với lớp k
- K : số lượng lớp

7.8.4 Sinh câu trả lời (Response Generation)

Có hai cách chính:

- **Rule-based**: Mỗi intent được gán với tập câu trả lời có sẵn
- **Generative**: Dùng mô hình để sinh phản hồi, ví dụ GPT

```
def get_response(intent):
    responses = {
        "book_ticket": "Where do you want to go?",
        "check_weather": "Where do you want to check the
            weather?",
        "greeting": "Hello! How can I help you?"
    }
    return responses.get(intent, "Sorry, I don't understand
        what you are saying.")
```

7.8.5 Tích hợp thành chatbot hoàn chỉnh

```
while True:
    user_input = input("You: ")
    if user_input.lower() in ["exit"]:
        print("Goodbye!")
        break

    # Given that the function detect_intent is readily trained
    intent = detect_intent(user_input)
    response = get_response(intent)
    print("Bot:", response)
```

7.8.6 Mở rộng: Dùng mô hình GPT

Dưới đây là ví dụ sử dụng OpenAI API để tạo phản hồi động:

```
import openai
openai.api_key = "YOUR_API_KEY"

def ask_gpt(prompt):
    response = openai.ChatCompletion.create(
        model="gpt-3.5-turbo",
        messages=[{"role": "user", "content": prompt}]
    )
    return response["choices"][0]["message"]["content"]
```

```
while True:
    user_input = input("You: ")
    if user_input.lower() in ["exit"]:
        break
    print("Bot:", ask_gpt(user_input))
```

7.8.7 Tổng kết

Chatbot là ứng dụng thiết thực của NLP, có thể xây dựng từ rất đơn giản đến rất thông minh. Bằng cách hiểu rõ từng bước từ tiền xử lý đến sinh phản hồi, bạn có thể phát triển chatbot phù hợp với mục tiêu học tập hoặc sản phẩm.

7.9 LLM Agents: Cơ bản và Ứng dụng

LLM Agents là hệ thống sử dụng mô hình ngôn ngữ lớn như một thành phần để thực hiện các chuỗi hành động, như gọi API, truy vấn dữ liệu, hoặc giải quyết bài toán.

7.9.1 Ý tưởng chính

Thay vì chỉ trả lời văn bản, LLM có thể "lên kế hoạch" và gọi các công cụ hỗ trợ. Một Agent có thể gồm các phần:

- Planner: lập kế hoạch hành động
- Executor: thực thi từng bước
- Memory: lưu lại thông tin giữa các lượt tương tác

7.9.2 Ứng dụng ví dụ

- Trợ lý lập trình (Code Assistant)
- Tự động xử lý email
- Tương tác với cơ sở dữ liệu qua ngôn ngữ tự nhiên

7.9.3 Ví dụ mã Python: Agent đơn giản với LangChain

```
from langchain.agents import load_tools, initialize_agent
from langchain.llms import OpenAI

llm = OpenAI(temperature=0)
tools = load_tools(["serpapi"])

agent = initialize_agent(tools, llm, agent="zero-shot-react-
description")
response = agent.run("Find the capital of Vietnam")
print(response)
```

7.10 Kết luận

NLP là lĩnh vực cực kỳ thú vị và đang phát triển nhanh chóng. Các mô hình ngôn ngữ lớn như GPT-4 mở ra nhiều khả năng mới. Việc hiểu rõ cách hoạt động và giới hạn của các mô hình này giúp chúng ta sử dụng chúng một cách hiệu quả và có trách nhiệm.

Chapter 8

Computer Vision

8.1 Fundamentals of Convolutional Layers

1. Convolutional Layer là gì?

Convolutional Layer (Lớp tích chập) là thành phần cốt lõi trong mạng nơ-ron tích chập (CNN - Convolutional Neural Network). Nó giúp máy tính tự động học các đặc trưng hình ảnh như cạnh, góc, hoạ tiết... bằng cách sử dụng một tập các bộ lọc (filter/kernel).

Hoạt động của Convolution

- Mỗi filter là một ma trận nhỏ (ví dụ: 3×3).
- Filter sẽ quét qua toàn bộ ảnh và tính toán tích chập giữa filter và từng phần nhỏ của ảnh.
- Kết quả là một **feature map** thể hiện đặc trưng được học.

2. Tại sao cần Convolutional Layer?

Vì ảnh có rất nhiều pixel, ta không thể xử lý hiệu quả nếu dùng mạng nơ-ron truyền thống. Convolution giúp:

- Giảm số lượng tham số

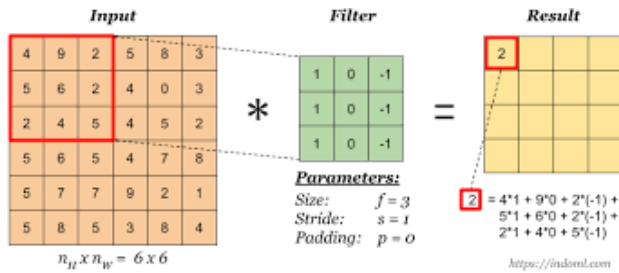


Figure 8.1: Hoạt động tích chập giữa ảnh và filter

- Tập trung vào các đặc trưng quan trọng
- Duy trì cấu trúc không gian của ảnh

3. Cách sử dụng Convolution trong PyTorch

Listing 8.1: Ví dụ đơn giản dùng Convolutional Layer trong PyTorch

```
import torch
import torch.nn as nn

# 3 Khi tạo 1 lớp CNN với 1 Convolutional Layer
class SimpleCNN(nn.Module):
    def __init__(self):
        super(SimpleCNN, self).__init__()
        self.conv1 = nn.Conv2d(in_channels=1, out_channels=1,
                                kernel_size=3, stride=1, padding=0)

    def forward(self, x):
        return self.conv1(x)

# 4 Tạo ảnh đầu vào ngẫu nhiên (1 ảnh xám 5x5)
input_image = torch.randn(1, 1, 5, 5) # batch=1, channel=1,
model = SimpleCNN()
output = model(input_image)
print(output)
```

4. Bài tập thực hành

1. Viết một mô hình CNN gồm 2 lớp tích chập và 1 lớp fully connected.
2. Thay đổi kích thước kernel từ 3×3 sang 5×5 . Quan sát sự khác biệt.
3. Dùng tập dữ liệu MNIST và thử phân loại chữ số bằng CNN.

5. Ghi nhớ nhanh

- Filter là "mắt thần" giúp mạng học cách phát hiện đặc trưng.
- Kích thước filter nhỏ sẽ học được chi tiết nhỏ (ví dụ cạnh), còn filter lớn học được hoạ tiết rộng hơn.
- Convolution giữ được tính cục bộ và không gian trong ảnh.

8.2 Pooling Techniques (Max, Average)

1. Pooling là gì?

Pooling là một kỹ thuật được sử dụng trong mạng nơ-ron tích chập (Convolutional Neural Networks - CNN) để giảm kích thước của ảnh đặc trưng (feature map). Mục tiêu chính là:

- Giảm số lượng tham số và tính toán trong mô hình.
- Kiểm soát quá khớp (overfitting).
- Giữ lại thông tin quan trọng trong khi bỏ qua các chi tiết không cần thiết.

2. Các loại Pooling phổ biến

Có hai kỹ thuật phổ biến nhất:

1. **Max Pooling**: Lấy giá trị lớn nhất trong vùng lọc (ví dụ: 2x2).
2. **Average Pooling**: Tính giá trị trung bình trong vùng lọc.

3. Cách hoạt động

Giả sử ta có ma trận đặc trưng đầu vào là:

$$\begin{bmatrix} 1 & 3 & 2 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 8 & 7 & 6 \\ 4 & 3 & 2 & 1 \end{bmatrix}$$

Max Pooling (với kích thước 2x2, bước nhảy 2)

Chia ma trận thành các vùng 2x2 không chồng lên nhau:

$$\begin{bmatrix} 1 & 3 & 2 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 8 & 7 & 6 \\ 4 & 3 & 2 & 1 \end{bmatrix} \Rightarrow \max = 6$$

Tương tự với các vùng còn lại, ta được đầu ra:

$$\begin{bmatrix} 6 & 8 \\ 9 & 7 \end{bmatrix}$$

Average Pooling (với kích thước 2x2, bước nhảy 2)

Tính trung bình từng vùng:

$$\begin{bmatrix} \frac{1+3+5+6}{4} & \frac{2+4+7+8}{4} \\ \frac{9+8+4+3}{4} & \frac{7+6+2+1}{4} \end{bmatrix} = \begin{bmatrix} 3.75 & 5.25 \\ 6 & 4 \end{bmatrix}$$

4. Khi nào dùng Max và Average Pooling?

- **Max Pooling** thường được sử dụng khi bạn muốn mô hình tập trung vào các đặc trưng nổi bật nhất (edges, textures).
- **Average Pooling** phù hợp khi bạn muốn lấy thông tin tổng quát và mượt mà hơn từ ảnh.

5. Cách dùng trong PyTorch (Ví dụ đơn giản)

Listing 8.2: Ví dụ Max và Avg Pooling trong PyTorch

```
import torch
import torch.nn as nn

# Giả sử input kích thước (batch, channel, height, width)
input = torch.tensor([[[[1, 3, 2, 4],
```

```

        [5, 6, 7, 8],
        [9, 8, 7, 6],
        [4, 3, 2, 1]]]], dtype=torch.float)

maxpool = nn.MaxPool2d(kernel_size=2, stride=2)
avgpool = nn.AvgPool2d(kernel_size=2, stride=2)

print("Max Pooling:\n", maxpool(input))
print("Average Pooling:\n", avgpool(input))

```

6. Bài tập thực hành

Bài tập 1

Cho ma trận 4x4 sau:

$$\begin{bmatrix} 4 & 2 & 1 & 3 \\ 0 & 5 & 6 & 7 \\ 9 & 8 & 3 & 2 \\ 4 & 1 & 0 & 5 \end{bmatrix}$$

Hãy tính:

- Kết quả sau khi áp dụng Max Pooling với kernel 2x2, stride 2.
- Kết quả sau khi áp dụng Average Pooling với kernel 2x2, stride 2.

Bài tập 2

Cài đặt lại ví dụ trong PyTorch và thử thay đổi:

- Kernel size = 3
- Stride = 1
- Nhận xét kết quả đầu ra khi so sánh Max và Average Pooling.

7. Kết luận

Pooling là công cụ quan trọng trong CNN giúp giảm kích thước dữ liệu đầu vào và làm nổi bật các đặc trưng quan trọng. Hiểu rõ cách dùng Max và Average Pooling sẽ giúp bạn áp dụng hiệu quả hơn vào các bài toán thị giác máy tính trong cuộc thi AI.

8.3 Basic Image Classification

1. Phân loại ảnh là gì?

Phân loại ảnh (Image Classification) là quá trình gán nhãn cho một bức ảnh đầu vào dựa trên nội dung của ảnh đó. Ví dụ:

- Ảnh mèo → Nhãn: mèo
- Ảnh chó → Nhãn: chó

Đây là một trong những bài toán nền tảng nhất của lĩnh vực Thị giác máy tính (Computer Vision).

2. Luồng xử lý cơ bản

Một hệ thống phân loại ảnh gồm các bước chính:

1. **Tiền xử lý dữ liệu (Preprocessing)**: Chuyển ảnh về kích thước đồng nhất, chuẩn hóa giá trị pixel.
2. **Trích xuất đặc trưng (Feature Extraction)**: Dùng CNN để học đặc trưng từ ảnh.
3. **Phân loại (Classification)**: Dùng tầng Fully Connected (FC) + Soft-max để đưa ra kết quả cuối cùng.

3. Ví dụ đơn giản sử dụng PyTorch

Dưới đây là ví dụ phân loại ảnh chữ số tay MNIST với mạng CNN đơn giản.

Listing 8.3: Phân loại ảnh MNIST với CNN trong PyTorch

```
import torch
import torch.nn as nn
import torchvision
import torchvision.transforms as transforms
```

```
train_dataset = torchvision.datasets.MNIST(root='./data',
      train=True, transform=transforms.ToTensor(), download=True)
train_loader = torch.utils.data.DataLoader(train_dataset, batch_s

class SimpleCNN(nn.Module):
    def __init__(self):
        super(SimpleCNN, self).__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(1, 16, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2)
        )
        self.fc = nn.Linear(16 * 14 * 14, 10)

    def forward(self, x):
        x = self.conv(x)
        x = x.view(-1, 16 * 14 * 14)
        return self.fc(x)

model = SimpleCNN()
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

for epoch in range(1): # 1 epoch đơn ăgin
    for images, labels in train_loader:
        outputs = model(images)
        loss = criterion(outputs, labels)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

print ("ấHunậluyn_xong!")
```

4. Một số ứng dụng thực tế

- Nhận dạng chữ viết tay (ví dụ: số CMND, biển số xe)
- Phân loại động vật (mèo, chó, ngựa)
- Hệ thống giám sát an ninh

5. Các khái niệm quan trọng cần nhớ

- **Epoch:** Số lần toàn bộ dữ liệu được đưa qua mô hình.
- **Loss Function:** Hàm đánh giá sai số giữa đầu ra và nhãn thật.
- **Accuracy:** Tỷ lệ dự đoán đúng.
- **Overfitting:** Khi mô hình học quá kỹ dữ liệu huấn luyện và hoạt động kém trên dữ liệu mới.

6. Bài tập thực hành

Bài tập 1

Thay đổi mô hình CNN đơn giản ở trên bằng cách:

- Thêm một lớp convolution mới với 32 filters
- Dùng ReLU + MaxPooling sau lớp đó
- In ra kích thước tensor sau mỗi lớp

Bài tập 2

Thử thay tập dữ liệu MNIST bằng CIFAR-10:

- Tải CIFAR-10 bằng `torchvision.datasets.CIFAR10`
- Thay đổi số kênh đầu vào trong `Conv2d` (3 thay vì 1)
- Huấn luyện và tính độ chính xác

7. Kết luận

Phân loại ảnh là bước đầu tiên quan trọng để bạn làm quen với Computer Vision. Hiểu được luồng xử lý cơ bản, biết cách xây dựng một mô hình CNN đơn giản là nền tảng để bạn tham gia hiệu quả hơn trong các cuộc thi AI.

8.4 Object Detection Basics (YOLO, SSD)

1. Object Detection là gì?

Object Detection (Phát hiện đối tượng) là kỹ thuật giúp xác định vị trí và loại đối tượng trong ảnh bằng cách:

- Dự đoán tọa độ của các **hộp bao (bounding boxes)**
- Gán nhãn phân loại (class label) cho từng hộp

Ví dụ: Trong một ảnh chụp ngoài đường, mô hình có thể phát hiện “xe hơi”, “người đi bộ”, “xe đạp” cùng lúc và khoanh vùng chúng.

2. Mô hình YOLO và SSD là gì?

2.1 YOLO (You Only Look Once)

YOLO là mô hình phát hiện đối tượng rất nhanh, hoạt động như sau:

- Chia ảnh thành lưới (grid)
- Mỗi ô lưới dự đoán bounding box và class
- Toàn bộ ảnh được xử lý trong **một lần nhìn** – rất nhanh!

Ưu điểm: Nhanh, phù hợp cho ứng dụng thời gian thực (real-time).

Nhược điểm: Có thể bỏ sót các vật nhỏ trong ảnh.

2.2 SSD (Single Shot Multibox Detector)

SSD cũng là mô hình một bước (single-shot), hoạt động như sau:

- Trích xuất đặc trưng từ nhiều tầng CNN
- Dự đoán nhiều bounding box với các tỷ lệ khác nhau

Ưu điểm: Cân bằng tốt giữa tốc độ và độ chính xác.

Nhược điểm: Chậm hơn YOLO một chút, nhưng nhận dạng vật nhỏ tốt hơn.

3. So sánh YOLO và SSD

Tiêu chí	YOLO	SSD
Tốc độ	Rất nhanh	Nhanh
Chính xác	Trung bình–cao	Cao hơn YOLO
Vật nhỏ	Dễ bỏ sót	Tốt hơn
Dễ dùng	Rất phổ biến	Phổ biến

4. Cách sử dụng YOLOv5 (thực hành nhanh)

Listing 8.4: Thử nghiệm YOLOv5 với ảnh thực tế

```

import torch
from PIL import Image
import requests
from io import BytesIO

# Tải mô hình YOLOv5 online
model = torch.hub.load('ultralytics/yolov5', 'yolov5s', pretrained=True)

# Dùng ảnh online
url = 'https://ultralytics.com/images/zidane.jpg'
image = Image.open(BytesIO(requests.get(url).content))

# Phát hiện
results = model(image)

# Hiển thị kết quả
results.print()
results.show()

```

5. Thực hành SSD (dùng thư viện torchvision)

Listing 8.5: Thử nghiệm SSD với torchvision

```
import torchvision
import torchvision.transforms as T
from PIL import Image
import matplotlib.pyplot as plt

# Dùng mô hình SSD đã huấn luyện
model = torchvision.models.detection.ssd300_vgg16(pretrained=True)
model.eval()

# Ảnh và tên ử lý
img = Image.open('your_image.jpg')
transform = T.Compose([T.Resize((300, 300)), T.ToTensor()])
input_tensor = transform(img).unsqueeze(0)

# ử dụng đoán
with torch.no_grad():
    output = model(input_tensor)[0]

# In kết quả
print(output)
```

6. Bài tập thực hành

Bài tập 1: Sử dụng YOLOv5

- Chạy ví dụ YOLOv5 với ảnh có nhiều đối tượng.
- Thử thay ảnh khác (người, xe, vật nuôi).
- Đếm số đối tượng được phát hiện.

Bài tập 2: So sánh YOLO và SSD

- Viết đoạn mã so sánh thời gian chạy giữa YOLOv5 và SSD với cùng một ảnh.
- Ghi nhận sự khác biệt về kết quả phát hiện và tốc độ.

7. Kết luận

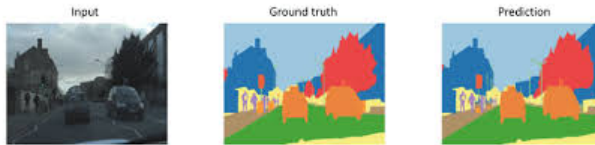
YOLO và SSD là hai trong số các kỹ thuật phát hiện đối tượng phổ biến và hiệu quả. Khi bạn hiểu được cách hoạt động và ứng dụng thực tế, bạn sẽ có thể áp dụng vào các bài toán AI trong thực tế, như giám sát an ninh, xe tự lái, hoặc nhận diện hình ảnh trong cuộc thi.

8.5 Image Segmentation Basics (U-Net)

1. Image Segmentation là gì?

Image Segmentation (Phân vùng ảnh) là kỹ thuật chia ảnh thành nhiều vùng có ý nghĩa. Mỗi điểm ảnh (pixel) sẽ được gán vào một lớp cụ thể.

- Mỗi pixel trong ảnh đầu ra tương ứng với một lớp (label): ví dụ “nền”, “vật thể”, “người”, “cây cối”.
- Đây là cấp độ chi tiết hơn so với Object Detection (chỉ có bounding box).



Ảnh gốc (trái) – Ground truth (giữa) – Kết quả U-Net (phải)

2. Mô hình U-Net là gì?

U-Net là một mạng nơ-ron tích chập (CNN) đặc biệt dùng trong Image Segmentation, ban đầu được thiết kế cho ảnh y tế.

Đặc điểm chính:

- Có kiến trúc hình chữ **U**: gồm phần thu gọn (encoding) và mở rộng (decoding).
- Giữ lại thông tin chi tiết qua kết nối skip connection.
- Dự đoán từng pixel trong ảnh.

3. Kiến trúc tổng quan của U-Net

- **Encoder (trái):** trích xuất đặc trưng bằng convolution + max pooling.
- **Decoder (phải):** phục hồi kích thước ảnh + phân lớp từng pixel.
- **Skip connections:** nối trực tiếp thông tin từ encoder sang decoder để tăng độ chính xác.

4. Ví dụ thực hành dùng PyTorch U-Net

Cài đặt U-Net đơn giản

Listing 8.6: Kiến trúc đơn giản của U-Net

```
import torch
import torch.nn as nn

class UNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.down1 = nn.Sequential(nn.Conv2d(1, 64, 3, padding=1)
        self.pool = nn.MaxPool2d(2)
        self.up = nn.ConvTranspose2d(64, 64, 2, stride=2)
        self.out = nn.Conv2d(64, 1, 1)

    def forward(self, x):
        x1 = self.down1(x)
        x2 = self.pool(x1)
        x3 = self.up(x2)
        out = self.out(x3)
        return out
```

Dữ liệu đầu vào và huấn luyện

Với bài toán segmentation, ta cần:

- Ảnh đầu vào (images)
- Mặt nạ đúng (ground truth mask): mỗi pixel có nhãn đúng

Dataset thường dùng: Oxford Pet Dataset, Cityscapes, COCO Stuff

5. Bài tập thực hành

Bài tập 1: Quan sát đầu ra của U-Net

- Tải một mô hình U-Net có sẵn (vd: từ PyTorch hoặc Hugging-Face)
- Dùng ảnh đơn giản (ví dụ ảnh đường, ảnh y tế) để thử nghiệm
- Hiển thị kết quả segmentation

Bài tập 2: Tự huấn luyện U-Net mini

- Dùng một tập ảnh nhỏ (10–20 ảnh) và mặt nạ để huấn luyện U-Net mini
- Tính độ chính xác bằng Dice coefficient hoặc IoU
- Vẽ ảnh đầu vào, ground truth và kết quả dự đoán

6. Ứng dụng của Segmentation

- Ảnh y tế: khoanh vùng mô tổn thương
- Xe tự lái: phân biệt đường, người, xe
- Nông nghiệp: phát hiện sâu bệnh, cây trồng
- Game và AR: tách người khỏi nền

7. Kết luận

Segmentation giúp nhận diện chi tiết từng phần trong ảnh – là bước tiến sâu hơn từ Image Classification và Object Detection. Với mô hình mạnh như U-Net, bạn có thể áp dụng cho nhiều bài toán thực tế trong cuộc thi AI hoặc sản phẩm thực tế.

8.6 Transfer Learning for Image Classification

1. Transfer Learning là gì?

Transfer Learning (học chuyển giao) là kỹ thuật sử dụng một mô hình đã được huấn luyện sẵn trên tập dữ liệu lớn (ví dụ ImageNet) để giải quyết một bài toán mới.

- Thay vì huấn luyện mô hình từ đầu, ta tận dụng các đặc trưng đã học được.
- Rất hiệu quả khi bạn có ít dữ liệu hoặc hạn chế về thời gian huấn luyện.

Ví dụ mô hình tiền huấn luyện phổ biến:

- **ResNet**: mạnh mẽ, phổ biến trong học sâu
- **MobileNet**: nhẹ, phù hợp thiết bị di động

2. Khi nào nên dùng Transfer Learning?

- Khi bạn có tập dữ liệu nhỏ (ít hơn vài ngàn ảnh)
- Khi bạn cần mô hình tốt mà không tốn nhiều thời gian huấn luyện
- Khi bạn không có GPU mạnh

3. Cách sử dụng Transfer Learning với PyTorch

Tải mô hình ResNet18 đã huấn luyện sẵn

Listing 8.7: Khởi tạo mô hình ResNet18

```
import torchvision.models as models

model = models.resnet18(pretrained=True)
```

Thay thế lớp phân loại cuối cùng

Giả sử bạn có 3 lớp mới: Cat, Dog, Bird

```
import torch.nn as nn

num_classes = 3
model.fc = nn.Linear(model.fc.in_features, num_classes)
```

Freeze các lớp trích đặc trưng

```
for param in model.parameters():
    param.requires_grad = False

# iCh áhùn ệluyn layer ócui
for param in model.fc.parameters():
    param.requires_grad = True
```

4. Ví dụ huấn luyện mô hình

Listing 8.8: Huấn luyện mô hình với DataLoader

```
import torch
from torch import optim

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.fc.parameters(), lr=0.001)

for epoch in range(10):
    for inputs, labels in dataloader:
        outputs = model(inputs)
        loss = criterion(outputs, labels)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```

5. MobileNet (dành cho thiết bị nhẹ)

```
mobilenet = models.mobilenet_v2(pretrained=True)
mobilenet.classifier[1] = nn.Linear(mobilenet.last_channel, n
```

Gợi ý: MobileNet phù hợp nếu bạn muốn triển khai trên điện thoại, máy nhúng, Raspberry Pi.

6. Bài tập thực hành

Bài tập 1: Huấn luyện ResNet18

- Tải bộ dữ liệu nhỏ (ví dụ: Oxford Pets hoặc Flowers102)
- Sử dụng ResNet18 làm mô hình nền
- Thay lớp phân loại và huấn luyện trên tập mới
- Đánh giá độ chính xác

Bài tập 2: So sánh ResNet và MobileNet

- Dùng cùng 1 tập dữ liệu cho cả hai mô hình
- So sánh: thời gian huấn luyện, độ chính xác, kích thước mô hình
- Trình bày kết quả dưới dạng bảng

7. Kết luận

Transfer Learning là một công cụ cực kỳ mạnh mẽ trong Computer Vision. Bạn có thể nhanh chóng xây dựng mô hình chính xác cao chỉ với vài dòng code. Việc sử dụng các mô hình như ResNet và MobileNet sẽ giúp bạn tiết kiệm thời gian huấn luyện, đặc biệt hữu ích khi tham gia các cuộc thi AI.

8.7 Image Augmentation Techniques

1. Image Augmentation là gì?

Image Augmentation là quá trình tạo ra nhiều biến thể khác nhau của ảnh gốc bằng cách áp dụng các kỹ thuật như xoay ảnh, lật ảnh, thay đổi độ sáng, cắt ảnh, v.v.

Mục tiêu: giúp mô hình học được các đặc trưng tổng quát hơn và tránh overfitting khi tập dữ liệu huấn luyện nhỏ.

2. Lợi ích của Image Augmentation

- Mở rộng tập dữ liệu mà không cần thu thập thêm ảnh mới
- Giúp mô hình học được nhiều tình huống khác nhau
- Giảm hiện tượng overfitting

3. Các kỹ thuật phổ biến

- Horizontal Flip (Lật ngang)
- Rotation (Xoay ảnh)
- Random Crop (Cắt ngẫu nhiên)
- Color Jitter (Thay đổi màu sắc)
- Gaussian Blur (Làm mờ)
- Random Erasing (Xóa ngẫu nhiên)

4. Cách sử dụng trong PyTorch

Ví dụ dùng `torchvision.transforms`

```
from torchvision import transforms
```

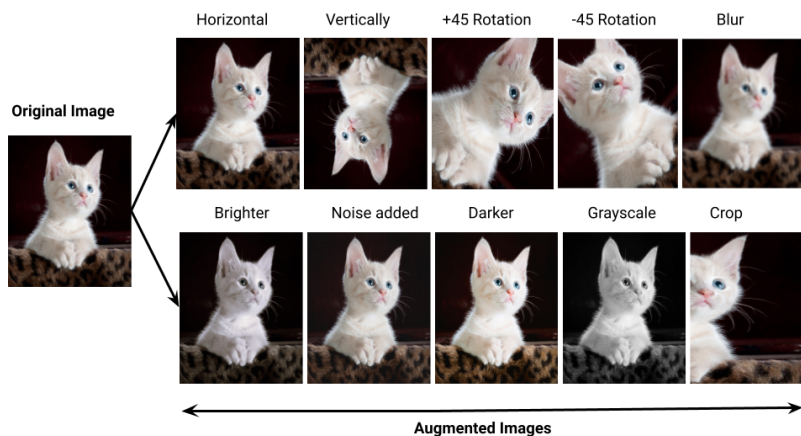
```
transform = transforms.Compose([
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomRotation(degrees=15),
    transforms.ColorJitter(brightness=0.2, contrast=0.2),
    transforms.ToTensor()
])
```

Sử dụng với Dataset

```
from torchvision.datasets import ImageFolder
```

```
dataset = ImageFolder(root='data/train', transform=transform)
```

5. So sánh ảnh trước và sau khi Augmentation



Ảnh gốc và các phiên bản được tăng cường (xoay, lật, điều chỉnh độ sáng).

6. Bài tập thực hành

Bài tập 1: Thử nghiệm các kỹ thuật tăng cường

- Dùng tập dữ liệu CIFAR-10 hoặc Oxford Flowers
- Áp dụng các kỹ thuật: Flip, Rotation, ColorJitter
- In ảnh gốc và ảnh sau khi transform để so sánh

Bài tập 2: So sánh độ chính xác

- Huấn luyện mô hình phân loại đơn giản (ResNet18)
- Một lần **không dùng augmentation**, một lần **dùng augmentation**
- So sánh độ chính xác sau khi huấn luyện

7. Kết luận

Image Augmentation là một bước cực kỳ quan trọng giúp mô hình Computer Vision học tốt hơn, đặc biệt là khi bạn không có nhiều dữ liệu. Việc làm chủ các kỹ thuật augmentation giúp bạn tạo ra mô hình mạnh mẽ, ổn định và dễ dàng giành lợi thế trong các cuộc thi AI.

8.8 Feature Extraction Using Pre-Trained Models

1. Feature Extraction là gì?

Feature Extraction (Trích xuất đặc trưng) là quá trình lấy ra các đặc điểm quan trọng từ ảnh (như hình dạng, cạnh, họa tiết...) để phục vụ cho việc phân loại hoặc xử lý hình ảnh.

Transfer Learning giúp tận dụng các mô hình đã huấn luyện sẵn như ResNet, VGG, MobileNet... để lấy đặc trưng mà không cần huấn luyện lại toàn bộ mạng.

2. Tại sao dùng Pre-Trained Models?

- Tiết kiệm thời gian huấn luyện
- Hoạt động tốt cả với dữ liệu nhỏ
- Tận dụng được các đặc trưng mạnh mẽ đã học từ bộ dữ liệu lớn như ImageNet

3. Cách trích xuất đặc trưng với PyTorch

Bước 1: Import mô hình huấn luyện sẵn

```
from torchvision import models

resnet = models.resnet18(pretrained=True)
```

Bước 2: Xóa lớp phân loại cuối cùng

```
# xóa lớp fully-connected cuối
feature_extractor = torch.nn.Sequential(*list(resnet.children())[:-1])
feature_extractor.eval()
```

Bước 3: Trích xuất đặc trưng từ ảnh đầu vào

```

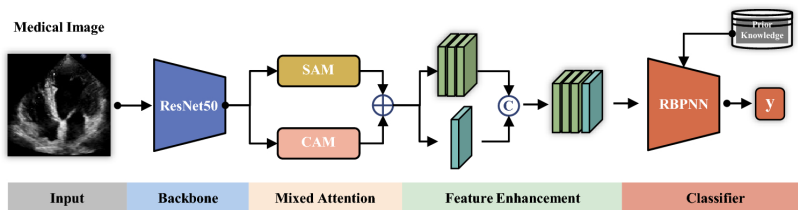
from PIL import Image
from torchvision import transforms
import torch

img = Image.open("sample.jpg")
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
])
input_tensor = transform(img).unsqueeze(0)

with torch.no_grad():
    features = feature_extractor(input_tensor)
    print(features.shape)  # Output: [1, 512, 1, 1]

```

4. Hình minh họa quy trình



Minh họa: Dùng ResNet để trích xuất đặc trưng, sau đó đưa vào mô hình phân loại riêng.

5. Ứng dụng của Feature Extraction

- Phân loại ảnh (Image Classification)
- Phát hiện đối tượng (Object Detection)

- Nhận dạng khuôn mặt (Face Recognition)
- Tìm kiếm ảnh tương tự

6. Bài tập thực hành

Bài tập 1: Trích xuất đặc trưng từ ảnh hoa

- Tải mô hình `MobileNetV2` đã huấn luyện trên ImageNet
- Dùng ảnh hoa từ Oxford Flowers Dataset
- Trích xuất đặc trưng từ mỗi ảnh, lưu thành vector

Bài tập 2: Dùng đặc trưng để phân loại

- Dùng `sklearn.svm.SVC` hoặc `LogisticRegression`
- Train mô hình từ đặc trưng đã trích xuất
- So sánh độ chính xác với mô hình train trực tiếp từ ảnh

7. Kết luận

Trích xuất đặc trưng từ mô hình đã huấn luyện sẵn là cách hiệu quả để áp dụng Deep Learning mà không cần nhiều dữ liệu hay thời gian huấn luyện. Kỹ thuật này đặc biệt hữu ích cho người mới bắt đầu trong các cuộc thi AI.

8.9 Introduction to GANs (Generating Images)

1. GAN là gì?

GAN (Generative Adversarial Network) là một loại mạng học sâu được sử dụng để tạo ra dữ liệu mới giống với dữ liệu huấn luyện. Mạng GAN bao gồm hai thành phần chính:

- **Generator (G)**: Tạo ảnh giả từ nhiễu ngẫu nhiên.
- **Discriminator (D)**: Phân biệt ảnh thật và ảnh giả.

Chúng cạnh tranh nhau trong một trò chơi đối kháng: Generator cố gắng "lừa" Discriminator, trong khi Discriminator cố gắng nhận diện ảnh giả.

2. Ứng dụng của GANs

- Tạo khuôn mặt người không có thật
- Nâng cao độ phân giải ảnh (Super-Resolution)
- Chuyển phong cách (Style Transfer)
- Tạo dữ liệu huấn luyện nhân tạo

3. Mô hình GAN đơn giản với PyTorch

Cấu trúc cơ bản

Generator:

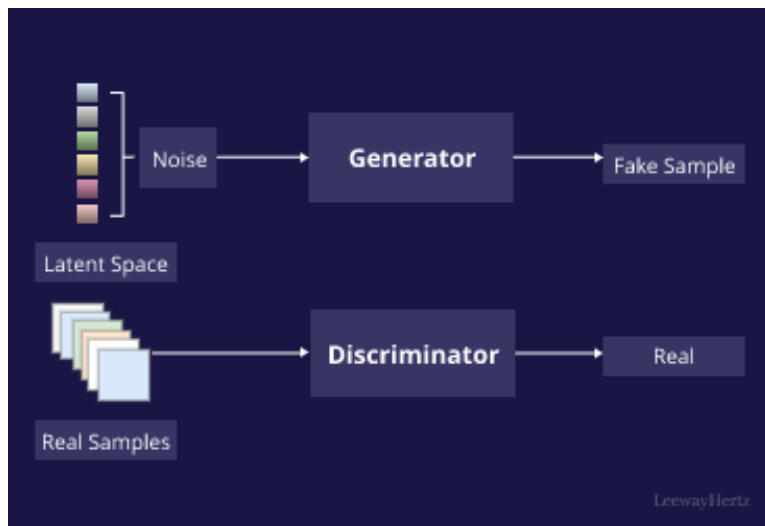
```
class Generator(nn.Module):
    def __init__(self):
        super().__init__()
        self.main = nn.Sequential(
            nn.Linear(100, 256),
            nn.ReLU(),
```

```
        nn.Linear(256, 784),
        nn.Tanh()
    )
    def forward(self, z):
        return self.main(z)
```

Discriminator:

```
class Discriminator(nn.Module):
    def __init__(self):
        super().__init__()
        self.main = nn.Sequential(
            nn.Linear(784, 256),
            nn.LeakyReLU(0.2),
            nn.Linear(256, 1),
            nn.Sigmoid()
        )
    def forward(self, x):
        return self.main(x)
```


4. Hình minh họa kiến trúc GAN



Minh họa: GAN gồm Generator và Discriminator đấu với nhau để cải thiện kết quả tạo ảnh.

5. Quá trình huấn luyện

1. Sinh nhiễu ngẫu nhiên $z \sim \mathcal{N}(0, 1)$
2. Tạo ảnh giả $G(z)$
3. Phân biệt ảnh thật và giả bằng $D(x), D(G(z))$
4. Tính loss cho cả G và D
5. Cập nhật tham số qua Gradient Descent

6. Bài tập thực hành

Bài tập 1: Tạo ảnh tay viết số với GAN

- Tải bộ dữ liệu MNIST (ảnh chữ số viết tay)
- Xây dựng GAN theo cấu trúc đơn giản như trên
- Sau khi huấn luyện 10 epochs, sinh 10 ảnh giả và hiển thị

Bài tập 2: Vẽ biểu đồ Loss

- Ghi lại loss của Generator và Discriminator mỗi epoch
- Vẽ biểu đồ bằng `matplotlib`
- Phân tích xem loss có ổn định không

7. Kết luận

GANs là công cụ mạnh mẽ để tạo ảnh mới và rất hữu ích trong các ứng dụng AI sáng tạo. Hiểu cơ bản về cách hoạt động của GAN sẽ giúp bạn áp dụng vào nhiều dự án Computer Vision trong cuộc thi AI.

8.10 Introduction to Self-Supervised Learning for Vision

1. Self-Supervised Learning là gì?

Self-Supervised Learning (SSL) là phương pháp học máy không cần nhãn (label) rõ ràng. Mô hình học từ chính dữ liệu đầu vào bằng cách tự tạo ra các nhãn (pseudo-label) để huấn luyện.

Thay vì yêu cầu con người gán nhãn, SSL tự định nghĩa các bài toán như:

- Dự đoán phần bị che trong ảnh
- Xác định xem hai ảnh có giống nhau không (augmentation)
- Dự đoán thứ tự các miếng ảnh

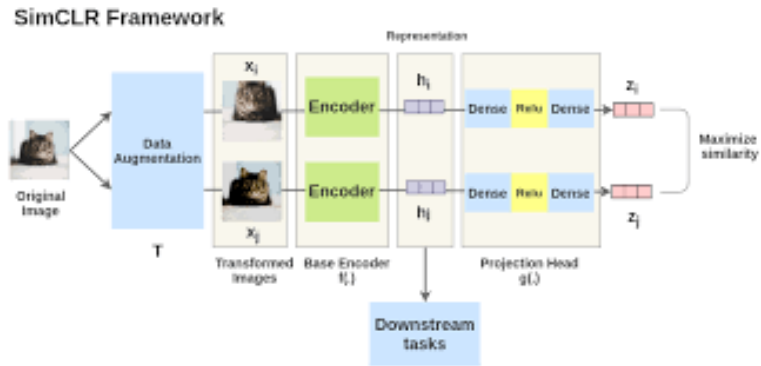
2. Ứng dụng của SSL trong Thị giác máy tính

- Tiết kiệm công sức gán nhãn cho dữ liệu
- Huấn luyện mô hình tiền huấn luyện (pre-trained) trên dữ liệu không gán nhãn
- Tăng độ chính xác trong các bài toán phân loại, phát hiện vật thể

3. Ví dụ về kỹ thuật SSL: SimCLR

SimCLR là phương pháp học biểu diễn ảnh bằng contrastive learning:

1. Áp dụng nhiều augmentation khác nhau lên một ảnh
2. Encode các ảnh thành vector nhúng
3. Dùng loss để đưa vector của cùng một ảnh gần nhau, còn khác ảnh thì xa nhau



Minh họa: Ảnh được biến đổi (augmentation) và đưa vào encoder để học biểu diễn.

4. Code Mô Phỏng SSL đơn giản với PyTorch

```
# Data augmentation cho SSL
transform = transforms.Compose([
    transforms.RandomResizedCrop(224),
    transforms.RandomHorizontalFlip(),
    transforms.ColorJitter(0.4, 0.4, 0.4, 0.1),
    transforms.RandomGrayscale(p=0.2),
    transforms.ToTensor(),
])

encoder = torchvision.models.resnet18(pretrained=False)
encoder.fc = nn.Identity()
```

5. Bài tập thực hành

Bài tập 1: So sánh biểu diễn ảnh

- Lấy một ảnh và tạo hai bản sao bằng hai augmentation khác nhau
- Encode cả hai bằng ResNet18
- Tính cosine similarity giữa hai vector
- Trả lời: Các vector có gần nhau không?

Bài tập 2: Tập huấn đơn giản với contrastive loss

- Lấy tập ảnh CIFAR-10 (không dùng nhãn)
- Huấn luyện mô hình học biểu diễn
- Vẽ loss và biểu đồ biểu diễn bằng PCA hoặc t-SNE

6. Kết luận

Self-Supervised Learning là kỹ thuật học sâu hiệu quả khi không có nhiều dữ liệu gán nhãn. Các mô hình như SimCLR, BYOL, DINO đã được áp dụng thành công trong nhiều lĩnh vực thị giác máy tính.

8.11 Vision Transformers (ViT) Basics

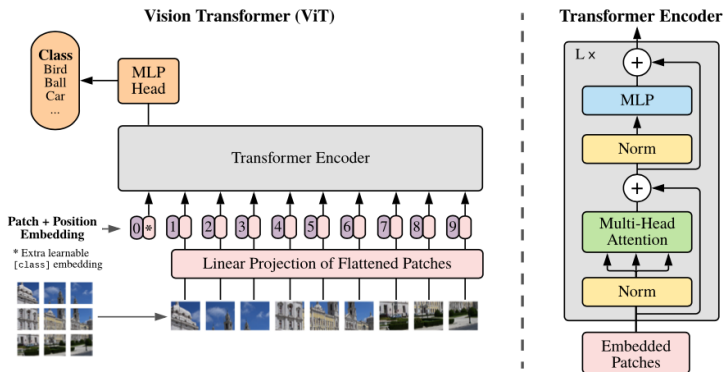
1. Vision Transformers (ViT) là gì?

Vision Transformer (ViT) là một mô hình học sâu áp dụng kiến trúc **Transformer**, vốn thành công trong xử lý ngôn ngữ tự nhiên (NLP), vào bài toán thị giác máy tính như phân loại ảnh.

Ý tưởng chính: biến ảnh thành các “miếng ảnh nhỏ” (patches), giống như chuỗi từ trong văn bản, rồi đưa chuỗi này vào Transformer.

- Mỗi ảnh được chia thành nhiều **patches nhỏ** (vd: 16x16 pixels).
- Các patches này được biến đổi thành vector (embedding).
- Dàn transformer attention sẽ học mối liên hệ giữa các phần của ảnh.

2. Kiến trúc tổng quan của ViT



Ảnh minh họa: Một ảnh được chia nhỏ thành các patch, rồi đưa qua Transformer để học đặc trưng.

3. Ưu điểm của ViT

- Không phụ thuộc vào CNN truyền thống (không cần convolution).
- Có khả năng học mối quan hệ xa giữa các vùng ảnh.
- Kết quả tốt trong các bài toán phân loại ảnh lớn như ImageNet.

4. Cách sử dụng Vision Transformer trong PyTorch

ViT đã được tích hợp trong thư viện `timm` (PyTorch Image Models).

```
import timm
import torch
from torchvision import transforms
from PIL import Image

# Tải model ViT từ Internet
model = timm.create_model('vit_base_patch16_224', pretrained=True)
model.eval()

# Định nghĩa biến đổi ảnh
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.5]*3, std=[0.5]*3)
])

img = Image.open('example.jpg').convert('RGB')
input_tensor = transform(img).unsqueeze(0) # thêm batch dimension

# Dự đoán
with torch.no_grad():
    output = model(input_tensor)
    predicted = torch.argmax(output, dim=1)
print('Dự đoán là: ', predicted.item())
```

5. Bài tập thực hành

Bài tập 1: Dự đoán ảnh với ViT

- Tải một ảnh bất kỳ từ internet
- Dùng mô hình ViT tiền huấn luyện để phân loại ảnh đó
- So sánh kết quả dự đoán với nội dung thực tế của ảnh

Bài tập 2: So sánh CNN và ViT

- Dự đoán ảnh với ResNet và với ViT
- So sánh tốc độ, độ chính xác và độ ổn định
- Viết một đoạn code để đo thời gian dự đoán

6. Tổng kết

Vision Transformers là bước tiến lớn trong thị giác máy tính, mở ra khả năng học sâu dựa hoàn toàn vào attention. Với sức mạnh từ mô hình tiền huấn luyện và thư viện như `timm`, sinh viên có thể dễ dàng áp dụng ViT vào các cuộc thi AI thực tế.

8.12 CLIP and Multimodal Learning

1. CLIP là gì?

CLIP (Contrastive Language–Image Pretraining) là mô hình do OpenAI phát triển để kết nối hình ảnh và ngôn ngữ. Nó có khả năng hiểu hình ảnh và mô tả văn bản cùng lúc bằng cách học biểu diễn đa phương thức.

Ý tưởng chính:

- CLIP học cách **ghép nối đúng** giữa ảnh và mô tả của ảnh.
- Ví dụ: Nếu ảnh là “một con mèo” thì mô tả “A cat sitting on a sofa” sẽ có điểm tương đồng cao.

2. Học Đa Phương Thức (Multimodal Learning)

Multimodal Learning là phương pháp học từ nhiều loại dữ liệu khác nhau (ví dụ: hình ảnh và văn bản). CLIP là một ví dụ nổi bật về điều này.

- Mỗi loại dữ liệu có một encoder riêng: **Image Encoder** và **Text Encoder**.
- Cả hai được đưa vào cùng một không gian vector để so sánh (embedding space).

3. Cách sử dụng CLIP trong Python

Bạn có thể sử dụng thư viện `openai/CLIP` hoặc `transformers` để dự đoán văn bản tương ứng với ảnh.

```
import torch
import clip
from PIL import Image
```

```
device = "cuda" if torch.cuda.is_available() else "cpu"
model, preprocess = clip.load("ViT-B/32", device=device)
```

```
image = preprocess(Image.open("cat.jpg")).unsqueeze(0).to(device)
text = clip.tokenize(["A photo of a cat", "A photo of a dog"])

with torch.no_grad():
    image_features = model.encode_image(image)
    text_features = model.encode_text(text)

    logits_per_image, logits_per_text = model(image, text)
    probs = logits_per_image.softmax(dim=-1).cpu().numpy()

print("Xác suất dự đoán:", probs)
```

4. Ứng dụng thực tế

- Tìm kiếm ảnh bằng văn bản (text-to-image search)
- Lọc nội dung ảnh theo mô tả
- Tạo hệ thống gợi ý tự động dựa trên mô tả người dùng

5. Bài tập thực hành

Bài tập 1: Dự đoán mô tả ảnh

- Tải một ảnh bất kỳ từ internet (vd: ảnh con chó).
- Dùng CLIP để dự đoán ảnh này giống mô tả nào hơn: "A photo of a dog" hay "A photo of a cat"?

Bài tập 2: Tìm ảnh đúng từ mô tả

- Chuẩn bị 3 ảnh khác nhau: mèo, xe đạp, laptop.
- Nhập mô tả: "A photo of a laptop".
- Sử dụng CLIP để tìm ảnh phù hợp nhất với mô tả.

6. Tổng kết

CLIP giúp máy tính hiểu và kết nối giữa hình ảnh và văn bản. Đây là một công cụ mạnh mẽ trong thị giác máy tính hiện đại, đặc biệt hữu ích trong các cuộc thi AI yêu cầu xử lý dữ liệu đa phương thức.

8.13 Generative Models

1. Generative Models là gì?

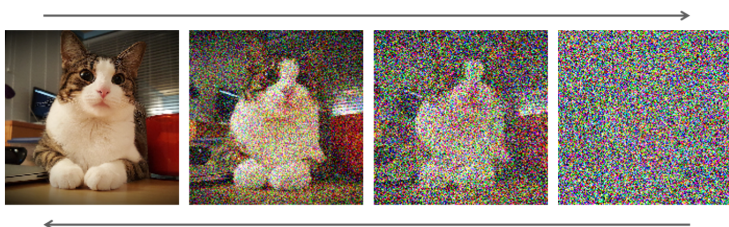
Generative Models là các mô hình AI có khả năng **tạo ra dữ liệu mới** như hình ảnh, âm thanh, hoặc văn bản. Trong lĩnh vực thị giác máy tính (Computer Vision), chúng thường dùng để sinh ra hình ảnh từ mô tả văn bản.

Ví dụ:

- Nhập: “A cat wearing sunglasses on the beach”
- Kết quả: Một bức tranh được AI tạo ra thể hiện chính xác mô tả đó.

2. Stable Diffusion và DALL · E là gì?

- **DALL · E:** Mô hình của OpenAI, có khả năng sinh ảnh từ văn bản cực kỳ sáng tạo và phong phú.
- **Stable Diffusion:** Mô hình mã nguồn mở, phổ biến nhờ khả năng tùy chỉnh và tốc độ xử lý nhanh.



3. Nguyên lý hoạt động (giản lược)

- Văn bản được chuyển thành vector (embedding).
- Mô hình học cách tạo ảnh sao cho phù hợp với mô tả.
- Với Stable Diffusion, ảnh được sinh ra từ **hiều ngẫu nhiên** (noise) và dần dần “gỡ nhiễu” để tạo thành ảnh hoàn chỉnh.

4. Cách sử dụng Generative Models (Python)

Dùng thư viện diffusers từ Hugging Face

```
from diffusers import StableDiffusionPipeline
import torch

pipe = StableDiffusionPipeline.from_pretrained(
    "CompVis/stable-diffusion-v1-4",
    torch_dtype=torch.float16
).to("cuda")

prompt = "A futuristic cityscape at sunset"
image = pipe(prompt).images[0]
image.save("generated_image.png")
```

5. Ứng dụng trong thực tế

- Tạo ảnh minh họa cho sách/truyện/tranh
- Thiết kế ý tưởng nhanh (concept art)
- Phát triển game và phim hoạt hình

6. Bài tập thực hành

Bài tập 1: Sáng tạo hình ảnh từ mô tả

Hãy viết mô tả (prompt) cho 3 chủ đề:

- Một con rồng đang bay trong bầu trời đêm
- Một robot đang chơi đàn piano
- Một ngôi làng cổ tích giữa rừng

Sau đó dùng mô hình Stable Diffusion để tạo hình ảnh tương ứng.

Bài tập 2: So sánh mô hình

Thử nghiệm tạo cùng một ảnh bằng DALL·E và Stable Diffusion với prompt giống nhau. Hãy nhận xét:

- Hình ảnh nào chân thực hơn?
- Hình ảnh nào sáng tạo hơn?

7. Tổng kết

Generative Models như DALL·E và Stable Diffusion giúp máy tính không chỉ nhận biết hình ảnh mà còn có thể **sáng tạo ra hình ảnh mới**. Đây là một công cụ rất mạnh trong các cuộc thi AI và sáng tạo nội dung.